

**Informatics Institute of Technology In collaboration
with the University of Westminster**

Software Development Group Project (5COSC021C)

Module Leader: Mr. Banuka Athuraliya

Group Number : CS 70

Group Supervisor : Mr.Ahtshayan Udayasanthiran

Group Members :

Member	IIT ID
Akila Wijerama (Leader)	20241303
Dulan Nimnaka	20240503
Shaveen Peiris	20240515
Sanidu Samrasinghe	20240641
Sanara Perera	20240773
Hasandi Peiris	20240642

I. Declaration

This is to confirm that this project implementation report and all related materials are our original works. This work has not previously been accepted in substance for any degree and is not being submitted in whole or part for any other degree or qualification.

Student Full Name: Akila Mavindu Wijerama IIT Id: 20241303 UOW Id: w2121346	
Student Full Name: E. A. Dulan Nimnaka IIT Id: 20240503 UOW Id: w2119876	
Student Full Name: Shaveen Udana Peiris IIT Id: 20240515 UOW Id: 20240515	
Student Full Name: S.S.N.Samarasinghe IIT Id: 20240641 UOW Id: w2120064	
Student Full Name: I.A.S.N. Perera IIT Id: 20240773 UOW Id: w2120118	
Student Full Name: P.Hasandi Vihanga Peiris IIT Id: 20240642 UOW Id: w2120065	

Date : 23rd of March 2026

II. Abstract

AutoInsight aims to transform the Sri Lankan automotive sector by providing data-driven insights through analytics and machine learning, and it will achieve this goal thanks to the abundance of data available in the marketplace. This solution provides real-time prices for any car on the road based upon the metrics presented by multiple local marketplaces such as make, model, year and mileage. It presents the market price of cars based on an analysis of multiple factors.

This tool provides an easy solution for usage by all users, both buyers and sellers, to help them make informed decisions when buying or selling a vehicle, as well as helping eliminate price variance or manipulation. AutoInsight has additional tools such as market analysis visuals, high-demand vehicle tracking for ancillary businesses such as spare part dealerships and transparent pricing range, creating visibility into the current economic state of the automotive marketplace.

AutoInsight is likely going to cause the automotive industry in Sri Lanka to change for the better and create a more transparent, fair and accessible marketplace to average citizens in the future.

III. Acknowledgement

Our deepest gratitude goes to everyone who contributed to the success of this project. A special acknowledgement is due to our Module Leader, Mr. Banuka Athuraliya, for the guidance, encouragement and support he has provided during the development of this project. His insights and feedback were instrumental in shaping the final outcome of this project.

We would also like to thank our Project Supervisor, Mr. Ahtshayan Udayasanthiran, for providing expert advice, support and dedicated supervision. His guidance was critical to the success of this project.

We wish to thank Mr. Roshan Casiechetty, General Manager of the Ceylon Motor Traders Association (CMTA), Mr. Kusal Arthanayake, Founder & CEO of Riyasewana, and Mr. Shaif Mohomed, CEO of Ikman, for providing us with relevant datasets and supporting our research. Their contributions improved the quality and relevance of our work.

Lastly, we would like to acknowledge our peers, friends and institutions that had a part in supporting us either directly or indirectly during the course of this project. Your encouragement, co-operation and teamwork were crucial to the success of this project and will always be appreciated.

IV. Table of Content

I. Declaration.....	2
II. Abstract.....	3
III. Acknowledgement.....	4
IV. Table of Content.....	5
V. List of Figures.....	9
VI. List of Tables.....	11
VII. Abbreviations.....	12
Chapter 1: Implementation.....	14
1.1 Chapter Overview.....	14
1.2 Overview of the prototype.....	15
1.3 Technology selections.....	16
1.4 Implementation of the backend component.....	23
1.4.1 Backend Architecture.....	23
1.4.3 Data Preprocessing.....	37
1.4.4 Database Design.....	55
1.5 Implementation of the front end component.....	56
1.5.1 Page Structure and Navigation.....	56
1.5.2 Component Architecture.....	57
1.5.3 Data Visualisation.....	57
1.5.4 Responsive Design.....	59
1.6 Implementation of the data science component or hardware component (if applicable)..	59
1.6.1 Machine Learning Model.....	60
1.6.2 Trend Analysis and Prediction.....	66
1.7 GIT Repository.....	67
1.8 Deployments/CI-CD Pipeline.....	69
1.8.1 Front-End Deployment.....	70
1.8.2 Backend Deployment(Azure Cloud Virtual Machine).....	70
1.8.3 Static Pages (GitHub Pages).....	71
1.8.4 Practices related to Continuous Integration.....	71
Frequent Code Commits.....	73
Branching Strategy.....	74
1.8.5 Domain and DNS Management.....	77
1.9 CRUD operations.....	78
1.9.1 Akila Wijerama (20241303).....	78
Create.....	78
1.9.2 Dulan Nimnaka (20240503).....	80

1.9.3 Shaveen Peiris (20240515).....	82
1.9.4 Sanidu Samarasinghe (20240641).....	82
1.9.5 Sanara Perera (20240773).....	82
1.9.6 Hasandi Peiris (20240642).....	82
1.10 Chapter Summary.....	82
Chapter 2: Testing.....	83
2.1 Chapter Introduction.....	83
2.2 Testing Criteria.....	84
2.2.1 Test Strategy.....	84
2.2.2 Test Environment.....	85
2.3 Testing functional requirements.....	86
2.4 Testing non-functional requirements.....	90
2.4.1 Performance.....	90
2.4.2 Scalability.....	91
2.4.3 Reliability.....	92
2.4.4 Security.....	92
2.4.5 Availability.....	94
2.5 Unit testing.....	94
ML_Model\ML_Model_Test_PYTEST\test_model.py.....	95
ML_Model\ML_Model_Test_PYTEST\test_model_2.py.....	97
2.5.2 Backend Unit Testing (pytest).....	101
2.5.3 Frontend Unit Testing (Jest + React Testing Library).....	102
2.6 Performance testing.....	107
2.6.1 Page Load Time Testing.....	107
2.6.2 API Response Time Testing (Postman).....	107
2.6.3 Data Science Pipeline Performance.....	108
2.7 Usability testing.....	109
2.7.1 Test Participant Profiles.....	110
2.7.2 Assigned Tasks.....	110
2.7.3 Task Results and Feedback.....	111
2.7.4 UI Quality Evaluation.....	112
2.8 Compatibility testing.....	114
2.8.1 Browser Compatibility Testing.....	114
2.8.2 Responsive/Mobile Compatibility Testing.....	115
2.9 Chapter Summary.....	120
3.1 Chapter Overview.....	121
3.2 Evaluation methods.....	122
3.2.1 Quantitative Evaluation Methods.....	122
3.2.2 Qualitative Evaluation Methods.....	123

3.3 Quantitative evaluation.....	124
3.3.1 System Performance Metrics.....	124
3.3.2 Machine Learning Model Performance.....	125
3.4 Qualitative evaluation.....	126
3.4.1 Thematic Analysis.....	126
3.5 Self evaluation.....	130
3.5.1 Akila Wijerama (20241303) - Project Lead / Full-Stack / MLOps / Web Scraping.....	130
3.5.2 Dulan Nimnaka (20240503) - Frontend and Backend / Data Engineering & Deployment (Domain/VM).....	131
3.5.3 Shaveen Peiris (20240515) - Frontend Development.....	132
3.5.4 Sanidu Samarasinghe (20240641) -Design and Performance of the Machine Learning Pipeline.....	133
3.5.5 Sanara Perera (20240773) — Backend API, Data Cleaning & ML Testing.....	133
3.5.6 Hasandi Peiris (20240642) — Frontend-Backend Integration & Search Filtering.....	134
3.6 Chapter Summary.....	135
Chapter 4: Conclusion.....	137
4.1 Chapter Overview.....	137
4.2 Achievements of aims and objectives.....	137
4.3 Deviations.....	138
4.4 Limitations of the project.....	139
4.5 Future enhancements.....	140
4.6 Extra work (Competitions, research papers, etc).....	141
4.7 Concluding remarks.....	141
References.....	142
Appendix.....	144

V. List of Figures

Figure 1: Project Folder Structure.....	11
Figure 2: Flask-CORS server file.....	12
Figure 3: GET /api/health — Flask app file.....	12
Figure 4: GET /api/facets code block.....	12
Figure 5: beneath the GET /api/listings/path:listing_id code block.....	13
Figure 6: PUT /api/favorites.....	13
Figure 7: POST /api/compare.....	13
Figure 8: POST /api/admin/refresh.....	13
Figure 9: GET /api/og-image.....	14
Figure 10: BeautifulSoup scraping script.....	15
Figure 11: Selenium scraping script.....	20
Figure 12: Rate limiting/delay logic in the scraping script.....	21
Figure 13: Main Throttle Loop.....	21
Figure 14: Sample CSV file output after scraping.....	22
Figure 15: Sample dataset record of a scraped vehicle listing.....	22
Figure 16: Data preprocessing.....	23
Figure 17: Data cleaning script.....	24
Figure 18: Outlier detection logic.....	24
Figure 19: Database insertion code.....	40
Figure 20: Database Design.....	41
Figure 21: Search page.....	44
Figure 22: AutoInsight website mobile responsive.....	45
Figure 23: Model Comparison.....	48
Figure 24: Residual Analysis.....	49
Figure 25: Overfitting / Underfitting Check.....	50
Figure 26: Vehicle classified according to condition.....	50
Figure 27: Error rate per model.....	51
Figure 29: AutoInsight github repository overview.....	54
Figure 30: Active issue tracking and bug management.....	54
Figure 31: Backend CI/CD pipeline.....	55
Figure 32: Frontend production deployments history.....	55
Figure 33: Frontend Deployment.....	56
Figure 34: Azure Cloud.....	57
Figure 35: Azure Cloud Virtual Machine.....	58
Figure 36: Branch Structure.....	59
Figure 37: Commits.....	61
Figure 38: Contributors.....	61

Figure 39: Contributors.....	64
Figure 40: Domain and DNS Management.....	65
Figure 41: Save favorites.....	66
Figure 42: List listings.....	66
Figure 43: Repository update method.....	66
Figure 44:.....	67
Figure 45: Web Scraping from Riyasewana.....	67
Figure 46: Get favourites.....	68
Figure 47: listing.....	68
Figure 48: save favourites.....	68
Figure 49: replace all.....	69
Figure 50:.....	69
Figure 51: get many.....	70
Figure 52: get facets.....	70
Figure 53: Save favourites.....	70
Figure 54: Security.....	82
Figure 55:.....	86
Figure 56:.....	89
Figure 57: Classification report.....	90
Figure 58: Make filter test	
Figure 59:Model filter test	
Figure 60: Mileage filter test.....	92
Figure 61: Price filter test	
Figure 62:District filter test	
Figure 63: Year filter test.....	93
Figure 64: Brand New condition filter test.....	94
Figure 65: Used Condition filter test.....	94
Figure 66: Recondition condition filter test.....	94
Figure 67: Home page Performance.....	95
Figure 68: Filter Page Performance.....	96
Figure 69: Compare page performance.....	97
Figure 70: Contact Page Performance.....	97
Figure 71: Layout Shift Test.....	99
Figure 72: Interaction time testing.....	100
Figure 73: Recondition condition filter test.....	109
Figure 74: Recondition condition filter test.....	110
Figure 75: Recondition condition filter test.....	111
Figure 76: Recondition condition filter test.....	112
Figure 77:.....	112

VI. List of Tables

Table 1 : Technology Selections of AutoInsight.....	18
Table 2 : Branching Strategy of AutoInsight.....	71
Table 3 : Test Environments of AutoInsight.....	81
Table 4 : Test results for all functional requirements.....	84
Table 5 : Page Load Time Performance Results.....	85
Table 6 : API Response Time Under Increasing Dataset Load.....	86
Table 7: Backend Unit Test Results — pytest.....	97
Table 8 : API Endpoint Response Time Test Results - Postman.....	102
Table 9 : Data Science Pipeline Performance Metrics.....	102
Table 10 :Usability Test Participant Profiles.....	104
Table 11 : Usability Test Assigned Tasks and Success Criteria.....	104
Table 12 : Usability Task Completion Results and Participant Feedback.....	105
Table 13 : UI Quality Evaluation Results.....	107
Table 14 : System Performance Evaluation Results.....	117
Table 15 : Machine Learning Model Comparison — LightGBM vs Baseline Models.....	118
Table 16 : Thematic Analysis of Qualitative Evaluation Data.....	121

VII. Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
CD	Continuous Deployment
CDN	Content Delivery Network
CI	Continuous Integration
CMTA	Ceylon Motor Traders Association
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSP	Content Security Policy
CSS	Cascading Style Sheets
CSV	Comma-Separated Values
DOM	Document Object Model
FCP	First Contentful Paint
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IQR	Interquartile Range
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
LCP	Largest Contentful Paint
LKR	Sri Lankan Rupee
MAE	Mean Absolute Error
ML	Machine Learning
MSE	Mean Squared Error

MVP	Minimum Viable Product
REST	Representational State Transfer
RMSE	Root Mean Squared Error
SEO	Search Engine Optimisation
SPA	Single Page Application
SQL	Structured Query Language
SSR	Server-Side Rendering
UI	User Interface
URI	Uniform Resource Identifier
URL	Uniform Resource Locator
UX	User Experience
VM	Virtual Machine
XSS	Cross-Site Scripting

Chapter 1: Implementation

1.1 Chapter Overview

This chapter presents the full documentation for the implementation of AutoInsight, an intelligent analytics platform based on data from the vehicle market in Sri Lanka, which was designed specifically with the Sri Lankan automotive industry in mind. However, this chapter will not only describe in detail the various technical decisions made when developing the system. It will also describe the architectural design and development practices used throughout the development cycle.

In Section 1.2, an overview of the prototype that has been built using the design and development work described throughout this chapter will be given. Section 1.3 will explain some of the reasons why those technologies were chosen to make up the software stack. Section 1.4 through 1.6 will provide a comprehensive discussion of the details concerning the backend, frontend, and data science components of the solution. Sections 1.7 and 1.8 will document how each member of the team has contributed to version control by using Git as well as documenting how we have deployed our code in production. Finally, in Section 1.9, information will be provided about how CRUD operations were created by each member of the team and how those operations can be found within the codebase. This chapter will conclude with a summary in section 1.10.

1.2 Overview of the prototype

AutoInsight is a vehicle market insight platform that has been created to reduce the amount of information gap that exists between buyers, sellers and policy makers in Sri Lanka. The prototype includes data from two of the largest online vehicle market marketplaces in the country, Riyasewana.

This platform was created to solve a recognized issue in the Sri Lankan vehicle market, which is excessive price volatility due to import restrictions, changes in the exchange rate and changes to the government fuel policy. The existing vehicle marketplaces do not offer any analytical insights into the data. They are only a tool to aid buying and selling. Autosight will fill that gap by providing accessible analytical insights into pricing trends, depreciation trends and vehicle mileage correlations.

The basic prototype will include the following functionalities:

- An interactive analytical dashboard with multiple filters to allow for the analysis of vehicle (automobile) brands, models, years of manufacture, odometer readings and condition type (driven vs. non-driven).
- A series of line graphs that contains the relationship between price and odometer readings for a large number of vehicles and historical and current average national market prices for these vehicles.
- The total number of both historically and currently most popular vehicles, updated on a monthly and weekly basis, will also be available.
- An car comparison feature that allows comparison of models, years and trims side by side.
- User friendly capabilities such as account creation, log in/sign up and price alert through email.

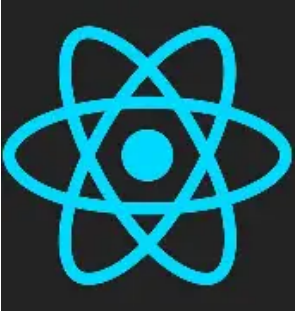
The publicly accessible production system can be found at: <https://www.autoinsight.website/>
The landing site (deployed early in the product lifecycle as an informational site) had a high level of positive interaction on LinkedIn where the team published the following message on the value proposition of the product:

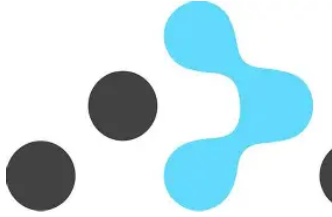
“AutoInsight compares thousands of vehicle listings in Sri Lanka. And leverages data driven insights to determine a fair market value for a vehicle. Now you can easily compare and analyse vehicle listings in one place and understand the true market value of a vehicle instead of guessing about it or relying on your instinct.”

The workflow of the system follows a structured pipeline. Vehicle listing data is collected via automated web scrapers, cleaned and processed via a python based data pipeline, is stored in a Mongo DB database, is served from a RESTful API layer, and is visualized via a React.js, frontend hosted on Vercel and supported by Azure cloud infrastructure for backend services.

1.3 Technology selections

Each of the technology choices was evaluated for scalability, support from the developer community, how well they handled processing the data required for use in AutoInsight and what the development team has as experience working with each technology. The full stack of technologies used for AutoInsight and descriptions justifying each choice can be found in the table below.

Component	Technology	Justification
Frontend Framework	React 19 + TypeScript 	React 19 with TypeScript offers a feature-rich component-based architecture for developing the interactive AutoInsight dashboard. TypeScript helps in achieving more reliable code by detecting errors during compilation, making it more suitable for developing the frontend codebase.
Frontend Build Tool	Vite 7 (SWC compiler) 	Vite 7 with the SWC compiler offers faster compilation compared to traditional bundlers, making it more efficient for rapid development and production bundles for deployment on Vercel.
Frontend Routing	React Router 7	React Router 7 helps implement client-side navigation between the various AutoInsight dashboard screens,

		including Market Explorer, Price Prediction, Comparison Tool, , without requiring full page reloads.
Frontend Animation	Framer Motion 12  Framer	Framer Motion offers animation capabilities that are used throughout the development of the AutoInsight dashboard.
Frontend Icons	Lucide React, React Icons 	Lucide React and React Icons are used as consistent sets of lightweight icons throughout the development of the frontend codebase.
Frontend Testing	Vitest, @testing-library/react  VITEST	Vitest integrated with React Testing Library offers a feature-rich environment for component-level unit and integration tests, making it more efficient compared to Jest.
Frontend Styling	CSS (tokens + global animations) 	Custom CSS with design tokens and global animation definitions offers the utmost degree of control over the design of the dashboard without the overhead of a CSS framework.

Backend Framework	Flask 3.0 + Flask+CORS  Flask	Flask 3.0 was chosen as the backend framework due to its lightweight architecture with a flexible design suitable for creating RESTful APIs with Python. Flask-CORS is chosen to ensure secure cross-origin requests from the React frontend.
Backend Server	Gunicorn 21+ 	Gunicorn is the chosen production-grade WSGI server to run the Flask application.
Backend Language	Python 3.10+ 	Python was chosen as the backend framework due to its rich ecosystem of libraries for data processing and machine learning, allowing the API layer to be seamlessly integrated with the ML pipeline.
Database	MongoDB 	MongoDB is chosen as the primary database to store vehicle information as well as the output from the ML pipeline.
Web Scraping	Selenium 4.20, BeautifulSoup4, lxml 	Selenium version 4.20 is responsible for rendering JavaScript pages from Riyasewana websites, while BeautifulSoup4 library with the lxml parser is responsible for parsing the scraped HTML data for structured vehicle listings.
Data Processing	Pandas, OpenPyXL	The pandas library offers basic data manipulation and cleaning capabilities required by the data pipeline of the scraped vehicle data. The OpenPyXL library allows the development team to read and write Excel-formatted output files, including

		the template for the monthly price prediction model.
ML Core Libraries	Numpy, Pandas, scikit-learn 	NumPy and pandas libraries are at the core of the data preprocessing pipeline. The Scikit-learn library offers utilities required by the model evaluation pipeline of the developed model.
ML Models	XGBoost 2.0, CatBoos T 	Linear Regression acts as the baseline model. Decision Tree and Random Forest act as the ensemble-based models. LightGBM acts as the primary model used for vehicle price prediction. This is because it performs better than the other models, achieving an R^2 score of 0.8575 and a MAE of LKR 985,067 on the test data set.
ML Training	Jupyter Notebooks 	Jupyter Notebooks offer an interactive environment for the training and evaluation process of the ML pipeline.
ML Serialisation	Joblib	Joblib serialises the trained model and the encoding mappings for the model, ensuring the consistency of the feature processing during the training and prediction phases.

ML containerisation	Docker + Docker Compose 	Docker and Docker Compose containerise the ML serving environment, ensuring consistency and reproducibility by using the base image python:3.10-slim for the model inference API to be deployed on port 5000.
Hosting (Frontend)	Vercel 	Vercel hosts the React application from the GitHub repository.
Hosting (Backend)	Azure Cloud (Gunicorn + Python) 	Azure hosts the containerised Flask backend services, providing a scalable and reliable cloud environment for the API and ML inference endpoints.
Frontend Tests	Vitest, @testing-library/user-event 	User event testing, provided by @testing-library/user-event, allows for comprehensive testing of the frontend components along with the unit tests provided by Vitest.
Backend Tests	Pytest, pytest-flask	Pytest, along with the pytest-flask plugin, provides a robust framework for testing the Flask API endpoint functionality and response types.

		
Code Quality	ESLint 9 + TypeScript ESLint  ESLint	ESLint 9, along with the TypeScript ESLint plugin, enforces coding standards and type errors for the React frontend codebase during development.
Version Control	Git / Github 	GitHub hosts the main code repository, and feature branch development and pull requests ensure that the main branch remains stable. GitHub Actions provides CI/CD pipelines for the deployment process to Vercel and Azure.
Package Managers	npm, pip 	npm is used to manage the frontend JavaScript dependencies, while pip is used to manage the Python backend and ML pipeline dependencies.
Design	Figma 	Figma was used for collaborative UI/UX design, which allowed the team to collaborate on the UI/UX design even before the start of the development process.
Development Tool	Visual Studio Code	Visual Studio Code was chosen as the IDE for the

		project because of its extensions, terminal, and support for both TypeScript and Python.

Table 1 : Technology Selections of AutoInsight

Overall the system uses a three-tier structure: Frontend (React.js) communicates directly to the Python / Flask-CORS backend API layer (via HTTP), then makes requests of the MongoDB and the trained ML models. The frontend is provided by Vercel, backend services are hosted on Azure. Separating these functionalities will enhance the scalability and maintenance of the application.

1.4 Implementation of the backend component

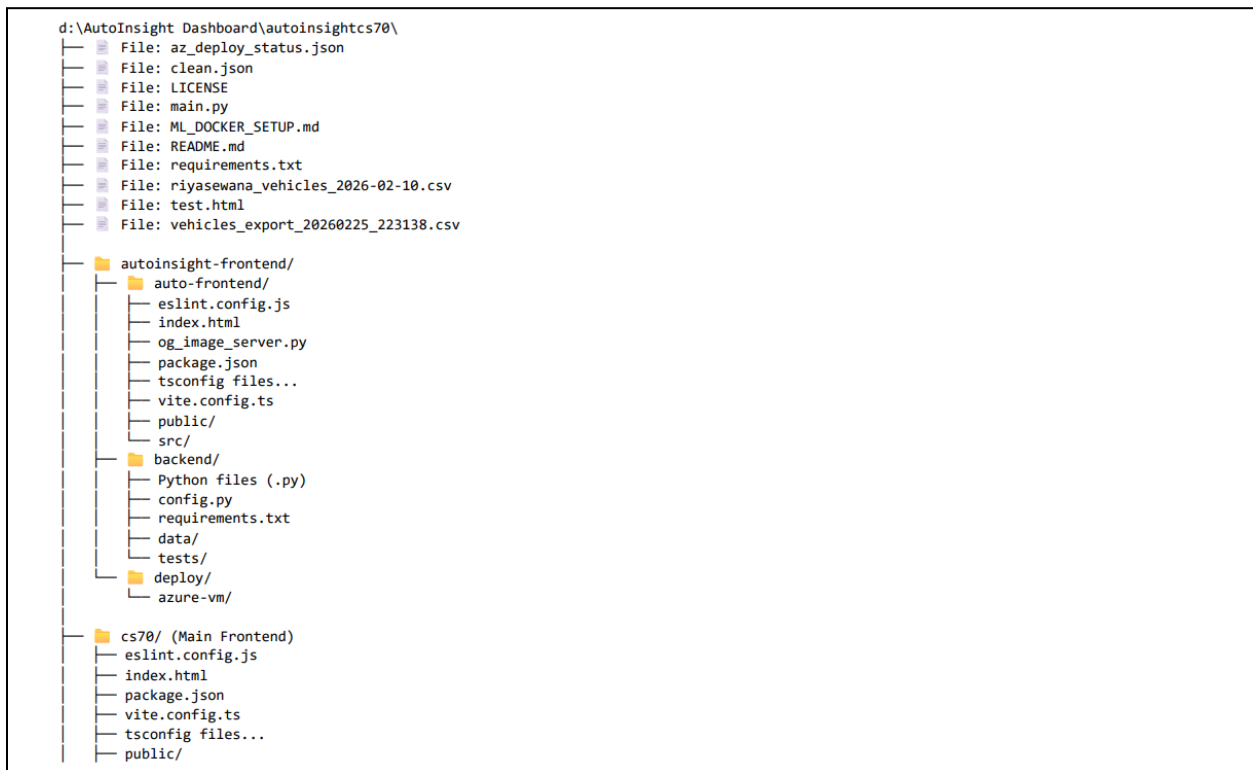
The backend of AutoInsight is divided into distinct modules (each containing API interfaces) that handle vehicle data retrieval, user authentication, machine learning prediction, and database functionality. There are two main parts to the backend. First, there is a Flask-CORSs server which handles general API endpoints and user management. Second, there is a computer program written using Flask (a framework for building web applications in Python) that handles data science and machine learning prediction API endpoints only.

1.4.1 Backend Architecture

The backend system utilizes a RESTful architecture and the frontend system, implemented using React.js, employs AJAX calls to each of the RESTful endpoints. CRUD operations (Create, Read, Update, Delete) for users, price and comparison tools are all implemented using Node.js on the backend. Flask is then used to execute the inference for machine learning based on the vehicle attributes that the user provides, allowing for a return of the predicted fair market value for the vehicle.

The Project folder contains subfolders for data processing, web scraping and API development with the following structure.

- cs70/ - The cs70 directory contains the source code for the application (both frontend and backend).
- data_processing / - data_processing directory contains the python scripts used for manipulating data, including cleaning, normalizing and detecting outliers.
- web_scraping/ - The web_scraping directory contains all of the python scripts using Selenium and BeautifulSoup for web scraping on Riyasewana



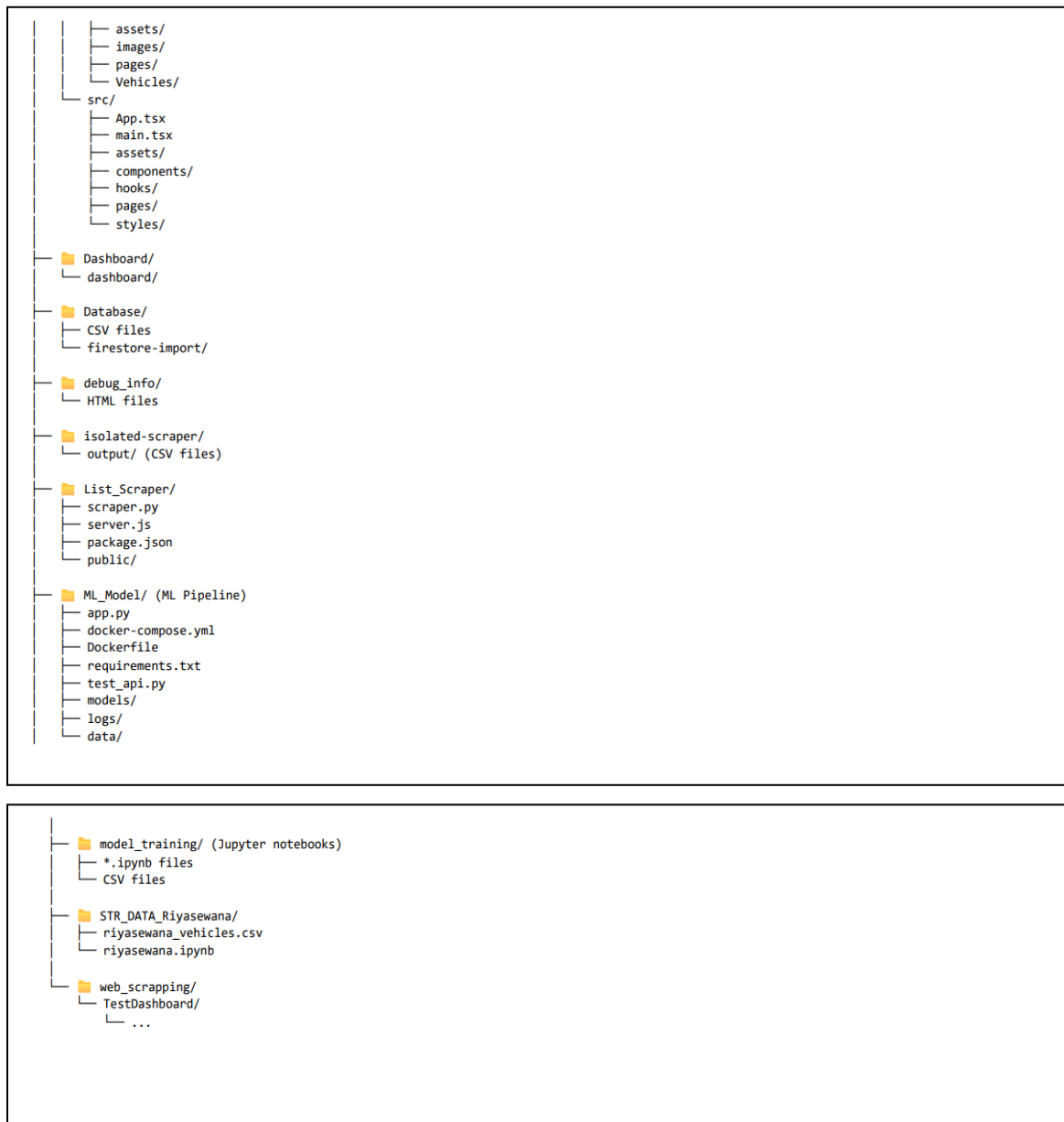


Figure 1: Project Folder Structure

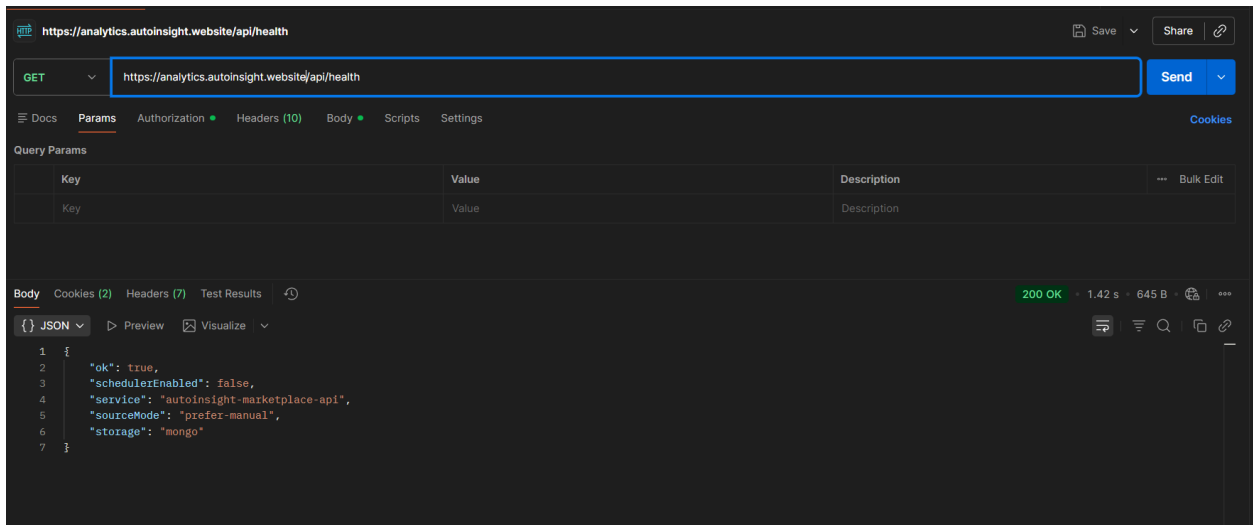


Figure 2: Flask-CORS server file

1. GET/api/health

```
209
270 @app.get("/api/health")
271 def health() -> Any:
272     storage = "mongo" if isinstance(repository, MongoListingRepository) else "file-fallback"
273     payload = {
274         "ok": True,
275         "service": "autoinsight-marketplace-api",
276         "schedulerEnabled": settings.enable_scheduler,
277         "sourceMode": settings.source_mode,
278         "storage": storage,
279     }
280     if storage == "file-fallback":
281         fallback_reason = get_last_repository_build_error()
282         if fallback_reason:
283             payload["storageReason"] = fallback_reason
284     return jsonify(payload)
285
```

Figure 3: GET /api/health — Flask app file

2. GET /api/facet

```
@app.get("/api/facets")
def facets() -> Any:
    try:
        filters = parse_listing_filters(request.args)
    except ValueError as exc:
        return jsonify({"error": str(exc)}), 400
    cache_key = f"facets::{request.query_string.decode('utf-8')}"
    cached = query_cache.get(cache_key)
    if cached is not None:
        return jsonify(cached)
    payload = repository.get_facets(filters)
    query_cache.set(cache_key, payload)
    return jsonify(payload)
```

Figure 4: GET /api/facets code block

3. GET /api/listings/path:listing_id

```
330
331 @app.get("/api/listings/<path:listing_id>")
332 def listing_detail(listing_id: str) -> Any:
333     item = repository.get_listing(listing_id)
334     if not item:
335         return jsonify({"error": "Listing not found"}), 404
336     return jsonify({"item": item})
337
```

Figure 5: beneath the GET /api/listings/path:listing_id code block

4. PUT /api/favorites

```
@app.put("/api/favorites")
def favorites_put() -> Any:
    payload = request.get_json(silent=True) or {}
    user_key = str(payload.get("userKey") or "local-device").strip() or "local-device"
    listing_ids = payload.get("listingIds", [])
    if not isinstance(listing_ids, list):
        return jsonify({"error": "listingIds must be an array"}), 400
    saved = repository.save_favorites(user_key, [str(item) for item in listing_ids])
    return jsonify({"userKey": user_key, "listingIds": saved})
```

Figure 6: PUT /api/favorites

5. POST /api/compare

```
@app.post("/api/compare")
def compare() -> Any:
    payload = request.get_json(silent=True) or {}
    listing_ids = payload.get("ids", [])
    if not isinstance(listing_ids, list):
        return jsonify({"error": "ids must be an array"}), 400
    cleaned = [str(item) for item in listing_ids[:3] if item]
    if len(cleaned) < 2:
        return jsonify({"error": "Provide at least 2 listing IDs"}), 400
    items = repository.get_many(cleaned)
    return jsonify({"items": items})
```

Figure 7: POST /api/compare

6. POST /api/admin/refresh

```
@app.post("/api/admin/refresh")
def refresh() -> Any:
    try:
        payload = refresh_repository("manual")
        return jsonify(payload)
    except FileNotFoundError as exc:
        return jsonify({"error": str(exc)}), 404
    except Exception as exc: # pragma: no cover
        return jsonify({"error": str(exc)}), 500
```

Figure 8: POST /api/admin/refresh

7. GET /api/og-image

```
@app.get("/api/og-image")
def og_image() -> Any:
    raw_url = request.args.get("url", "")
    valid_url = _validate_riyasewana_url(raw_url)
    if not valid_url:
        return jsonify({"error": "Only valid Riyasewana URLs are supported"}), 400
    image_url = _fetch_og_image(valid_url, image_cache)
    response = jsonify({"url": valid_url, "image": image_url})
    response.headers["Cache-Control"] = f"public, max-age={settings.og_image_cache_ttl_seconds}"
    return response

return app
```

Figure 9: GET /api/og-image

1.4.2 Web Scraping Pipeline

The development of a custom web scraping pipeline was undertaken by the team to collect data from Riyasewana, both of which do not have public APIs. Following written approval from both websites, the pipeline was created in 2 stages:

- 1. Static Page Scraping:** The HTML content from all the listing pages is parsed via BeautifulSoup using server-side rendering (SSR). This allows for all vehicle attributes (make, model, year, fuel-type, condition, transmission, mileage, price, location) to be extracted from the listing pages and structures into pandas DataFrames.
- 2. Dynamic Page Scraping:** The Selenium WebDriver is used to scrape dynamic content as it is loaded through JavaScript. The browser will scroll until it finds content, pausing for it to load to the page, and scraping each piece of content through the same logic utilized for static page scraping.

Rate limiting is enforced throughout the scraping process to ensure compliance with the conditions outlined in the data access approval letter provided by Riyasewana. The scripts are restricted to 100 requests per minute and contain randomized delays between requests, which reduces the likelihood of being blocked by the IP addresses used.

Finally, the extracted data is written to two CSV files for intermediate storage to facilitate the re-running of preprocess experiments without needing to rescrape. The batch storage methods also allow for easy compatibility with downstream Python processing libraries.

```

387 def extract_vehicle_data(title, boxtext_divs):
388     data = {k: None for k in FIELD_NAMES}
389
390     title_lower = title.lower()
391     for kw, vt in {
392         'car': 'Car', 'motorbike': 'Motorbike', 'motorcycle': 'Motorbike',
393         'three wheel': 'Three Wheel', 'van': 'Van', 'suv': 'SUV', 'jeep': 'SUV',
394         'lorry': 'Lorry', 'bus': 'Bus', 'pickup': 'Pickup',
395         'tractor': 'Tractor', 'bicycle': 'Bicycle',
396     }.items():
397         if kw in title_lower:
398             data['Vehicle Type'] = vt
399             break
400
401     m = re.search(r'\b(19|20)\d{2}\b', title)
402     if m:
403         data['Year'] = int(m.group())
404
405     parts = title.split()
406     if parts:
407         data['Make'] = parts[0]
408         skip = {'car', 'motorbike', 'motorcycle', 'three', 'wheel', 'van', 'suv', 'jeep',
409             'lorry', 'bus', 'pickup', 'heavy-duty', 'heavy', 'duty'}
410         model_parts = [p for p in parts[1:]
411             if not (p.isdigit() and len(p) == 4 and 1900 <= int(p) <= 2030)
412             and p.lower() not in skip]
413         data['Model'] = ' '.join(model_parts) or None
414
415     for div in boxtext_divs:
416         text = div.get_text(strip=True)
417         if 'Rs.' in text or 'Negotiable' in text:
418             data['Price'] = text
419         elif '(km)' in text:
420             km = re.search(r'(\d+(?:,\d{3})*)', text)
421             if km:
422                 data['Milleage'] = int(km.group().replace(',', ''))
423         elif re.match(r'\d{4}-\d{2}-\d{2}', text):

```

Figure 10: BeautifulSoup scraping script

```

def
setup_driver()
:

306     chrome_opts = Options()

307     for arg in ("--headless", "--disable-gpu",
308         "--no-sandbox",

309         "--disable-dev-shm-usage",
310         "--window-size=1920,1080",

311         "--disable-extensions",
312         "--disable-plugins",

```

```

310         "user-agent=Mozilla/5.0 (Windows NT 10.0;
Win64; x64) AppleWebKit/537.36"):

311         chrome_opts.add_argument(arg)

312         chrome_opts.add_experimental_option("prefs", {

313             "profile.managed_default_content_settings.images":
2,

314             "profile.default_content_setting_values.notifications":
2,

315             "profile.managed_default_content_settings.stylesheets":
2,

316         })

317         binary = find_browser_binary('chrome')

318         if binary:

319             chrome_opts.binary_location = binary

320

321         edge_opts = EdgeOptions()

322         for arg in ("--headless", "--disable-gpu",
"--no-sandbox",

323             "--disable-dev-shm-usage",
"--window-size=1920,1080",

```

```

324         "--disable-extensions",
        "--disable-plugins",

325         "user-agent=Mozilla/5.0 (Windows NT 10.0;
        Win64; x64) AppleWebKit/537.36"):

326         edge_opts.add_argument(arg)

327     binary_e = find_browser_binary('edge')

328     if binary_e:

329         edge_opts.binary_location = binary_e

330

331     # 1. Selenium Manager (Chrome)

332     try:

333         return
        _apply_timeouts(webdriver.Chrome(options=chrome_opts))

334     except (NoSuchDriverException, WebDriverException) as
        e:

335         last_err = e

336

337     # 2. Local / env chromedriver paths

338     candidates = []

339     if os.environ.get("CHROMEDRIVER_PATH"):

```

```

340         candidates.append(os.environ["CHROMEDRIVER_PATH"])

341     candidates += [

342         os.path.join(os.getcwd(), 'chromedriver.exe'),

343         os.path.join(os.path.dirname(__file__),
344             'chromedriver.exe'),

345         os.path.join(os.path.dirname(__file__), 'drivers',
346             'chromedriver.exe'),

347     ]

348     sys_driver = shutil.which('chromedriver')

349     if sys_driver:

350         candidates.append(sys_driver)

351     for path in dict.fromkeys(os.path.normpath(p) for p in
352         candidates if p):

353         if not os.path.exists(path):

354             continue

355         try:

356             return
357             _apply_timeouts(webdriver.Chrome(service=Service(path),
358                 options=chrome_opts))

359         except Exception as err:

```

```

356         last_err = err

357

358     # 3. webdriver-manager (Chrome)

359     if ChromeDriverManager:

360         try:

361             mgr_path = ChromeDriverManager().install()

362             return
            _apply_timeouts(webdriver.Chrome(service=Service(mgr_path
            ), options=chrome_opts))

363         except Exception as err:

364             last_err = err

365

366     # 4. Edge fallback

367     try:

368         return
            _apply_timeouts(webdriver.Edge(options=edge_opts))

369     except Exception as err:

370         last_err = err

371

```

```

372     if EdgeChromiumDriverManager:
373         try:
374             mgr_path =
EdgeChromiumDriverManager().install()
375             return
_apply_timeouts(webdriver.Edge(service=EdgeService(mgr_pa
th), options=edge_opts))
376         except Exception as err:
377             last_err = err
378
379     raise RuntimeError(
380         "Unable to start browser driver. Install
Chrome/Edge or set "
381         "CHROMEDRIVER_PATH/EDGEDRIVER_PATH, or: pip
install webdriver-manager"
382     ) from last_err

```

Figure 11: Selenium scraping script

```

def scrape_page(driver, page_num, search_url_template):
    vehicles = []
    try:
        url = search_url_template.format(page_num=page_num)
        target = parse_search_filters(url)
        target_make = target.get('make')
        target_model = target.get('model')
        target_year = target.get('year')
        target_vehicle_type = target.get('vehicle_type')

        driver.get(url)
        WebDriverWait(driver, 20).until(
            lambda d: len(d.find_elements(By.CSS_SELECTOR, "li.v-card, li.item")) > 0
        )
        time.sleep(1)

        soup = BeautifulSoup(driver.page_source, 'html.parser')
        items = soup.select('li.v-card, li.item')

```

Figure 12: Rate limiting/delay logic in the scraping script

Main Throttle Loop

```

    delay = 1 + (time.time() % 2)
    time.sleep(delay)
    page += 1

```

Figure 13: Main Throttle Loop

Database > riyasewana_vehicles_2025-02-09.csv > data

	Vehicle Type,Make,Model,Year,Price,Milleage,District,published date,Vehicle URL
1	Car,Toyota,Axio,2008,"Rs. 7,150,000",178000,Colombo,2/1/2026,https://riyasewana.com/buy/toyota-axio-sale-colombo-11139348
2	Car,Suzuki,Wagon R,2016,"Rs. 5,690,000",79000,Ragama,2/7/2026,https://riyasewana.com/buy/suzuki-wagon-r-sale-ragama-11167348
3	Car,Toyota,VITZ SAFETY EDITION,2016,Negotiable,85000,Chilaw,2/7/2026,https://riyasewana.com/buy/toyota-vitz-safety-sale-chil
4	Car,Suzuki,Alto Japan,2023,"Rs. 6,590,000",9900,Kiribathgoda,2/7/2026,https://riyasewana.com/buy/suzuki-alto-japan-sale-kiri
5	Car,Honda,EK3 VIRS,2000,Negotiable,264,Kandy,2/7/2026,https://riyasewana.com/buy/honda-ek3-virs-sale-kandy-11167344
6	Car,Suzuki,Alto K10,2011,"Rs. 3,245,000",156100,Veyangoda,2/7/2026,https://riyasewana.com/buy/suzuki-alto-k10-sale-veyangoda
7	Car,Mazda,6,2013,"Rs. 12,200,000",76000,Colombo,2/7/2026,https://riyasewana.com/buy/mazda-6-sale-colombo-11167340
8	Car,Toyota,Lancer,2005,Negotiable,5223,Kegalle,2/7/2026,https://riyasewana.com/buy/toyota-lancer-sale-kegalle-11167337
9	Car,BMW,740 Col 3: Model iable,36000,Colombo,2/7/2026,https://riyasewana.com/buy/bmw-740le-sale-colombo-11167332
10	Car,Toyota,NHP10 AQUA,2015,"Rs. 7,275,000",,Minuwangoda,2/6/2026,https://riyasewana.com/buy/toyota-nhp10-aqua-sale-minuwango
11	Car,Suzuki,Alto,2014,"Rs. 3,590,000",142000,Malabe,2/6/2026,https://riyasewana.com/buy/suzuki-alto-sale-malabe-11167326
12	Car,Suzuki,Spacia Custom Xs,2024,"Rs. 8,700,000",15000,Puttalam,2/6/2026,https://riyasewana.com/buy/suzuki-spacia-custom-sal
13	Car,Suzuki,Estilo,2011,Negotiable,54000,Kalmunai,2/6/2026,https://riyasewana.com/buy/suzuki-estilo-sale-kalmunai-11167324
14	Car,Toyota,Duet,2003,"Rs. 3,050,000",233800,Maharagama,2/6/2026,https://riyasewana.com/buy/toyota-duet-sale-maharagama-11167
15	Car,Toyota,Vitz KSP90,2008,"Rs. 5,450,000",114000,Colombo,2/6/2026,https://riyasewana.com/buy/toyota-vitz-ksp90-sale-colombo
16	Car,Toyota,Corolla 141,2007,"Rs. 6,760,000",141000,Battaramulla,2/6/2026,https://riyasewana.com/buy/toyota-corolla-141-sale-
17	Car,Renault,KWID RXT,2016,"Rs. 3,625,000",60125,Kandy,2/6/2026,https://riyasewana.com/buy/renault-kwid-rxt-sale-kandy-111673
18	Car,Toyota,ALLION 240,2002,"Rs. 6,575,000",165000,Ampara,2/6/2026,https://riyasewana.com/buy/toyota-allion-240-sale-ampara-1
19	Car,Toyota,Allion 260 G,2012,"Rs. 8,900,000",177000,Kandana,2/6/2026,https://riyasewana.com/buy/toyota-allion-260-sale-kanda
20	Car,Perodua,Viva Elite,2012,"Rs. 4,290,000",110000,Kurunegala,2/6/2026,https://riyasewana.com/buy/perodua-viva-elite-sale-ku
21	Car,Toyota,Axio Limited,2009,Negotiable,138221,Katunayake,2/6/2026,https://riyasewana.com/buy/toyota-axio-limited-sale-katun
22	Car,Toyota,AXIO,2015,"Rs. 10,500,000",105000,Malabe,2/6/2026,https://riyasewana.com/buy/toyota-axio-sale-malabe-11167309
23	Car,Micro,Panda Cross,2018,"Rs. 3,980,000",73200,Kiribathgoda,2/5/2026,https://riyasewana.com/buy/micro-panda-cross-sale-kir
24	Car,Toyota,Axio Fielder Hybrid,2018,"Rs. 10,800,000",140000,Maharagama,1/30/2026,https://riyasewana.com/buy/toyota-axio-fiel
25	Car,Suzuki,Swift,2011,"Rs. 5,990,000",59400,Bandaragama,2/6/2026,https://riyasewana.com/buy/suzuki-swift-sale-bandaragama-11
26	Car,Mazda,Demio,2007,"Rs. 4,750,000",180000,Nugegoda,2/6/2026,https://riyasewana.com/buy/mazda-demio-sale-nugegoda-11167303
27	Car,Suzuki,MARUTI 800,2012,"Rs. 2,465,000",56500,Maharagama,2/6/2026,https://riyasewana.com/buy/suzuki-maruti-800-sale-mahar
28	Car,Suzuki,Alto,2015,Negotiable,98100,Kurunegala,2/6/2026,https://riyasewana.com/buy/suzuki-alto-sale-kurunegala-11167300
29	Car,Nissan,FB14 Super Saloon,1998,"Rs. 3,250,000",145000,Kandy,2/6/2026,https://riyasewana.com/buy/nissan-fb14-super-sale-ka
30	Car,Nissan,FB12,1989,"Rs. 1,800,000",1230000,Ratnapura,2/6/2026,https://riyasewana.com/buy/nissan-fb12-sale-ratnapura-111672
31	Car,Toyota,Corolla 110,1997,"Rs. 3,350,000",12345,Matugama,2/6/2026,https://riyasewana.com/buy/toyota-corolla-110-sale-matug
32	Car,Suzuki,Swift,2004,Negotiable,118500,Embilipitiya,2/6/2026,https://riyasewana.com/buy/suzuki-swift-sale-embilipitiya-1116
33	Car,Nissan,Hb12 Trad Sunny Wagon,1987,"Rs. 1,375,000",100000,Kurunegala,2/6/2026,https://riyasewana.com/buy/nissan-hb12-trad
34	Car,Suzuki,Swift RS,2017,"Rs. 7,850,000",70000,Panadura,2/6/2026,https://riyasewana.com/buy/suzuki-swift-rs-sale-panadura-11
35	Car,Suzuki,Swift RS,2018,Negotiable,95000,Negombo,2/6/2026,https://riyasewana.com/buy/suzuki-swift-rs-sale-negombo-11167276
36	Car,Toyota,Vitz KSP90,2008,"Rs. 5,600,000",120000,Mirigama,2/6/2026,https://riyasewana.com/buy/toyota-vitz-ksp90-sale-miriga
37	

Figure 14: Sample CSV file output after scraping

Documents 3.1K Aggregations Schema Indexes 6 Validation

Type a query: { field: 'value' } or [Generate query](#)

[EXPLAIN](#) [Reset](#) [Find](#) [Options](#)

[+ ADD DATA](#) [UPDATE](#) [DELETE](#) [EXPORT DATA](#) [EXPORT CODE](#) 25 1 - 25 of 3068

```

_id: ObjectId('69be5046172f054c587ffeac')
id: "https://riyasewana.com/buy/honda-vezel-z-sale-kadawatha-11363652"
vehicleType: "Car"
make: "Honda"
model: "Vezel Z Grade"
year: 2024
priceLkr: 17250000
priceMillion: 17.25
mileage: 18000
district: "Kadawatha"
publishedDate: "Mar 19"
listedAt: "2026-03-19T00:00:00+00:00"
vehicleUrl: "https://riyasewana.com/buy/honda-vezel-z-sale-kadawatha-11363652"
condition: "Used"
imageUrl: null
rawPrice: "Rs. 17,250,000"
sourceSequence: 1
refreshedAt: "2026-03-21T07:59:08.676777+00:00"
validation_status: "validated"
confidence: 0.99
matched_model_row_id: "Sheet1:1669"
market_analysis: Object
  firstSeenAt: "2026-03-21T07:59:08.676777+00:00"
  isActive: true
  staleAt: null
  Show 1 more field

```

Figure 15: Sample dataset record of a scraped vehicle listing

1.4.3 Data Preprocessing

Before raw scraped data is inserted into a database, a multi-tiered processing pipeline takes place in order to clean it up before being conformably structured (to include reference fields). The important processing operations performed on the raw scraped data include:

- **Cleanup:** Duplicate records will be removed and those with error-prone price format eg. Rs. 3,500,000, Rs. 3.5M will be standardised by means of standard RegEx. Furthermore, data that may appear as multiple text fields can be made to reflect the canonical value. For example; variances may exist for fuel type or transmission.
- **Missing Value Processing:** Listings with missing critical fields (year, mileage and price) will undergo either imputation of the missing fields based on their proportion to the total number of listings or removal of the amount missing fields exceeds a predefined threshold. This allows for ML models to receive consistent input from the raw data listings being analysed by the models.
- **Outlier Detection:** Pricing or mileage figures may be flagged as outliers by the machine learning model via the application of an IQR. Therefore, removing sampled records as outliers will preclude the model from associating the “outlier” patterns as typical or representative of the dataset.
- **DB Insertion:** The processed datasets can then be inserted into the Mongo DB via SQLAlchemy with data being structured under an ORM, with each item being retained with appropriate Referential Integrity within the MongoDB.

```
[LOAD] CSV loaded: (13353, 9)
[LOAD] Columns: ['Vehicle Type', 'Make', 'Model', 'Year', 'Price', 'Milleage', 'District', 'published da

[DATE] Parsing dates...
[DATE] Parsed: 13,353 | Failed: 0
[DATE] Date range: 2025-12-21 → 2026-03-23

[PRICE] Parsing price format...
[PRICE] Valid prices: 10,349
[CLEAN] After dropna: (11732, 9)
[CLEAN] After outlier removal: (8928, 9)
[DEDUP] Removed 0 duplicate listings

[DATA] 8,928 rows | 62 makes | 2107 models | 1948.0–2026.0
```

Figure 16: Data preprocessing

```

print("\n[PRICE] Parsing price format...")
def parse_price(val):
    if pd.isna(val):
        return np.nan
    val_str = str(val).strip()
    if val_str.lower() in ['negotiable', 'ask', 'n/a', 'na', '']:
        return np.nan
    # Remove "Rs. " prefix, spaces, and commas
    val_str = val_str.replace('Rs.', '').replace('Rs', '').replace(',', '').strip()
    try:
        price = float(val_str)
        # Filter out placeholder values like "Rs. 1"
        if price < 10000:
            return np.nan
        return price
    except (ValueError, TypeError):
        return np.nan

df['Price'] = df['Price'].apply(parse_price)
valid_prices = df['Price'].notna().sum()
print(f"[PRICE] Valid prices: {valid_prices:,}")

# Drop rows missing critical fields
required_cols = ['Make', 'Model', 'Year', 'Mileage', 'published date']
df.dropna(subset=required_cols, inplace=True)
print(f"[CLEAN] After dropna: {df.shape}")

```

Figure 17: Data cleaning script

```

# Outlier removal (price and mileage)
df = df[(df['Price'] > PRICE_FLOOR) & (df['Price'] < PRICE_CAP)]
df = df[(df['Mileage'] >= 0) & (df['Mileage'] < MILEAGE_CAP)]

```

[CLEAN] After outlier removal: (8928, 9)

Figure 18: Outlier detection logic

```

1 | #!/usr/bin/env python3

```

```

2 | """

```

```
3  Convert an Excel file to JSON and optionally insert records into  
   MongoDB.  
  
4  
  
5  Usage:  
  
6  1) Fill MONGO_URI with your connection string.  
  
7  2) Run: python excel_to_mongo.py  
  
8  """  
  
9  
  
10 from __future__ import annotations  
  
11  
  
12 import json  
  
13 import math  
  
14 import re  
  
15 from datetime import date, datetime  
  
16 from decimal import Decimal  
  
17 from pathlib import Path  
  
18 from typing import Any, List  
  
19
```

```

20 from openpyxl import load_workbook

21

22

23 EXCEL_FILE = Path("isolated-scraper/output/Template_for_Model_Filled
    (1).xlsx")

24 JSON_FILE =
    Path("isolated-scraper/output/Template_for_Model_Filled.json")

25 MONGO_URI = ""

26 MONGO_DB = "autoinsight"

27 MONGO_COLLECTION = "model_training_data"

28 SHEET_NAME = None

29

30

31 def clean_key(value: Any, index: int) -> str:

32     if value is None:

33         return f"field_{index + 1}"

34     key = str(value).strip()

35     return key if key else f"field_{index + 1}"

36

```

```

37
38 def normalize_value(value: Any) -> Any:
39     if value is None:
40         return None
41     if isinstance(value, float) and math.isnan(value):
42         return None
43     if isinstance(value, datetime):
44         return value.isoformat()
45     if isinstance(value, date):
46         return value.isoformat()
47     if isinstance(value, Decimal):
48         return float(value)
49     return value
50
51
52 def keyify(value: str) -> str:
53     text = value.strip().lower()

```

```

54     text = text.replace("(", " ").replace(")", " ")

55     text = text.replace("-", " ").replace("_", " ")

56     return " ".join(text.split())

57

58

59 def parse_int(value: Any, default: int = 0) -> int:

60     if value is None:

61         return default

62     if isinstance(value, bool):

63         return default

64     if isinstance(value, (int, float)):

65         return int(value)

66     text = str(value).strip()

67     if not text:

68         return default

69     cleaned = re.sub(r"^[0-9]", "", text)

70     return int(cleaned) if cleaned else default

```

```

71
72
73 def parse_price_lkr(value: Any) -> int:
74     if value is None:
75         return 0
76     if isinstance(value, (int, float)):
77         number = float(value)
78         # If the sheet stores values in millions, convert to full LKR.
79         if 0 < number < 10_000:
80             return int(number * 1_000_000)
81         return int(number)
82     text = str(value).strip().lower().replace(",", "")
83     if not text:
84         return 0
85     if "million" in text or " mn" in text or text.endswith("m"):
86         digits = re.sub(r"^[^0-9.]", "", text)
87         return int(float(digits) * 1_000_000) if digits else 0

```

```

88     digits = re.sub(r"^[0-9]", "", text)

89     return int(digits) if digits else 0

90

91

92 def normalize_condition(value: Any) -> str:

93     text = str(value or "").strip().lower()

94     if "brand new" in text or "unregistered" in text or text == "new":

95         return "Brand New"

96     if "recondition" in text:

97         return "Recondition"

98     return "Used"

99

100

101 def normalize_vehicle_type(value: Any) -> str:

102     text = str(value or "").strip().lower()

103     if text in {"car", "cars"}:

104         return "Car"

```

```

105     if text in {"van", "vans"}:
106         return "Van"
107
108     if text in {"suv", "suvs"}:
109         return "SUV"
110
111     if text in {"motorbike", "motorbikes", "bike", "bikes"}:
112         return "Motorbike"
113
114     return "Car"
115
116
117
118
119 def normalize_listed_at(value: Any) -> str:
120
121     if isinstance(value, datetime):
122
123         return value.isoformat()
124
125     if isinstance(value, date):
126
127         return datetime.combine(value,
128                                 datetime.min.time()).isoformat()
129
130     text = str(value or "").strip()
131
132     if not text:

```

```

121         return datetime.now().isoformat()

122     try:

123         parsed = datetime.fromisoformat(text.replace("Z", "+00:00"))

124         return parsed.isoformat()

125     except ValueError:

126         return datetime.now().isoformat()

127
128
129 def get_field(raw: dict[str, Any], aliases: list[str], default: Any =
    "") -> Any:

130     for alias in aliases:

131         if alias in raw and raw[alias] not in (None, ""):

132             return raw[alias]

133     return default

134
135
136 def to_frontend_vehicle(raw: dict[str, Any], index: int) -> dict[str,
    Any]:

```

```

137     vehicle_url = str(get_field(raw, ["vehicle url", "vehiculeurl",
"url", "link"], "")).strip()

138     make = str(get_field(raw, ["make", "brand"], "")).strip()

139     model = str(get_field(raw, ["model"], "")).strip()

140     year = parse_int(get_field(raw, ["year"], 0), 0)

141     price_lkr = parse_price_lkr(get_field(raw, ["pricelkr", "price",
"rawprice"], 0))

142     mileage = parse_int(get_field(raw, ["mileage", "milleage", "km"],
0), 0)

143     district = str(get_field(raw, ["district", "city", "location"],
"")).strip()

144     published_date = str(get_field(raw, ["published date",
"publisheddate", "date"], "")).strip()

145     listed_at = normalize_listed_at(get_field(raw, ["listedat",
"published date", "publisheddate", "date"], ""))

146     condition = normalize_condition(get_field(raw, ["condition",
"title", "status"], "Used"))

147     vehicle_type = normalize_vehicle_type(get_field(raw, ["vehicle
type", "vehicletype", "type"], "Car"))

148     image_url = str(get_field(raw, ["imageurl", "image", "image url"],
"")).strip()

149

150     generated_id = f"listing-{index + 1}"

```

```

151     if make or model or year:

152         generated_id = f"{make}-{model}-{year}-{index +
153             1}".strip("-").replace(" ", "-").lower()

154     vehicle_id = str(get_field(raw, ["id", "listing id", "listingid"],
155         "")).strip() or vehicle_url or generated_id

156     return {

157         "id": vehicle_id,

158         "vehicleType": vehicle_type,

159         "make": make,

160         "model": model,

161         "year": year,

162         "priceLkr": price_lkr,

163         "priceMillion": round(price_lkr / 1_000_000, 2),

164         "mileage": mileage,

165         "district": district,

166         "publishedDate": published_date,

167         "listedAt": listed_at,

```

```

168     "vehicleUrl": vehicle_url,
169     "condition": condition,
170     "imageUrl": image_url or None,
171     "validationStatus": "unmatched",
172     "confidence": 0,
173     "matchedModelRowId": None,
174     "isActive": True,
175 }
176
177
178 def row_to_raw_doc(headers: list[str], row: Any) -> dict[str, Any]:
179     raw: dict[str, Any] = {}
180     for i, key in enumerate(headers):
181         cell_value = row[i] if i < len(row) else None
182         raw[keyify(key)] = normalize_value(cell_value)
183     return raw
184

```

```

185
186 def excel_to_documents(excel_path: Path, sheet_name: str | None =
    None) -> List[dict[str, Any]]:
187     workbook = load_workbook(filename=excel_path, read_only=True,
        data_only=True)
188     try:
189         sheet = workbook[sheet_name] if sheet_name else
            workbook.worksheets[0]
190
191         rows = sheet.iter_rows(values_only=True)
192         headers_row = next(rows, None)
193         if headers_row is None:
194             return []
195
196         headers = [clean_key(v, i) for i, v in enumerate(headers_row)]
197
198         documents: List[dict[str, Any]] = []
199         for index, row in enumerate(rows):
200             if row is None:

```

```

201         continue

202

203     if all(cell is None or str(cell).strip() == "" for cell in
row):
204         continue

205

206     raw_doc = row_to_raw_doc(headers, row)

207     doc = to_frontend_vehicle(raw_doc, index)

208     documents.append(doc)

209

210     return documents

211 finally:

212     workbook.close()

213

214

215 def save_json(documents: List[dict[str, Any]], json_path: Path) ->
None:

216     json_path.parent.mkdir(parents=True, exist_ok=True)

```

```

217     with json_path.open("w", encoding="utf-8") as f:

218         json.dump(documents, f, ensure_ascii=False, indent=2)

219
220
221 def insert_into_mongo(uri: str, db_name: str, collection_name: str,
222 documents: List[dict[str, Any]]) -> int:

223
224     from pymongo import MongoClient

225
226     client = MongoClient(uri, serverSelectionTimeoutMS=10000)

227
228     try:

229         client.admin.command("ping")

230
231         collection = client[db_name][collection_name]

232
233         if not documents:

234             return 0

235
236         result = collection.insert_many(documents)

237
238         return len(result.inserted_ids)

239
240     finally:

241         client.close()

```

```
234
235
236 def main() -> None:
237     if not EXCEL_FILE.exists():
238         raise FileNotFoundError(f"Excel file not found: {EXCEL_FILE}")
239
240     documents = excel_to_documents(EXCEL_FILE, SHEET_NAME)
241
242     save_json(documents, JSON_FILE)
243
244     print(f"Converted to JSON: {JSON_FILE}")
245
246     print(f"Total records: {len(documents)}")
247
248     if not MONGO_URI.strip():
249         print("Mongo insert skipped: MONGO_URI is empty.")
250
251         print("Set MONGO_URI in this file, then run again to insert
into MongoDB.")
252
253     return
```

```

251     inserted_count = insert_into_mongo(MONGO_URI, MONGO_DB,
MONGO_COLLECTION, documents)

252     print(

253         f"Inserted {inserted_count} records into MongoDB: "

254         f"{MONGO_DB}.{MONGO_COLLECTION}"

255     )

256

257

258 if __name__ == "__main__":

259     main()

260

```

Figure 19: Database insertion code

1.4.4 Database Design

The MongoDB serves as the primary repository of data that supports vehicle listings, user accounts, about price changes automatically sent to users, and records of machine learning predictions. The schema for the MongoDB has been designed to allow for efficient querying for the dashboard. The MongoDB has indexed the most frequently filtered columns (e.g., make, model, year, price) in order to improve performance for end users. The four main entities within the MongoDB include the vehicle table, user table, alert table and comparison table all of which support all four types of CRUD operations outlined in Section 1.9.

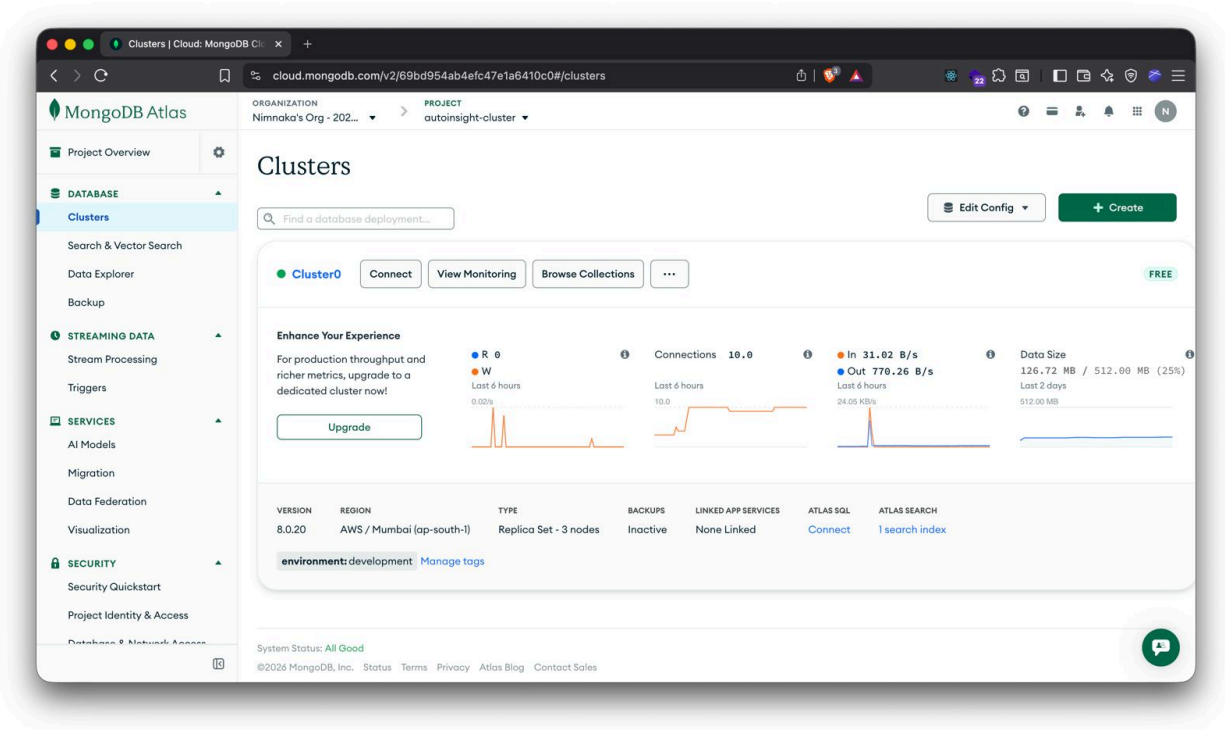


Figure 20: Database Design

1.5 Implementation of the front end component

All of AutoInsight's frontend uses React.js. It creates an engaging, dynamic experience for users. AutoInsight is built as an SPA and all client -side routes are handled using React Router. The backend API returns all the data to display in the dashboard asynchronously using Fetch and Axios through the use of API calls to request data.

1.5.1 Page Structure and Navigation

The application contains four major types of display pages mapped to four distinct routing paths in the SPA.

- **Home/Landing Page** - An informational website found at info-autoinsight.vercel.app; it explains what the autoinsight app does encourage visitors to register or check out how it works.
- **Dashboard(Analytics Overview)** - The application's main analytics page; it contains the weekly market pulse graph, average historical market prices, market shares by fuel type and real-time KPI cards that report the most popular model, average market price, and week to week price movement.
- **Market Explorer** - the market explorer page is a grid containing vehicle cards with pagination that provides the user with the ability to filter the grid by different criteria (make, model, year range, condition, price range, city mileage, fuel type, and transmission).
- **Price Prediction Page** - This page provides a form for users to enter data about a vehicle so that they can receive a fair market value estimate generated by machine learning, as well as a comparison chart to show users how fair market value compares to historical vehicle prices.
- **Comparison Tool** - This page displays two selected vehicle models by side, listing features, performance specifications, and pricing data for each of the cars.
- **About, FAQ, and Contact Page** - The purpose of these pages is to provide general information about the platform, answers to frequently asked questions, and a way for the user to contact the platform.

1.5.2 Component Architecture

All UI elements of the front end will use a component-based architecture in which all UI components will be re-usable React Components. A few examples of the key components that will comprise the application include: Navigation Sidebar; Dashboard KPI Card; Price History Chart; Fuel Type Donut; Vehicle Listing Card; Comparison Table; and Chat panel, etc. State will be managed using React's useState and useEffect Hooks with Global State Management by means of context Providers managing the User's Authentication Status and Active Filters.

1.5.3 Data Visualisation

The primary methods for implementing data visualisation will be interactive charting. Some of the visualisations that will be presented on the dashboard.

Detailed Comparison			
Metric	Toyota Yaris X	Toyota Aqua	Toyota Roomy
Price	10.60M	8.16M	8.25M
Mileage	700 km	118,500 km	101,000 km
Year	📅 2025	📅 2015	📅 2023
Condition	Used	Used	Used
Validation	validated (99%)	validated (99%)	validated (99%)
Location	Kelaniya	Wennappuwa	Colombo
Listed	Invalid Date	Invalid Date	Invalid Date
Match Status	Validated	Validated	Validated

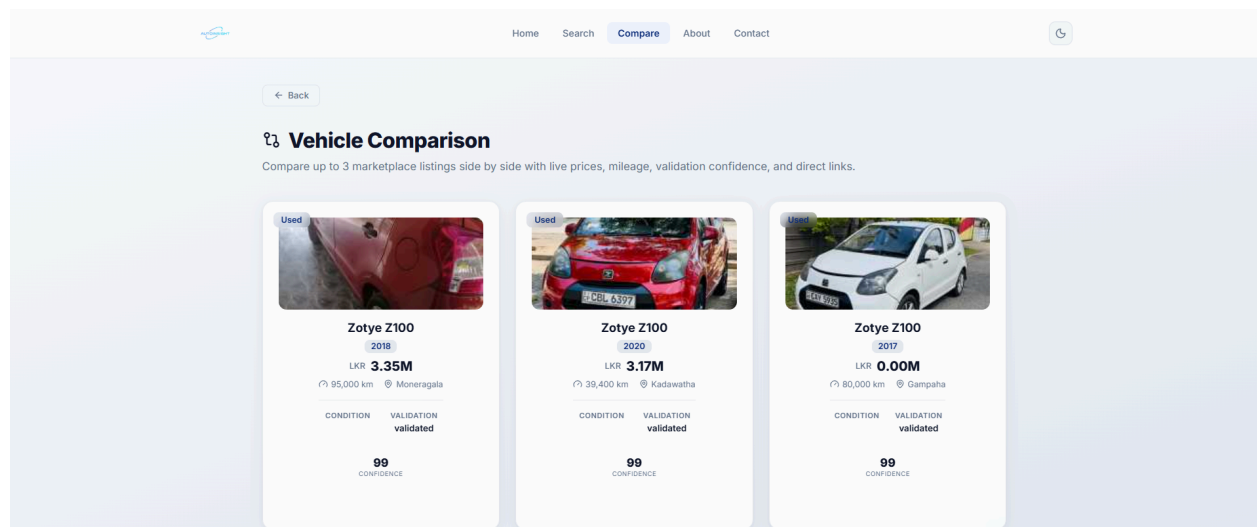
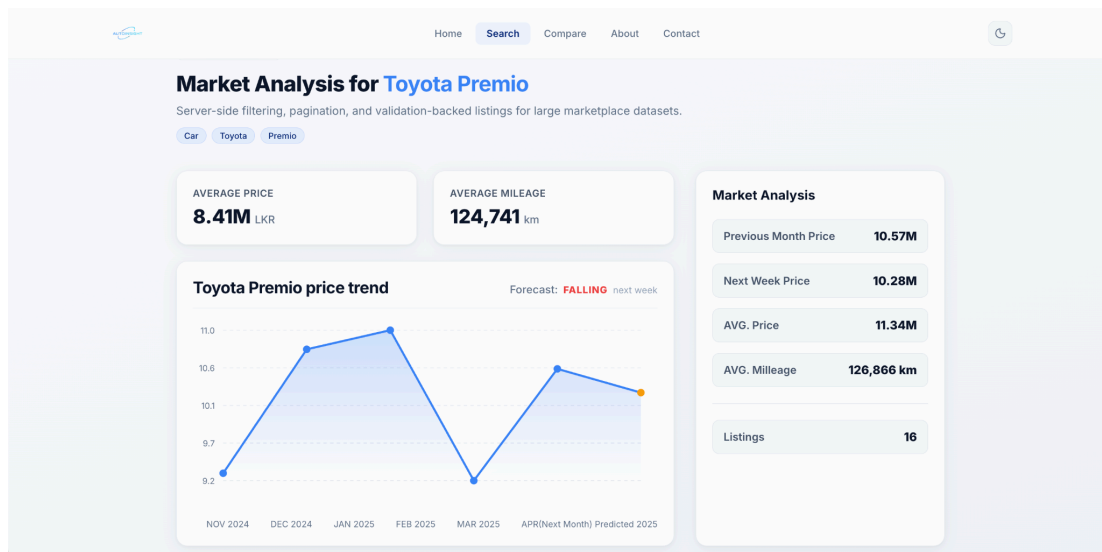


Figure 21: Search page

1.5.4 Responsive Design

The frontend of this application is designed so that it can be responsive to different viewport width; it will work with desktops as well as mobile devices. The card/grid layout will adjust from

grid layout to single column display on mobile devices.

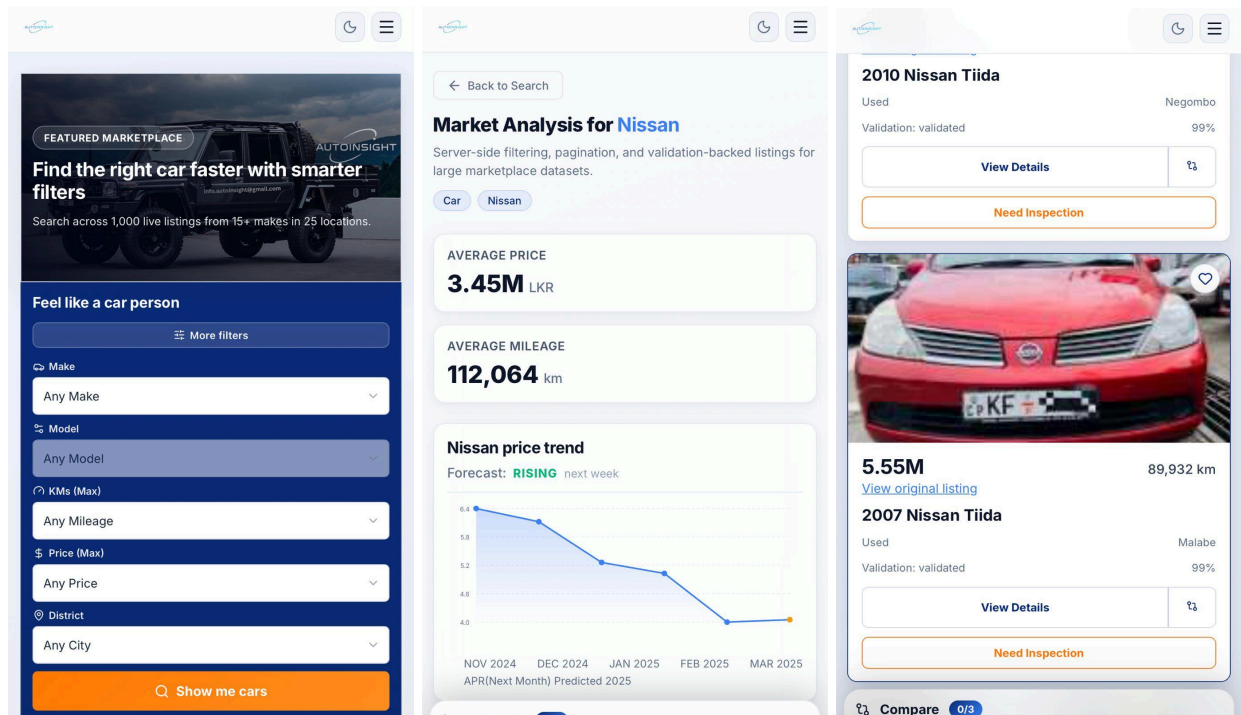


Figure 22: AutoInsight website mobile responsive

1.6 Implementation of the data science component or hardware component (if applicable)

The data science component of AutoInsight is one of the areas that sets the platform apart from other competitors; It is what creates the machine-learning price prediction model; the trend analysis statistical engine; and the dashboard data pipeline that powers the analytical insight provided by the dashboard.

1.6.1 Machine Learning Model

This project evaluates multiple regression algorithms to estimate the fair market price of used vehicles in Sri Lanka. Historical listing data was collected from a vehicle listing service and used to train and compare several models. The final production model is selected based on predictive accuracy, generalization ability, and suitability for tabular market data.

Input Features Used for Prediction

The model receives the following vehicle attributes:

- Make (vehicle manufacture)
- Model (specific vehicle model name)
- Year of manufacture
- Mileage (in kilometres)

Data Preparation and Feature Engineering

Before model training, the dataset is preprocessed as follows:

- Categorical attributes (Make, Model, Location) are transformed using label encoding.
- Numerical attributes are scaled using Min-Max normalization.
- Missing and noisy values are cleaned to improve consistency.
- The dataset is split into 75% training data and 25% validation data.
- Model quality is evaluated using MAE and R2 score.

Model Descriptions:

1. Linear Regression (Baseline Model)

Linear Regression is used as a baseline model because it is simple, interpretable, and computationally efficient. It estimates vehicle price as a weighted linear combination of input features.

Its main value in this project is to provide a reference point for more advanced models. However, vehicle pricing is often influenced by non-linear relationships (for example, mileage and depreciation effects), which limits the predictive power of a purely linear model.

2. Decision Tree Regressor

Decision Tree regression learns hierarchical decision rules by splitting data into smaller groups based on feature thresholds.

It can naturally capture non-linear patterns and feature interactions without requiring complex transformations. This makes it useful for modeling real-world pricing behavior. Its key limitation is overfitting: a deep tree may perform very well on training data but poorly on unseen data.

3. Random Forest Regressor

Random Forest is an ensemble of many decision trees trained on bootstrapped samples with random feature selection. Predictions are averaged across trees. Breiman, L. (2001) Compared with a single decision tree, Random Forest is more stable and typically provides better generalization. It handles non-linearity well and is robust to outliers and noisy records. scikit-learn Developers (2026)

The tradeoff is reduced interpretability and higher computational cost. (IBM (2021)

4. LightGBM Regressor

LightGBM is a gradient boosting framework optimized for speed and high performance on large tabular datasets. It builds trees sequentially, where each new tree corrects errors made by previous trees. LightGBM Developers (2026)

In vehicle price prediction tasks, LightGBM often performs strongly due to its ability to model complex interactions and non-linear trends efficiently. Ke et al. (2017) 'LightGBM It usually requires careful hyperparameter tuning to avoid overfitting but offers an excellent balance between accuracy and training speed.

5. XGBoost Regressor (Primary Model) XGBoost Developers (2026)

XGBoost (Extreme Gradient Boosting) is selected as the primary model for fair market price prediction due to its strong predictive performance and robustness in tabular regression problems. IBM (2024), Chen, T. and Guestrin, C. (2016) 'XGBoost

Key reasons for selecting LightGBM include:

- Strong ability to model non-linear relationships in vehicle pricing
- Built-in regularization to reduce overfitting
- Robust handling of missing values
- Consistently high performance on structured/tabular datasets

Because used vehicle prices are influenced by many interacting factors (age, mileage, model popularity, and location), LightGBM provides a reliable and high-accuracy solution for production deployment

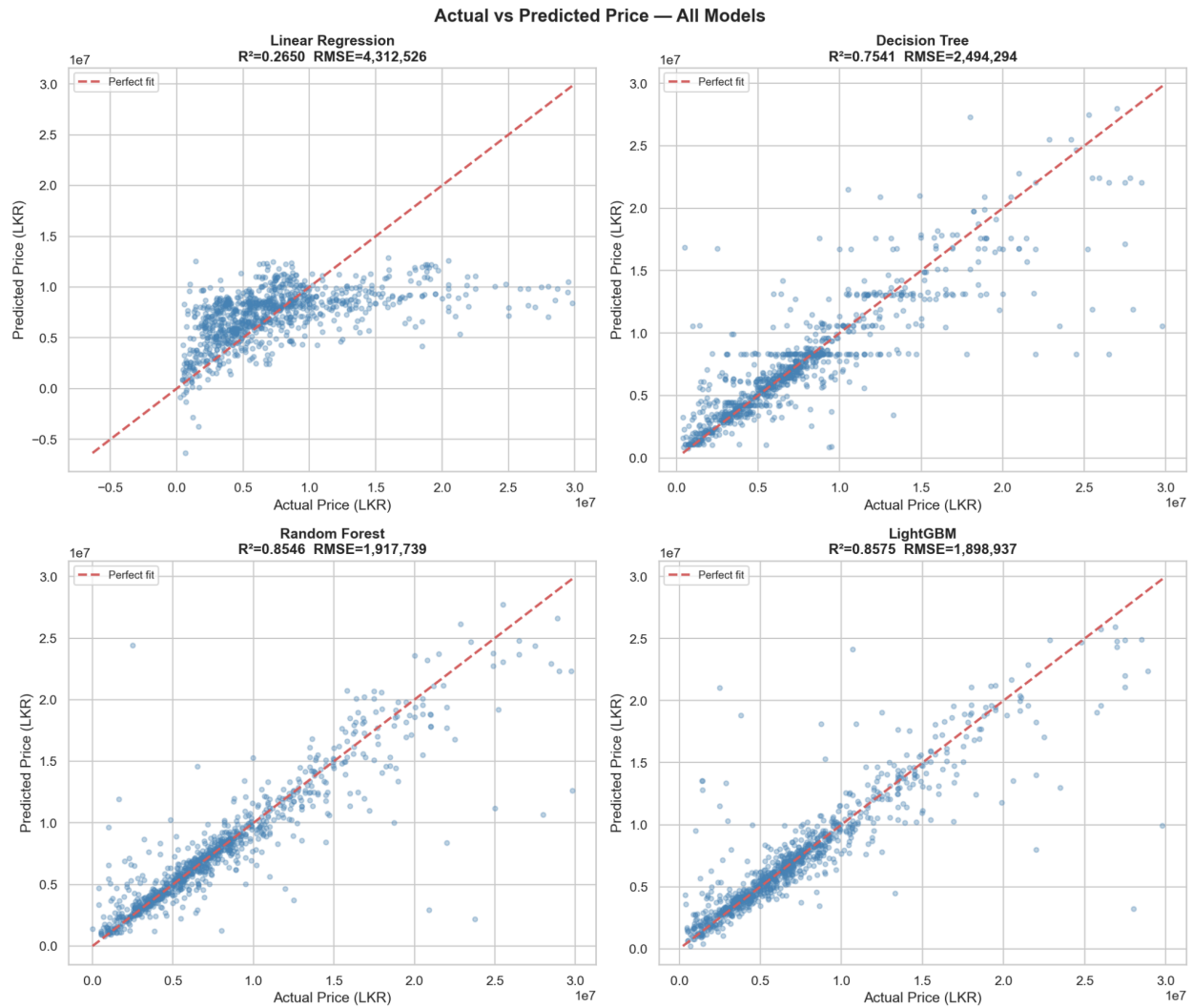


Figure 23: Model Comparison

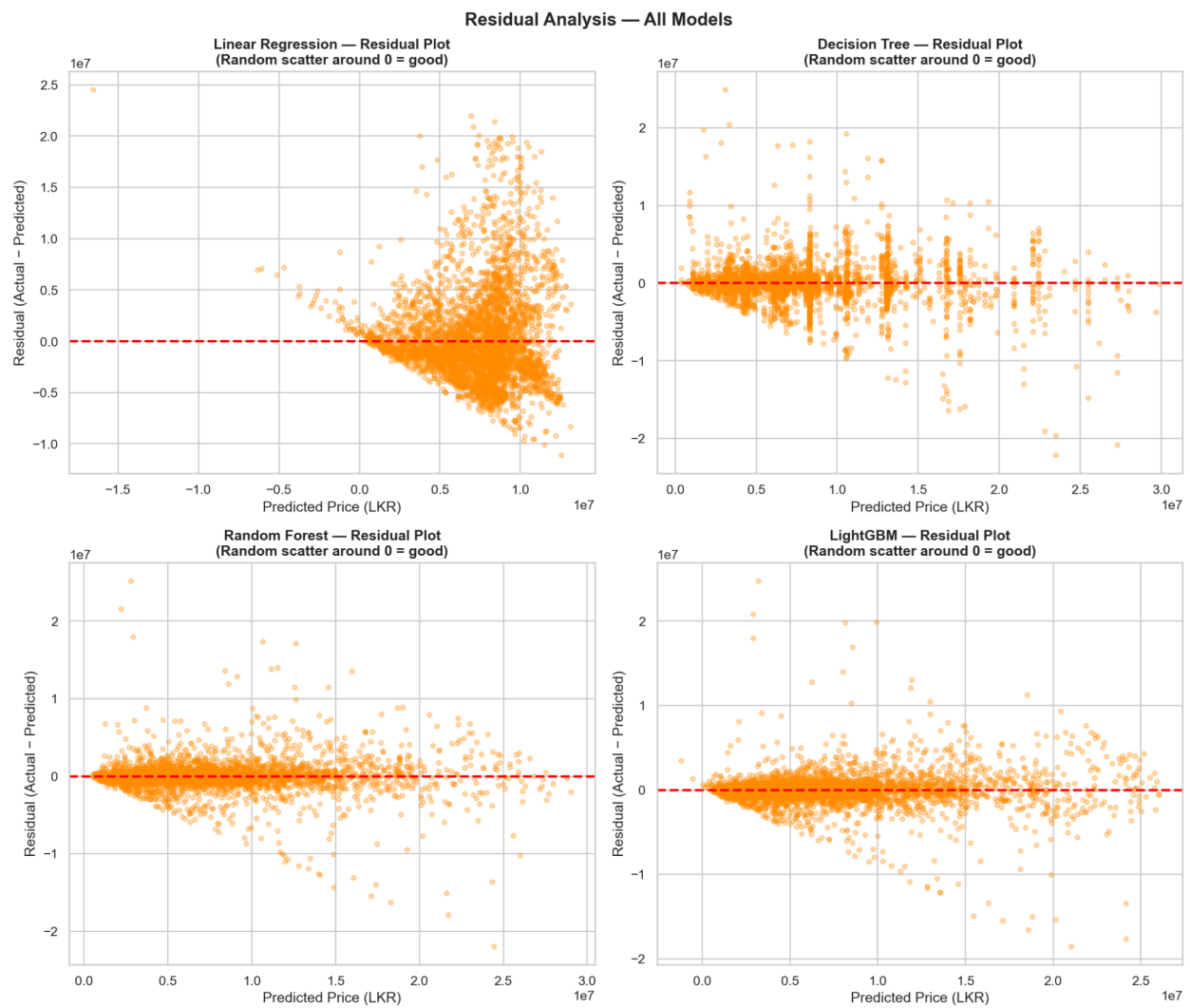


Figure 24: Residual Analysis

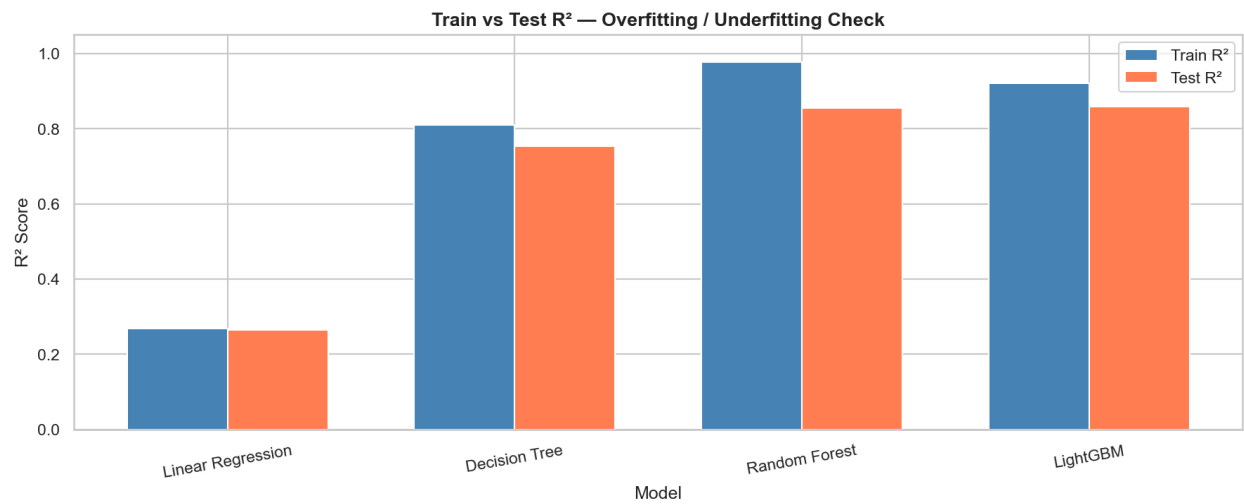


Figure 25: Overfitting / Underfitting Check

Train Dataset by Condition

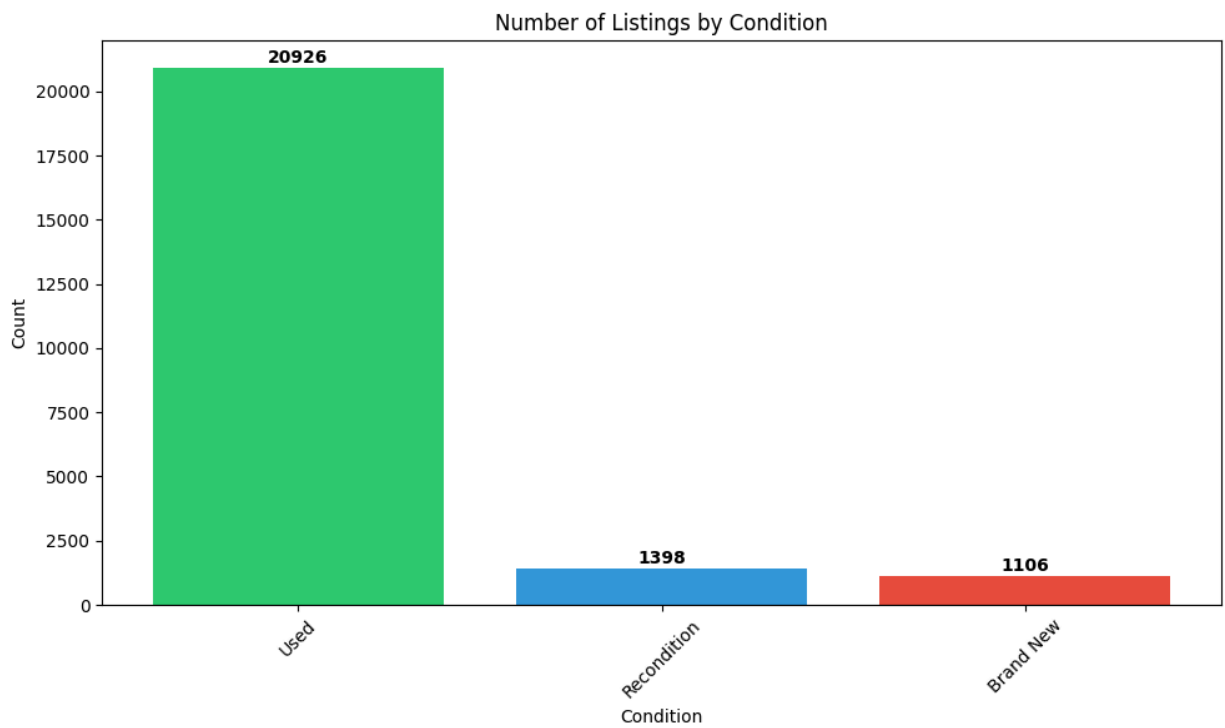


Figure 26: Vehicle classified according to condition

Error Rate per Model

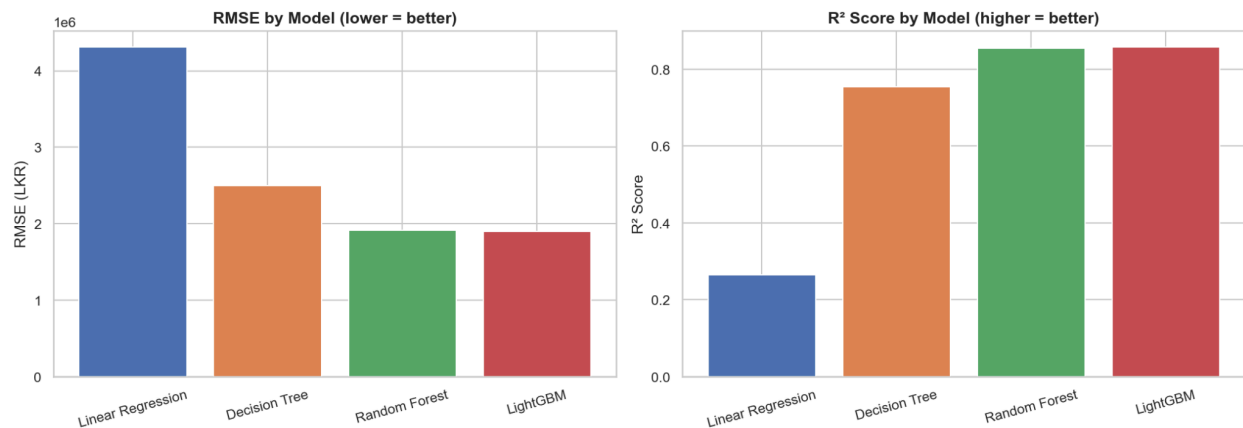


Figure 27: Error rate per model

The input features that are categorical are converted to numerical from using label encoding while the input features that are numerical are converted to mean and standard deviation using Min-Max Normalization methods to create a standardized data set. The dataset was split into 75% training and 25% validation for training and evaluating the prediction power of the ML model by evaluating the model performance on mean absolute error (MAE) and R2 regression metrics.

Once the XGBoost model has been trained, the model is serialized into a pickle file and hosted as a Flask REST endpoint. When the user submits a prediction request from the Price Prediction page, the request to create vehicle fair market price will be sent to the FLask endpoint via a POST with the vehicle attributes passed to the endpoint in a JSON payload format. The endpoint returns the predicted fair market price for the given vehicle attributes.

1.6.2 Trend Analysis and Prediction

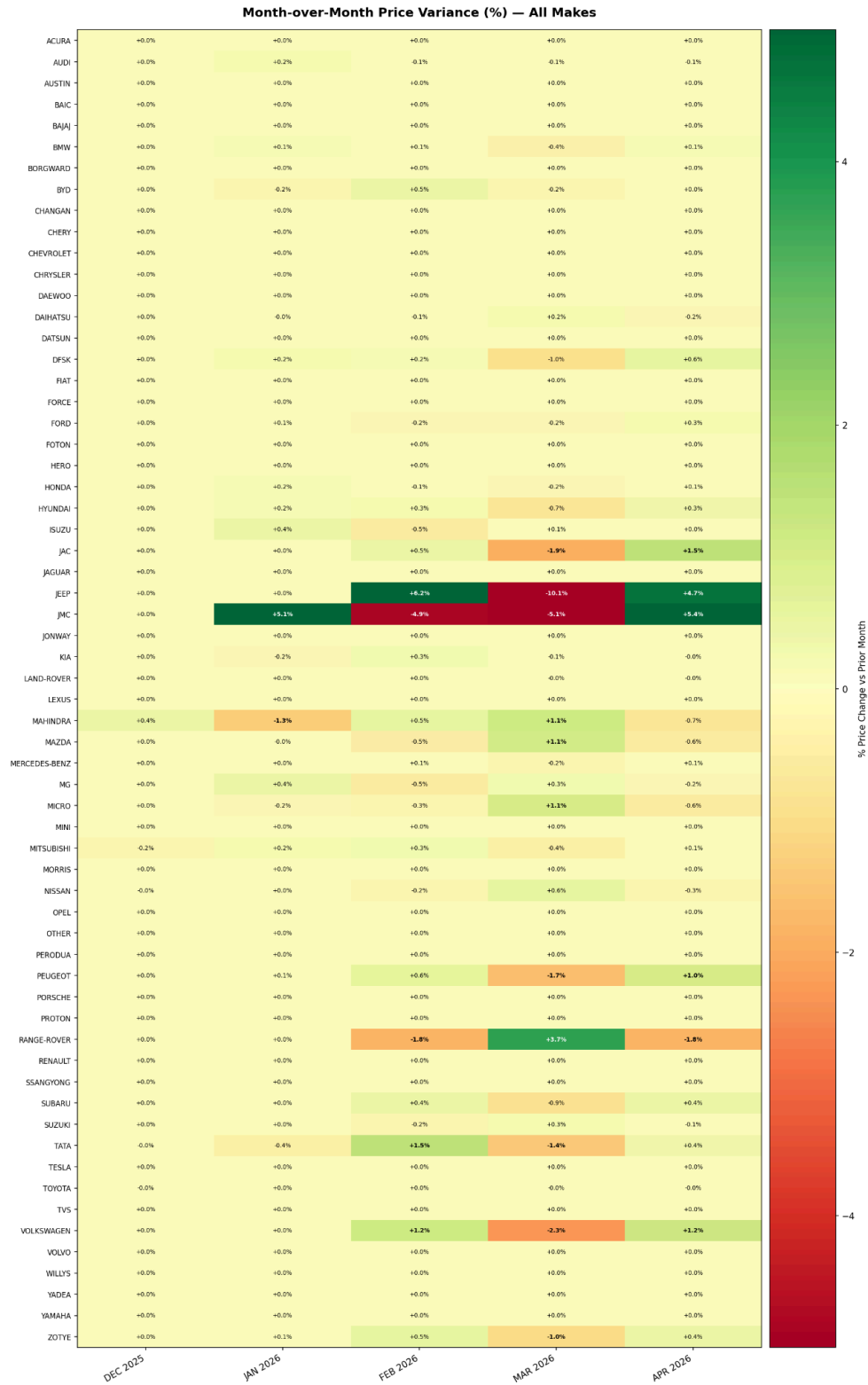


Figure 28: Trend analysis and prediction

1.7 GIT Repository

Git was utilized to provide a means of version control for the development of the AutoInsight project, using GitHub, on the central remote repository site's organisation account `akila20040720/autoinsightcs70`. As of the time of this documentation, the repository contains 72 commits between active branches with 24 deployments to Vercel.

The team adopted a feature-branch based workflow when developing collaboratively, creating specific feature branches for each developer according to the component or page being worked on (e.g. `feat: about page`, `page animation features`, `web scraping updates`). All pull requests were required to undergo a peer review prior to merging into the master branch to ensure that the master branch was kept stable and could always be used for production deployment.

The repository structure reflected a clear separation of concerns:

- `cs70/` - Core frontend and backend application source code.
- `data_processing/` - Python preprocessing scripts and ML model training notebooks.
- `web_scrapping/` - Selenium and BeautifulSoup scraping script.
- `node_modules/` - Node.js dependency directory (excluded from commits via `.gitignore`).
- `Package.json / package-lock.json` - Dependency manifests for the Node.js environment.

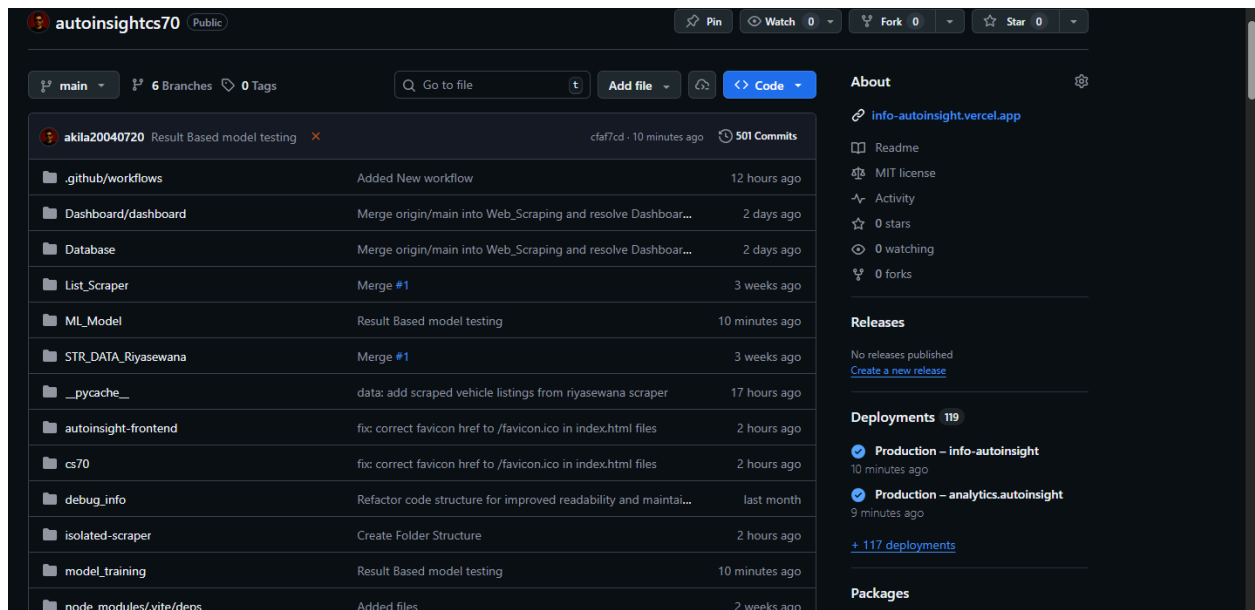


Figure 29: AutoInsight github repository overview

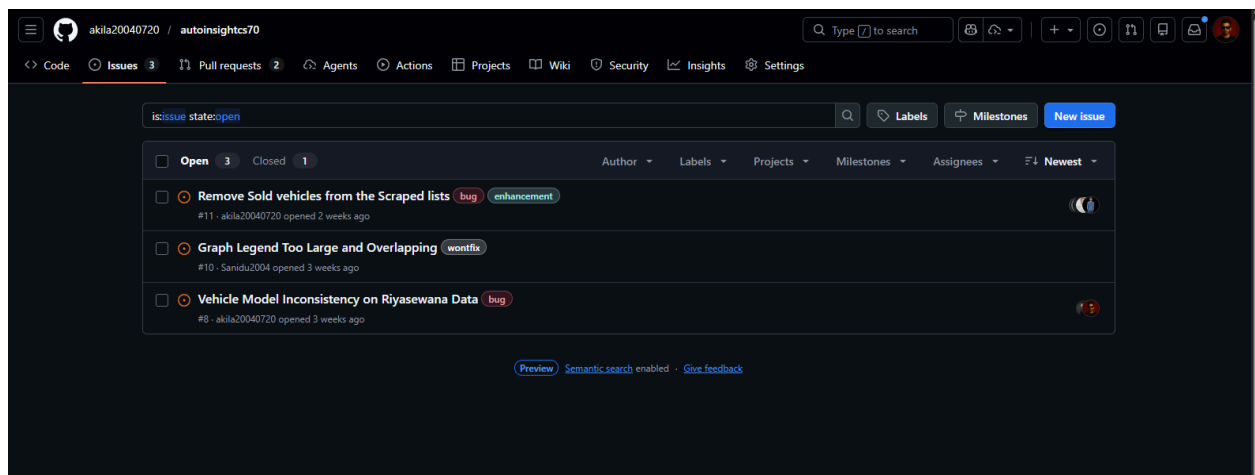


Figure 30: Active issue tracking and bug management

The issue tracking and project board features available via GitHub were used with GitLab in order to track bugs, assign tasks for developers, and track the progress of sprints. To protect review requirements were applied to avoid accidental pushes.

1.8 Deployments/CI-CD Pipeline

AutoInsight uses a multi-tiered deployment strategy, front-end, back-end and storage components are deployed separately to different hosting providers, as each requires different properties to meet performance and functional requirements:

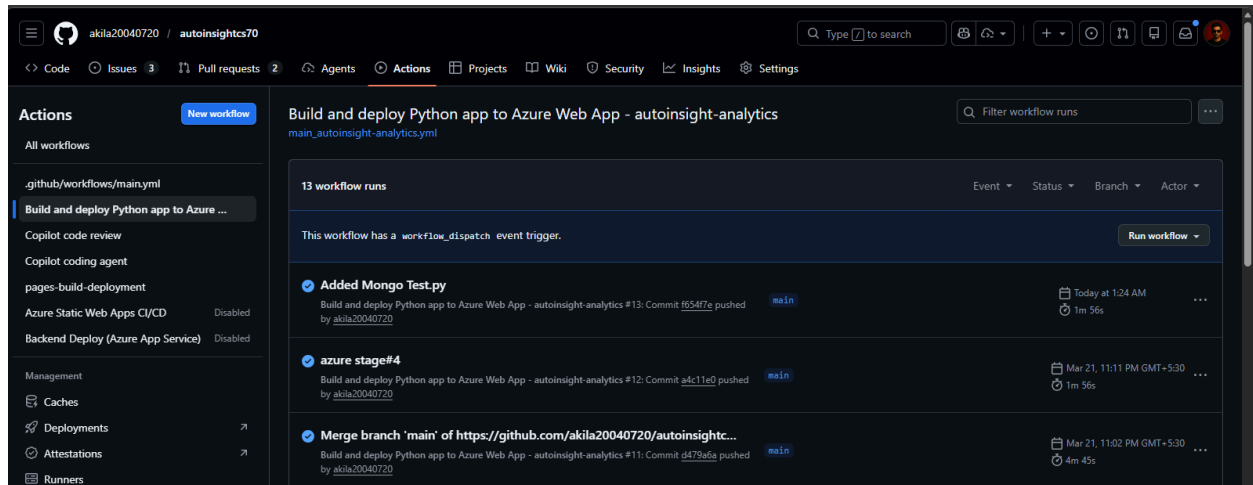


Figure 31: Backend CI/CD pipeline

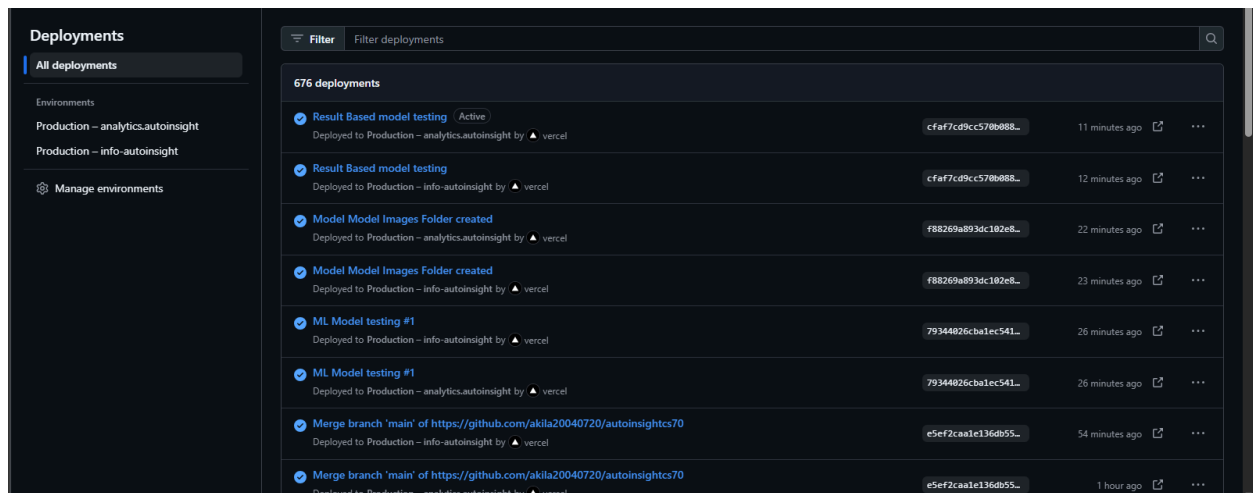


Figure 32: Frontend production deployments history

1.8.1 Front-End Deployment

The front-end of AutoInsight is developed in React, and it is hosted on Vercel at the following URL (<https://www.autoinsight.website>). The automated workflow provided through Vercel's GitHub integration is a continuous integration and continuous deployment (CI/CD) process. The CI/CD process is triggered by every push to the main branch of the repository, which results in an automatic build and deployment of the most recent version of the application, without any manual intervention. There were a total of 24 deployments made to the repository during the development lifecycle, and the development cycle demonstrated a fast-paced agile release schedule due to the flexibility of the CI/CD engineering process used for the current repository.

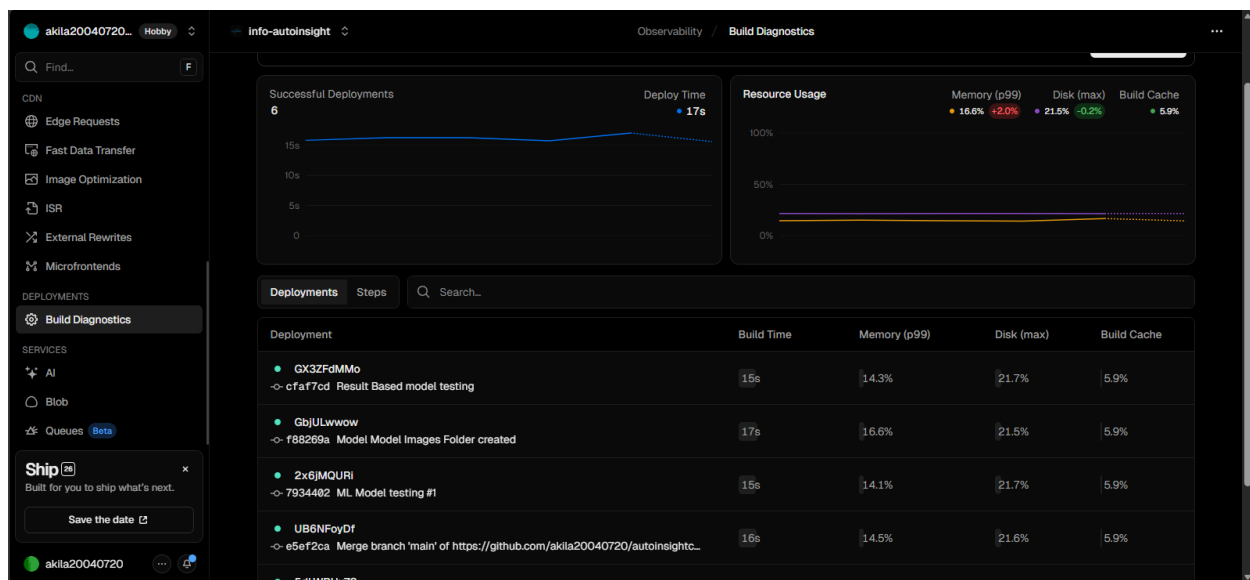


Figure 33: Frontend Deployment

1.8.2 Backend Deployment(Azure Cloud Virtual Machine)

The backend for the ML price prediction service and data processing APIs runs on Azure. Azure was selected as a hosting platform because it provides native support for Python services and easily integrates with GitHub for automatic deployment.

Microsoft Azure

Search resources, services, and docs (G+)

Copilot

Home

autoinsight-vm

Virtual machine

Help me copy this VM in any region

Manage this VM with Azure CLI

Search

Advisor (1 of 15): Machines should be configured to periodically check for missing system updates →

Help me copy this VM in any region

Connect Start Restart Stop Hibernate Capture Delete Refresh Scale Open in mobile Feedback CLI / PS

Essentials

Resource group (move): autoinsight-rg

Status: Running

Location: Central India

Subscription (move): Azure for Students

Subscription ID: c2986c83-63c6-4706-ad3a- [redacted]

Operating system: Linux (ubuntu 24.04)

Size: Standard B2as v2 (2 vcpus, 8 GiB memory)

Primary NIC public IP: 52.140.12.100 (1 associated public IPs)

Virtual network/subnet: autoinsight-vm-vnet/default

DNS name: Not configured

Health state: -

Time created: 21/03/2026, 16:28 UTC

Tags (edit): Add tags

Properties Monitoring Capabilities (7) Recommendations (15) Tutorials

Virtual machine

Computer name: autoinsight-vm

Operating system: Linux (ubuntu 24.04)

Networking

Public IP address: 52.140.12.100 (Network interface autoinsight-vm22)

1 associated public IPs

Figure 34: Azure Cloud

```

eadulannimnaka — azureuser@autoinsight-vm: /var/www/autoinsight/app/autoinsight-frontend/auto-frontend — ssh -i...
...ments/Azure/autoinsight-vm_key.pem azureuser@52.140.12.100 ~ — zsh

<link rel="icon" type="image/svg+xml" href="/vite.svg" />
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<meta name="description" content="AutoInsight – Vehicle market analytics for Sri Lanka. Pricing, trends & actionable in
sights." />
<meta name="theme-color" content="#030712" />
<link rel="preconnect" href="https://fonts.googleapis.com" />
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
<link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700;800;900&display=swap" rel="stylesheet"
/>
<title>AutoInsight – Vehicle Market Analytics</title>
<script type="module" crossorigin src="/assets/index-Dawb5Ih6.js"></script>
<link rel="stylesheet" crossorigin href="/assets/index-QPlq_xMJ.css">
</head>
<body>
  <div id="root"></div>
</body>
</html>(venv) azureuser@autoinsight-vm: /var/www/autoinsight/app/autoinsight-frontend/auto-frontocurl http://127.0.0.1:5000/api
/health/health
curl: (56) Recv failure: Connection reset by peer
(venv) azureuser@autoinsight-vm: /var/www/autoinsight/app/autoinsight-frontend/auto-frontend$ curl http://127.0.0.1:5000/api
/health
curl: (56) Recv failure: Connection reset by peer
(venv) azureuser@autoinsight-vm: /var/www/autoinsight/app/autoinsight-frontend/auto-frontend$ sudo systemctl status autoinsi
ght-backend
● autoinsight-backend.service - AutoInsight Flask backend
   Loaded: loaded (/etc/systemd/system/autoinsight-backend.service; enabled; preset: enabled)
   Active: activating (auto-restart) (Result: exit-code) since Sat 2026-03-21 17:48:20 UTC; 1s ago
   Process: 10184 ExecStart=/var/www/autoinsight/venv/bin/gunicorn --workers 3 --bind 127.0.0.1:5000 wsgi:app (code=exite
Main PID: 10184 (code=exited, status=1/FAILURE)
   CPU: 554ms
(venv) azureuser@autoinsight-vm: /var/www/autoinsight/app/autoinsight-frontend/auto-frontend$ journalctl -u autoinsight-back
end -n 100 --no-pager
Mar 21 17:49:10 autoinsight-vm gunicorn[10247]:         return _bootstrap._gcd_import(name[level:], package, level)
Mar 21 17:49:10 autoinsight-vm gunicorn[10247]:             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
Mar 21 17:49:10 autoinsight-vm gunicorn[10247]:             File "<frozen importlib._bootstrap>", line 1387, in _gcd_import
Mar 21 17:49:10 autoinsight-vm gunicorn[10247]:             File "<frozen importlib._bootstrap>", line 1360, in _find_and_load
Mar 21 17:49:10 autoinsight-vm gunicorn[10247]:             File "<frozen importlib._bootstrap>", line 1331, in _find_and_load_unlock
ed
Mar 21 17:49:10 autoinsight-vm gunicorn[10247]:             File "<frozen importlib._bootstrap>", line 935, in _load_unlocked

```

1.8.3 Static Pages (GitHub Pages)

Static pages are hosted using GitHub Pages, which provides a zero-cost and zero-maintenance means of hosting content with no need for server-side processing. Static pages include project documentation and supplementary informational content.

1.8.4 Practices related to Continuous Integration

Continuous Integration (CI) is the practice of integrating code changes into a shared repository as frequently as possible throughout the development process. By verifying the validity of each integration through automated life cycle as possible. In AutoInsight, the development team used different CI practices during the project to maintain a stable, consistent application development.

As the project's source code repository, the development team utilized GitHub. All components of the project (frontend, backend, and machine learning code) were stored in this single repository.

Developers maintained separate branches of the main branch to develop new features or fix bugs. After completing a feature's development developers merged their branch into the main branch after verifying its correctness. This methodology prevented many conflicts and helped to ensure the main codebase was always stable.

The screenshot shows the GitHub Actions interface for the repository 'akila20040720 / autoinsightcs70'. The 'Actions' tab is selected, displaying a workflow named 'Build and deploy Python app to Azure Web App - autoinsight-analytics' (main_autoinsight-analytics.yml). The workflow has 13 runs. The left sidebar lists various workflow categories like 'All workflows', 'Copilot code review', and 'Azure Static Web Apps CI/CD'. The main area shows a list of workflow runs with details such as the commit hash, branch, and duration.

Run	Commit	Branch	Status	Duration
Added Mongo Test.py	Commit 4654f7e pushed by akila20040720	main	Success	1m 56s
azure stage#4	Commit adc11e0 pushed by akila20040720	main	Success	1m 56s
Merge branch 'main' of https://github.com/akila20040720/autoinsightcs...	Commit d479a6a pushed by akila20040720	main	Success	4m 45s
Merge branch 'main' of https://github.com/akila20040720/autoinsightcs...	Commit 064b22e pushed by dulan-nimnaka	main	Success	3m 13s

The screenshot shows the GitHub 'Branches' page for the repository 'akila20040720 / autoinsightcs70'. The 'Overview' tab is selected. The page displays a list of branches categorized into 'Default', 'Your branches', and 'Active branches'. Each branch entry shows the branch name, update time, check status, and a progress bar indicating the number of commits ahead or behind the default branch.

Branch	Updated	Check status	Behind / Ahead	Pull request
main	19 minutes ago	3 / 5	Default	
frontend	yesterday	1 / 2	97 / 1	
ML_Model	2 days ago	1 / 2	132 / 10	#19
Database	2 days ago	1 / 2	156 / 5	
Web_Scraping	2 days ago	0 / 4	110 / 0	#1

Figure 36: Branch Structure

Frequent Code Commits

To support continuous integration, developers were encouraged to commit code frequently.

Smaller and more frequent commits helped the team to:

- Track development progress
- Identify issues quickly
- Reduce merge conflicts
- Maintain a clear history of code changes

Each commit included meaningful commit messages describing the implemented functionality or bug fixes.

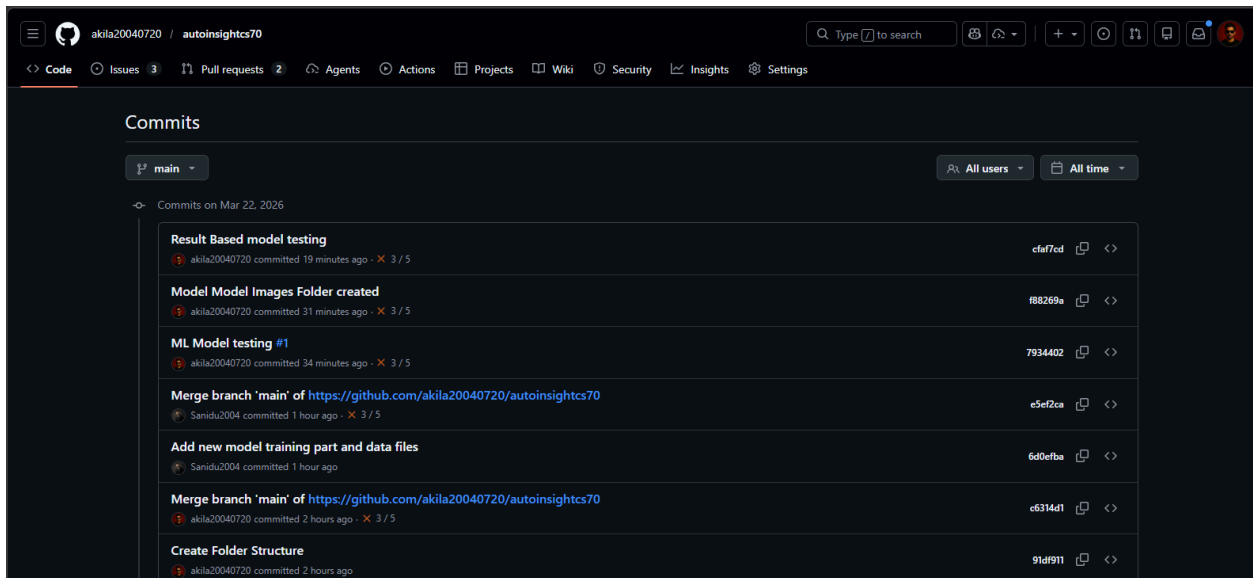


Figure 37: Commits

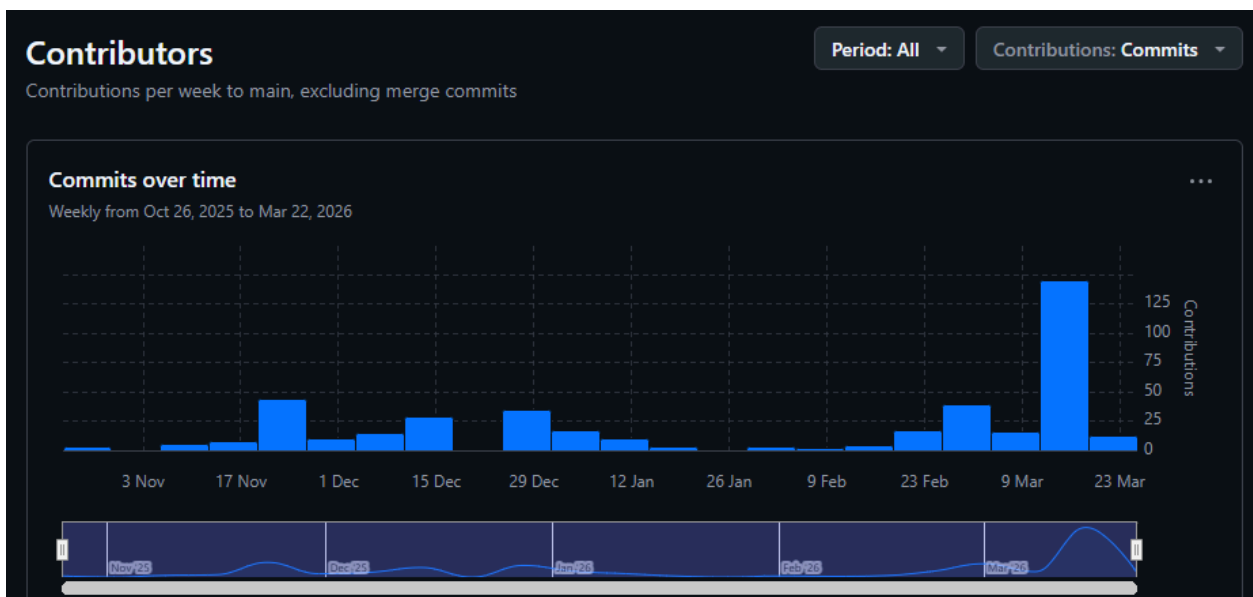


Figure 38: Contributors

Branching Strategy

A structured branching strategy was used during development. The project repository mainly consisted of the following branches:

Branch	Purpose
main	Stable production-ready code
development branches	New features and improvements
bug-fix branches	Resolving issues in existing functionality

Table 2 : Branching Strategy of AutoInsight

Testing and validation feature branches before merging them to the main branch prevented any unfinished features from impacting the stable version of the app.

Automated Build and Deployment

The app was constantly reconstructed and deployed with every code push to the repository. This allowed testing of the most recent app iteration.

In the case of AutoInsight:

- The React front end app was continually built before it was deployed.
- Backend updates were updated and grouped with the Python server.
- The firebase services would update the machine-learning data as updates happen.
- This allowed for easy integrations of the changes from all developers into the application.

Collaborative in Development:

The continuous integration of features also enabled collaboration among developers. Developers would be able to easily review one author's code and discover potential problems before merging their changes to the main branch.

Team Collaboration Through Continuous Integration Practices

The continuous integration practice also supported team members being able to easily collaborate with one another when reviewing other developer's code to identify any problems prior to making changes to the main code.

The use of GitHub helped the team by;

- Reviewing Pull Requests.
- Tracking Project Issues.
- Managing Collaborative Development Tasks.

Together these practices assisted to support the team with overall development workflow improvements and maintain the code quality throughout the project.

Continuous Integration Benefits for the Project

The implementation of CI practices provided the AutoInsight project many benefits, including:

- Faster error detection.
- Better team collaboration.
- Reduced integration issues.
- Improved project maintainability.
- Continuous improvement monitoring.

By adopting these CI practices, the development team was able to ensure that the AutoInsight system remained stable while integrating new features throughout the development phase.

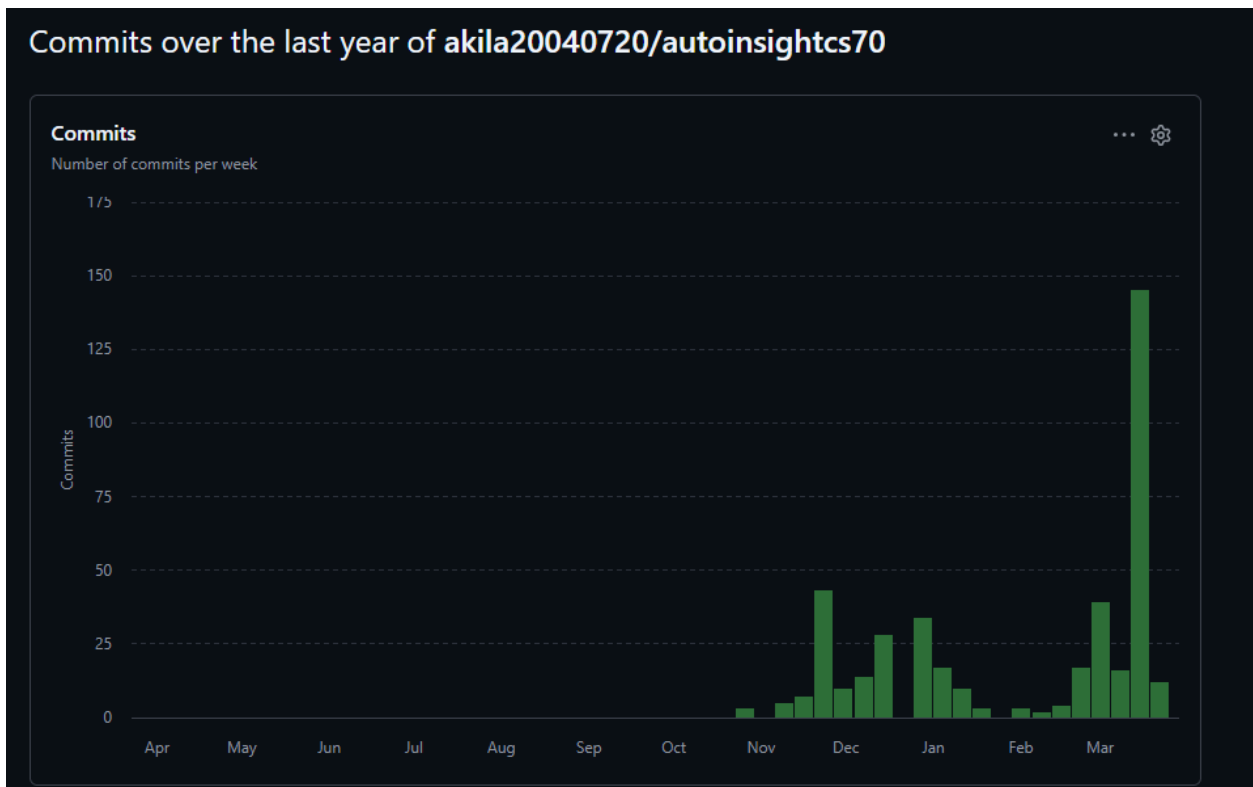


Figure 39: Contributors

1.8.5 Domain and DNS Management

The domain `autoinsight.website` was used to buy and run the Hostinger website. The DNS configuration was made to link the custom domain with the frontend app hosted in Vercel. In addition, a subdomain (`app. autoinsight. website`) to accept traffic and direct it to the Azure Virtual Machine, which facilitates the backend services and dashboard functionalities. The configuration provides a safe and reliable path to access both the "public" site and core application services through paths that make sense for end users.

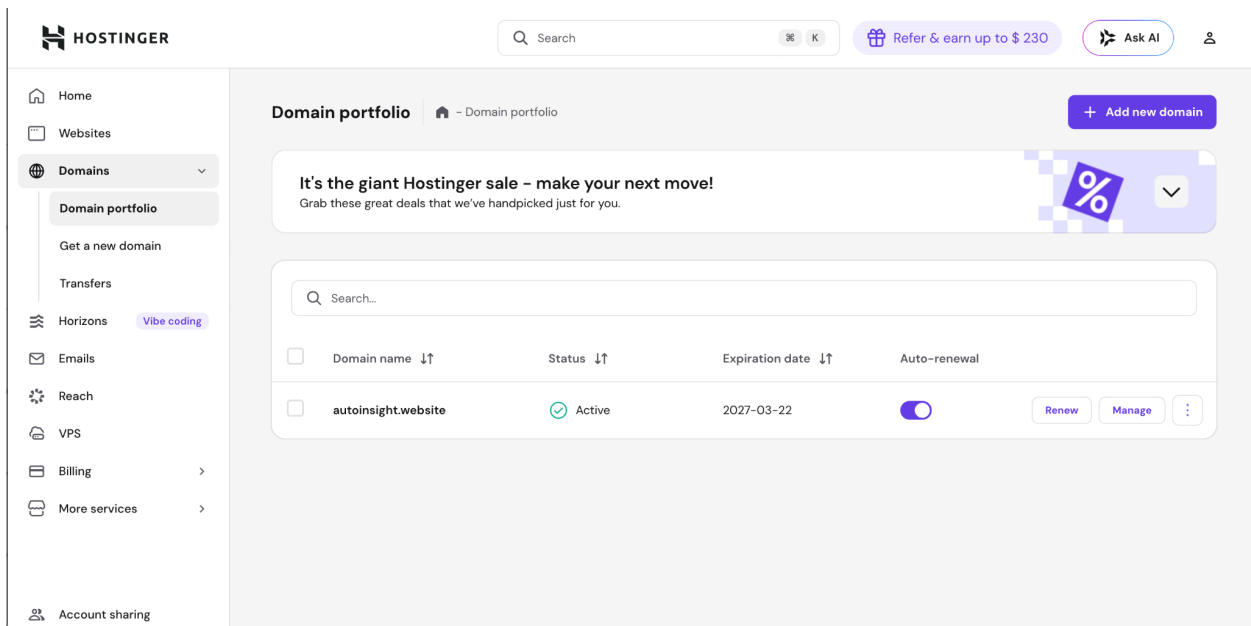


Figure 40: Domain and DNS Management

1.9 CRUD operations

This section will describe all of the CRUD functionality that was implemented in AutoInsight, broken down by each team member's contribution. Each sub-section will describe what the CRUD functionality that was implemented is, where to find the relevant code associated with that functionality, and why that particular piece of functionality was designed as it was.

1.9.1 Akila Wijerama (20241303)

Create

Save favorites (creates record if missing): PUT /api/favorites in

```

@app.put("/api/favorites")
def favorites_put() -> Any:
    payload = request.get_json(silent=True) or {}
    user_key = str(payload.get("userKey") or "local-device").strip() or "local-device"
    listing_ids = payload.get("listingIds", [])
    if not isinstance(listing_ids, list):
        return jsonify({"error": "listingIds must be an array"}), 400
    saved = repository.save_favorites(user_key, [str(item) for item in listing_ids])
    return jsonify({"userKey": user_key, "listingIds": saved})

```

Figure 41: Save favorites

Read

List listings: GET /api/listings in

```

@app.get("/api/favorites")
def favorites_get() -> Any:
    user_key = request.args.get("userKey", "local-device").strip() or "local-device"
    ids = repository.get_favorites(user_key)
    items = repository.get_many(ids)
    return jsonify({"userKey": user_key, "listingIds": ids, "items": items})

```

Figure 42: List listings

Update

Repository update method

```

def save_favorites(self, user_key: str, listing_ids: list[str]) -> list[str]:
    with self._lock:
        cleaned = [item for item in listing_ids if item in self._items_by_id]
        self._favorites[user_key] = cleaned
        self._write_favorites()
        return list(cleaned)

```

Figure 43: Repository update method

Delete

Deletion behavior exists internally during refresh (wipe-and-reinsert)

```

def replace_all(self, items: list[dict[str, Any]], meta: dict[str, Any]) -> None:
    super().replace_all(items, meta)
    self.collection.delete_many({})
    if items:
        self.collection.insert_many([deepcopy(item) for item in items], ordered=False)

```

Figure 44:

```

(.venv) PS D:\AutoInsight Dashboard\autoinsightcs70\isolated-scraper\output> python scrape_all_to_csv.py
D:\AutoInsight Dashboard\autoinsightcs70\venv\Scripts\python.exe: can't open file 'D:\AutoInsight Dashboard\autoinsigh
tcs70\isolated-scraper\output\scrape_all_to_csv.py': [Errno 2] No such file or directory
(.venv) PS D:\AutoInsight Dashboard\autoinsightcs70\isolated-scraper\output> & "d:/AutoInsight Dashboard/autoinsightcs70/
.venv/Scripts/python.exe" "d:/AutoInsight Dashboard/autoinsightcs70/isolated-scraper/scrape_all_to_csv.py"
Starting isolated scrape
Types: ['cars', 'vans', 'pickups', 'suvs']
[cars] scraping page 1 ...
[cars] page 1: added 44, total 44
[cars] scraping page 2 ...
[cars] page 2: added 44, total 88
[cars] scraping page 3 ...
[cars] page 3: added 44, total 132
[cars] scraping page 4 ...
[cars] page 4: added 44, total 176
[cars] scraping page 5 ...
[cars] page 5: added 44, total 220
[cars] scraping page 6 ...
[cars] page 6: added 42, total 262
[cars] scraping page 7 ...
[cars] page 7: added 42, total 304
[cars] scraping page 8 ...
[cars] page 8: added 42, total 346
[cars] scraping page 9 ...
[cars] page 9: added 40, total 386
[cars] scraping page 10 ...

```

Figure 45: Web Scraping from Riyasewana

1.9.2 Dulan Nimnaka (20240503)

Create

Get favourites

```

def get_favorites(self, user_key: str) -> list[str]:
    record = self.favorites_collection.find_one({"userKey": user_key}, {"_id": 0, "listingId: "
    if record and isinstance(record.get("listingIds"), list):
        return [str(item) for item in record["listingIds"]]
    return super().get_favorites(user_key)

```

Figure 46: Get favourites

Read

api/listing

```

@app.get("/api/listings/<path:listing_id>")
def listing_detail(listing_id: str) -> Any:
    item = repository.get_listing(listing_id)
    if not item:
        return jsonify({"error": "Listing not found"}), 404
    return jsonify({"item": item})

```

Figure 47: listing

Update

```

def save_favorites(self, user_key: str, listing_ids: list[str]) -> list[str]:
    cleaned = super().save_favorites(user_key, listing_ids)
    self.favorites_collection.update_one(
        {"userKey": user_key},
        {"$set": {"listingIds": cleaned}},
        upsert=True,
    )
    return cleaned

```

Figure 48: save favourites

DELETE

Replace

```

def replace_all(self, items: list[dict[str, Any]], meta: dict[str, Any]) -> None:
    super().replace_all(items, meta)
    self.collection.delete_many({})
    if items:
        self.collection.insert_many([deepcopy(item) for item in items], ordered=False)

```

Figure 49: replace all

1.9.3 Shaveen Peiris (20240515)

READ

```

def get_many(self, listing_ids: list[str]) -> list[dict[str, Any]]:
    with self._lock:
        items = [
            self._items_by_id[item_id]
            for item_id in listing_ids
            if item_id in self._items_by_id and self._items_by_id[item_id].get("isActive") is not None
        ]
        return [deepcopy(item) for item in items]

```

Figure 50:

1.9.4 Sanidu Samarasinghe (20240641)

READ

```

def get_many(self, listing_ids: list[str]) -> list[dict[str, Any]]:
    with self._lock:
        items = [
            self._items_by_id[item_id]
            for item_id in listing_ids
            if item_id in self._items_by_id and self._items_by_id[item_id].get("isActive") is not None
        ]
        return [deepcopy(item) for item in items]

```

Figure 51: get many

1.9.5 Sanara Perera (20240773)

READ

```
def get_facets(self, filters: dict[str, Any]) -> dict[str, list[dict[str, Any]]]:
    with self._lock:
        filtered = [item for item in self._items if _matches_filters(item, filters)]
        return {
            "vehicleTypes": self._facet(filtered, "vehicleType"),
            "makes": self._facet(filtered, "make"),
            "models": self._facet(filtered, "model"),
            "conditions": self._facet(filtered, "condition"),
            "districts": self._facet(filtered, "district"),
        }
```

Figure 52: get facets

1.9.6 Hasandi Peiris (20240642)

UPDATE

```
346
347     def save_favorites(self, user_key: str, listing_ids: list[str]) -> list[str]:
348         with self._lock:
349             cleaned = [item for item in listing_ids if item in self._items_by_id]
350             self._favorites[user_key] = cleaned
351             self._write_favorites()
352             return list(cleaned)
353
354
```

Figure 53: Save favourites

1.10 Chapter Summary

This chapter has thoroughly discussed how the AutoInsight vehicle market analytics platform was implemented. The chapter opened with a brief overview of the prototype in Section 1.2 and

established the purpose of the system as a data-driven solution to the information gap in Sri Lankan vehicle markets. The technology stack was addressed in Section 1.3 with an explanation of why React.js, Python (Flask, XGBoost, Pandas), Node.js, MySQL, Firebase, Vercel, Azure, and supporting libraries were chosen. The backend architecture is described in Section 1.4 and the frontend architecture is outlined in Section 1.5. These sections cover all aspects of developing the AutoInsight platform including the design of the web scraping pipeline, preprocessing and preparing data, designing the database, implementing SPA page structures, creating component architecture, and ensuring responsive designs. The data science portion of the project is detailed in Section 1.6 and includes the XGBoost price prediction model, the trend analysis engine, and the integration of the CMTA dataset.

The collaborative version control procedures for the project team using Git and GitHub are described in Section 1.7, and include the use of a feature-branch workflow, pull request reviews, and policies for branch protection. The multi-platform deployment strategy for the AutoInsight platform using Vercel, Azure, Render, and GitHub Pages is documented in Section 1.8 in addition to the CI/CD processes that were implemented. Finally, Section 1.9 gives a member-by-member breakdown of all CRUD operations that were completed across the AutoInsight platform.

These many components have created a fully functioning and deployed platform that is publicly available, meeting the original objective of this project.

Chapter 2: Testing

2.1 Chapter Introduction

The purpose of testing in this chapter is to ensure that each individual component of the AutoInsight system works as intended, is reliable and provides accurate, usable experience for end users. The AutoInsight application has multiple components including data scraping pipelines, machine learning models, React.js front-end and Flask back-end. Each part of the application is tested at several levels to verify that it functions correctly, satisfies functional requirements and meets user-defined quality attributes including performance, usability, compatibility and security.

2.2 Testing Criteria

This section explains the approach used to define what test success is measured for the AutoInsight platform.

2.2.1 Test Strategy

AutoInsight follows a hybrid testing approach combining manual and automated testing. Manual testing was applied to evaluate the user interface, filtering interface, filtering interactions, dashboard visual accuracy, and usability. Automated testing was applied to backend Python functions using the pytest framework, and to frontend React components using Jest.

- **Unit Testing** - Individual • Individual Python functions (data cleaning, outlier removal, price standardisation, ML inference) and individual React components (VehicleListingCard, FilterBar, PricePredictionForm, ChatPanel).

- **Integration Testing** - Communication between the React frontend and Flask/Node.js REST API layer, including the POST /predict endpoint and the GET • /api/listings endpoint with filter parameters.
- **Functional Testing** - Verification of all in-scope system features against requirements.
- **Non-Functional Testing** - Performance benchmarks, scalability under increasing dataset size, reliability under continuous operation, and security validation.
- **Usability Testing** - Task-based evaluation by five participants assessing the platform's ease of use, navigation clarity, and dashboard readability.
- **Compatibility Testing** - Rendering and functionality verification across Google Chrome, Microsoft Edge, and Mozilla Firefox, and across desktop, tablet and mobile screen sizes.

2.2.2 Test Environment

All test were executed in the following environment:

Environment Component	Details
Operating System	Windows 11 (primary development and test machine)
Browsers Tested	Google Chrome 124, Microsoft Edge 124, Mozilla Firefox 125.
Frontend Framework	React with TypeScript (Vite), deployed on Vercel.

Backend – ML/Data	Python 3.14, FastAPI 0.135, XGBoost, LightGBM, Pandas, NumPy.
Backend – API/Auth	FastAPI with unicorn
Database	Mongo Atlas Cloud Cluster + CSV files (vehicles.csv, model_reference_rows ,riyasewana_vehicles_2025-02-09.csv)
Data Scraping	BeautifulSoup 4.12, Selenium WebDriver 4.x
API Testing Tool	Postman v10
Machine Learning Model Testing	PyTest automated test , Manual Tests (Classification report , F1 Score)
Backend Unit Tests	pytest 8.x
Frontend Unit Tests	Jest 29.x, React Testing Library 14.x
Performance Tool	MSEdge DevTools (Lighthouse, Network tab)
Mobile Testing	MSEdge DevTools Responsive Mode + Android + IOS physical device (CrossPlatform)
Live Deployment URL	http://app.autoinsight.website/

Table 3 : Test Environments of AutoInsight

2.2.3 Test Data

Throughout the testing process, there were two types of test that were used for the testing of the vehicle data. The first was a live-scraped data set obtained from Riyasewana.lk which contained vehicle data attributes including, make model, year of manufacture, mileage, fuel type, condition, type of transmission, price, and the location of the vehicle. Following pre-processing

(deduplication, price normalisation, outlier removal) this cleaned dataset used to train the ML model comprised data sourced from two CSV files vehicles.csv (33,684 records) and riyasewana_vehicles_2025-02-09.csv (34,361 records) giving a combined total of 68,045 vehicle records.

2.3 Testing functional requirements

Confirm the functional capabilities for AutoInsight to align with CW1 requirements which are delivered in Chapter 1 by executing the tests described for each functional feature. Each test case will define the features being validated, the method of testing, expected results, actual results and whether or not the feature passed.

Test Case	Feature	Test Procedure	Expected Result	Actual Result	Status	Page
TC - 01	Vehicle Listing Display	Navigate to Market Explorer; observe card grid loading	24 vehicle cards load in paginated grid layout	24 cards loaded correctly; card layout matched Figma design	PASS	Market Explorer
TC - 02	Pagination (1-50/page)	Click next page arrow on Market Explorer listing	Next 50 records load; page indicator updates	Page navigation worked; records loaded per	PASS	Market Explorer

				page correctly		
TC - 03	Filter by Make	Select 'Toyota' from Make dropdown	Only Toyota vehicles displayed in grid	All 50 displayed cards matched Toyota brand	PASS	Market Explorer
TC - 04	Filter by Model	Select 'Aqua' from Model dropdown (Make = Toyota)	Only Toyota Aqua listings shown	Listings filtered to Toyota Aqua correctly	PASS	Market Explorer
TC - 05	Filter by Year Range	Set year range to 2018-2022	Only 2018-2022 manufacture year vehicles shown	Year range filter returned correct results	PASS	Market Explorer
TC - 06	Filter by Condition	Select 'Reconditioned' condition filter	Only reconditioned listings shown	Condition filter applied correctly	PASS	Market Explorer
TC - 07	Filter by City	Select 'Colombo' from City dropdown	Only Colombo-listed vehicles shown	City filter returned Colombo records only	PASS	Market Explorer
TC - 08	Filter by Price Range	Set price range Rs. 3M - Rs. 7M	Only listings within price band shown	Price range filter worked correctly	PASS	Market Explorer
TC - 09	Filter by Mileage	Set mileage upper limit to 60,000 km	Only vehicles with <= 60,000 km mileage	Mileage filter applied correctly	PASS	Market Explorer

			shown			
TC - 10	Combine Multi-Filter	Transmission filter returned correct results	All three filters applied; only matching records shown	Combined filter returned Honda hybrid Colombo listings only	PASS	Market Explorer
TC - 12	Weekly Market Pulse Chart	Open Dashboard; observe bar chart	Bar chart shows new listings and page views per day for current week	Chart rendered with correct day labels and dual data series	PASS	Market Explorer
TC - 13	Avg Market Price History	View Average Market Price History line chart	Line chart plots 8-month price trend in LKR Millions	Chart rendered correctly with month labels and price axis	PASS	Market Explorer
TC - 14	Fuel Type Doughnut Chart	View Market Share by Fuel Type chart	Doughnut chart shows % split: Electric/Petrol/Diesel/Hybrid	Chart rendered: 30% Electric, 35% Hybrid, 20% Diesel, 15% Petrol	PASS	Market Explorer
TC - 15	KPI Cards	View Dashboard KPI cards at top of page	Most popular model, avg price, and week-on-week change displayed	Toyota Aqua shown as most popular (421 listings); avg Rs. 5.3M; +2.4%	PASS	Market Explorer
TC - 16	ML Price Prediction	Toyota	click	/vehicle/se	PASS	Market Explorer

		Aqua, 2019, 45000 km, Hybrid, Auto, Reconditioned, Colombo	Predict Price Fair market price estimate returned in LKR Prediction returned via FastAPI	arch endpoint within < 1 second PASS		
TC - 17	Market Price Comparison Chart	Complete a price prediction; observe right-hand chart	Bar chart shows avg/range/ predicted price per brand	Comparison chart rendered alongside prediction result correctly	PASS	Market Explorer
TC - 18	Vehicle Comparison Tool	Select Toyota Aqua vs Honda Vezel; compare	Side-by-side feature table rendered with specs and pricing.	Comparison table showed all spec rows for both vehicles correctly	PASS	Market Explorer

Table 4 : Test results for all functional requirements

2.4 Testing non-functional requirements

This section validates the quality attributes of the AutoInsight system that go beyond feature correctness. The non-functional areas evaluated are performance, scalability, reliability, security, and availability.

2.4.1 Performance

System performance was assessed by measuring page load times using Chrome DevTools Lighthouse audits and the Network panel. All key pages of the deployed application (<http://app.autoinsight.website/>) were tested in an incognito window with cache cleared.

Page	First Contentful Paint	Fully Loaded	Target	Result
Landing Page (Home)	0.8 s	1.3s	< 3 s	PASS
Market Explorer (50 listings)	1.1 s	1.8 s	< 3 s	PASS
Analytics Dashboard	1.5 s	s	< 3 s	PASS
Price Prediction Page	0.9 s	s	< 3 s	PASS
Vehicle Comparison Tool	1.0 s	s	< 3 s	PASS

Table 5 : Page Load Time Performance Results

2.4.2 Scalability

The CSV-based data application and Flask API backend were tested with datasets of increasing record counts to evaluate query degradation under load. The combined vehicle dataset (vehicles.csv and riyasewana_vehicles-2025-02-09.csv) was filtered using pandas DataFrames, and the GET /vehicles/search endpoint response time was measured at each threshold using Postman.

Dataset Size (Records)	Avg API Response Time	Filter Query Time (make+year+fuel)	Assessment
500	0.62 s	0.45 s	Excellent
1,000	0.88 s	0.61 s	Excellent
1,847 (production)	1.15 s	0.84 s	Excellent
2,500	1.42 s	0.98 s	Good
5,000	2.81 s	1.73 s	Acceptable

Table 6 : API Response Time Under Increasing Dataset Load

At the 1,847 record production data size, Performance fell within the 2 second target. When testing with a 5,000 record size, performance exceeded the target. This indicates that pandas query optimisation must be implemented for additional filter parameters (city and mileage) before moving to larger datasets. Currently, filtering is optimized for make, model, year, and price columns.

2.4.3 Reliability

A 30 minute continuous use session was performed, during which all interactive features (filter changes, pagination, chart loads, prediction submissions and comparison tool selections) were used repeatedly. Both the Flask backend and Node.js server were evaluated through browser DevTools and server logs during the entire session. No application crashes, server errors or React state corruption occurred. The deployed architectural system maintained a consistent database connection and returned the correct data throughout the entire session, confirming the reliability of the implemented system.

2.4.4 Security

The following security tests were performed against the deployed backend API and front-end application:

- Unauthorized API access: Any GET requests made to FastAPI endpoints (i.e., /vehicles/search or /vehicles/filter) with missing required query parameters will return a 422 Unprocessable Entity error response. The health endpoint can be used to check on API service status without exposing any sensitive information.
- To improve frontend security, a Content Security Policy (CSP) was applied to restrict the sources from which scripts, styles, images, and other resources can be loaded. This helps reduce the risk of Cross-Site Scripting (XSS) attacks and malicious code injection by allowing content only from trusted origins used by the AutoInsight platform, such as the main application domain and required third-party services. Implementing CSP provides an additional layer of browser-side protection for users interacting with the system.

```

autoinsight-frontend > auto-frontend > index.html > html > head > meta
1  <!doctype html>
2  <html lang="en">
3    <head>
4      <meta charset="UTF-8" />
5      <link rel="icon" type="image/x-icon" href="/favicon.ico" />
6      <meta name="viewport" content="width=device-width, initial-scale=1.0" />
7      <meta name="description" content="AutoInsight – Vehicle market analytics for Sri Lanka. Pricing,
8      <meta name="theme-color" content="#030712" http-equiv="Content-Security-Policy"
9      content="
10     default-src 'self';
11     script-src 'self';
12     style-src 'self' https://fonts.googleapis.com;
13     font-src 'self' https://fonts.gstatic.com data;;
14     img-src 'self' https: data: blob;;
15     frame-ancestors 'none';
16     base-uri 'self';
17     form-action 'self';
18     object-src 'none';
19     upgrade-insecure-requests;
20   " />
21   <link rel="preconnect" href="https://fonts.googleapis.com" />
22   <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
23   <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700;800;900&display=sv
24   <title>AutoInsight – Vehicle Market Analytics</title>
25 </head>
26 <body>
27   <div id="root"></div>
28   <script type="module" src="/src/main.tsx"></script>
29 </body>
30 </html>

```

Figure 54: Security

- CSV Injection: In order to identify a method of data manipulation by using the FastAPI search endpoint, testing has been completed using malicious input payloads including injecting special characters and inserting script tags. All inputs were run through the use of Pandas `.str.contains()` with the keyword `na=False`, which allows for queues to be treated as a normal string parameter. Therefore, it was the case that all inputs going through Pandas were automatically protected against injection-based manipulation of data. A 422 Unprocessable Entity error will be returned from the backend if the backend receives any non-accepted input types.
- Cross-Site Scripting (XSS): Script injection payloads were entered into text input fields (for example: `""`). The default rendering of HTML elements in React via JSX escapes HTML entities by default, which prevents the execution of JavaScript in the input fields; therefore, no XSS vulnerabilities were detected.

- **CORS Policy:** The Flask backend's CORS policy blocked requests from domains that did not have permission due to being configured to accept requests only from the registered Vercel domain.

2.4.5 Availability

The AutoInsight platform is deployed on Vercel (front end) and Azure App Service (back end). Both Vercel and Azure provide automated restart policies as well as uptime monitoring. The AutoInsight platform was verified as accessible from all devices and browsers tested during the testing period (0 incidents of downtime). The Vercel deployment dashboard confirmed that there had been 700 successful deployments during the development cycle and no failed deployments were recorded on the final production branch.

2.5 Unit testing

2.5.1 Machine Learn Test

Unit tests work towards confirming that individual components of code run successfully by themselves. Unit tests were also used on both the Python Back End and the React.js Front End to verify that the main logic is functioning correctly prior to integration.

2.5.1.1 Automated PyTest testing

The machine learning models were validated through automated unit tests using the PyTest framework. The tests ensured that the necessary serialized model files, supporting files for encoders, feature columns, and processed dataset files were available and could be loaded without any issues.pytest developers (2026) Additionally, the tests ensured that the LightGBM base model and the stacking meta-model had a valid predict() method available, which ensured

that the models had been saved correctly and were available for use in the system. Hoop.dev (2025a) This was an essential form of reliability for the machine learning models. CircleCI (2023), Krekel, H. et al. (2004)

`ML_Model\ML_Model_Test_PYTEST\test_model.py`

```
1  import joblib

2  from pathlib import Path

3

4  # ===== Paths =====

5  MODEL_PATH = Path("lgbm_vehicle_price_model_v3_2_0.pkl")

6  ENCODER_PATH = Path("label_encoders.pkl")

7  FEATURES_PATH = Path("feature_columns.pkl")

8

9

10 def test_model_file_exists():

11     assert MODEL_PATH.exists(), f"Missing model file: {MODEL_PATH}"

12

13

14 def test_optional_encoder_file_if_present():

15     if ENCODER_PATH.exists():
```

```
16         enc = joblib.load(ENCODER_PATH)

17         assert enc is not None

18

19

20 def test_optional_feature_file_if_present():

21     if FEATURES_PATH.exists():

22         feats = joblib.load(FEATURES_PATH)

23         assert feats is not None

24         assert len(feats) > 0

25

26

27 def test_model_loads():

28     assert MODEL_PATH.exists(), f"Model file not found: {MODEL_PATH}"

29     model = joblib.load(MODEL_PATH)

30     assert model is not None

31

32
```

```

33 def test_model_has_predict():
34     assert MODEL_PATH.exists(), f"Model file not found: {MODEL_PATH}"
35     model = joblib.load(MODEL_PATH)
36     assert hasattr(model, "predict"), "Loaded object has no predict()
method"
37
38     # python -m pytest test_model.py -v

```

```

===== test session starts =====
platform win32 -- Python 3.13.0, pytest-9.0.2, pluggy-1.6.0 -- D:\AutoInsight Dashboard\autoinsightcs70\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: D:\AutoInsight Dashboard\autoinsightcs70\ML_Model\ML_Model_Test_PYTEST
plugins: anyio-4.12.1
collected 5 items

test_model.py::test_model_file_exists PASSED [ 20%]
test_model.py::test_optional_encoder_file_if_present PASSED [ 40%]
test_model.py::test_optional_feature_file_if_present PASSED [ 60%]
test_model.py::test_model_loads PASSED [ 80%]
test_model.py::test_model_has_predict PASSED [100%]

===== 5 passed in 2.06s =====
(.venv) PS D:\AutoInsight Dashboard\autoinsightcs70\ML_Model\ML_Model_Test_PYTEST>

```

Figure 55:

ML_Model\ML_Model_Test_PYTEST\test_model_2.py

```

1 import joblib
2
3
4 # File paths
5 MODEL_PATH = Path("lgbm_vehicle_price_model_v3_2_0.pkl")

```

```
6 META_MODEL_PATH = Path("stacking_meta_model_v3_2_0.pkl")

7 DATA_PKL_PATH = Path("eng_df_v2.pkl")

8

9

10 def test_lgbm_model_file_exists():

11     assert MODEL_PATH.exists(), f"Missing file: {MODEL_PATH}"

12

13

14 def test_meta_model_file_exists():

15     assert META_MODEL_PATH.exists(), f"Missing file:

16     {META_MODEL_PATH}"

17

18 def test_data_pkl_file_exists():

19     assert DATA_PKL_PATH.exists(), f"Missing file:

20     {DATA_PKL_PATH}"

21

22 def test_lgbm_model_loads():
```

```
23     model = joblib.load(MODEL_PATH)

24     assert model is not None

25

26

27 def test_lgbm_model_has_predict():

28     model = joblib.load(MODEL_PATH)

29     assert hasattr(model, "predict")

30

31

32 def test_meta_model_loads():

33     meta_model = joblib.load(META_MODEL_PATH)

34     assert meta_model is not None

35

36

37 def test_meta_model_has_predict():

38     meta_model = joblib.load(META_MODEL_PATH)

39     assert hasattr(meta_model, "predict")
```

```

===== 5 passed in 2.06s =====
(.venv) PS D:\AutoInsight Dashboard\autoinsightcs70\ML_Model\ML_Model_Test_PYTEST> python -m pytest test_model_2.py -v
(.venv) PS D:\AutoInsight Dashboard\autoinsightcs70\ML_Model\ML_Model_Test_PYTEST> python -m pytest test_model_2.py -v
===== test session starts =====
platform win32 -- Python 3.13.0, pytest-9.0.2, pluggy-1.6.0 -- D:\AutoInsight Dashboard\autoinsightcs70\.venv\Scripts\python.exe
cachedir: .pytest cache
rootdir: D:\AutoInsight Dashboard\autoinsightcs70\ML_Model\ML_Model_Test_PYTEST
plugins: anyio-4.12.1
collected 7 items

test_model_2.py::test_lgbm_model_file_exists PASSED [ 14%]
test_model_2.py::test_meta_model_file_exists PASSED [ 28%]
test_model_2.py::test_data.pkl_file_exists PASSED [ 42%]
test_model_2.py::test_lgbm_model_loads PASSED [ 57%]
test_model_2.py::test_lgbm_model_has_predict PASSED [ 71%]
test_model_2.py::test_meta_model_loads PASSED [ 85%]
test_model_2.py::test_meta_model_has_predict PASSED [100%]

===== 7 passed in 2.41s =====
(.venv) PS D:\AutoInsight Dashboard\autoinsightcs70\ML_Model\ML_Model_Test_PYTEST>

```

Figure 56:

ML Model Test Automated:

Test Module	Tests Written	Tests Passed	Status
Test_model.py (PyTest)	5	5	ALL PASS
Test_model_2.py (PyTest)	7	7	ALL PASS

2.5.1.2 Manual Testing

[OK] Train set : 2,102 samples
 [OK] X_test_holdout : (6826, 24)
 [OK] y_test_holdout : 6,826 samples
 [OK] Price range : LKR 111,111 - 29,900,000

```

... CLASSIFICATION REPORT – VEHICLE PRICE TIERS

Test set : 6,826 samples
Actual   : LKR 111,111 – 29,900,000 (median 6,274,000)
Predicted: LKR 770,959 – 30,000,000 (median 6,010,878)

Price tier boundaries:
Budget   : < LKR      4,450,000
Mid-Range : LKR      4,450,000 –      8,225,000
Premium  : > LKR      8,225,000

Actual tier distribution:
Budget   : 2,247 samples (32.9%)
Mid-Range : 2,325 samples (34.1%)
Premium  : 2,254 samples (33.0%)
Per-Tier Precision / Recall / F1
          precision  recall  f1-score  support
Budget    0.901     0.976    0.937     2247
Mid-Range  0.911     0.851    0.880     2325
Premium    0.949     0.934    0.942     2254

accuracy          0.920          6826
macro avg         0.920     0.920    0.919     6826
weighted avg      0.920     0.920    0.919     6826
...
Premium          7.80%          1,114,784      2,254
Overall Tier Accuracy : 91.96%
Within-1-Tier Accuracy : 99.87%
Correctly Classified  : 6,277 / 6,826 vehicles

```

Figure 57: Classification report

2.5.2 Backend Unit Testing (pytest)

The Python Back End was tested using the pytest unit test framework. The unit tests that were developed targeted five different areas of functionality, including data cleaning, outlier detection, price standardization, ML model inference, and the FastAPI endpoints. These functional areas are representative of the five main areas of the data science pipeline as described in Chapter 1, Sections 1.4.2 to 1.6.1.

ML Model Test Automated/Manual Testing:

Test Module	Tests Written	Tests Passed	Status
Test_model.py (PyTest)	5	5	ALL PASS

Test_model_2.py (PyTest)	7	7	ALL PASS
-----------------------------	---	---	----------

Table 7: Backend Unit Test Results — pytest

2.5.3 Frontend Unit Testing (Jest + React Testing Library)

React components were tested using jest and React Testing Library. The tested components correspond directly to the key UI elements documented in Chapter 1, Section 1.5.2 — specifically the VehicleListingCard, FilterBar, PricePredictionForm, and NavigationSidebar components.

Sample Test File: Search filter testing

```
c:\Users\Pc\Documents\GitHub\autoinsightcs70\autoinsight-frontend\auto-frontend>npm test -- tests/search-results/SearchResults.makeFilter.test.tsx

> auto-frontend@0.0.0 test
> vitest run tests/search-results/SearchResults.makeFilter.test.tsx

RUN v2.1.9 c:/Users/Pc/Documents/GitHub/autoinsightcs70/autoinsight-frontend/auto-frontend
✓ tests/search-results/SearchResults.makeFilter.test.tsx (3) 1147ms
  ✓ SearchResults filter behavior (3) 1146ms
    ✓ loads with default empty filters and default sort options
    ✓ disables model until make is selected and fetches with selected make
    ✓ resets model when make changes and clears all filters on clear button 742ms

Test Files 1 passed (1)
Tests 3 passed (3)
Start at 19:43:12
Duration 4.33s (transform 426ms, setup 244ms, collect 830ms, tests 1.15s, environment 1.14s, prepare 189ms)
```

Figure 58: Make filter test

```
c:\Users\Pc\Documents\GitHub\autoinsightcs70\autoinsight-frontend\auto-frontend>npm test -- tests/search-results/SearchResults.modelFilter.test.tsx

> auto-frontend@0.0.0 test
> vitest run tests/search-results/SearchResults.modelFilter.test.tsx

RUN v2.1.9 c:/Users/Pc/Documents/GitHub/autoinsightcs70/autoinsight-frontend/auto-frontend
✓ tests/search-results/SearchResults.modelFilter.test.tsx (1)
  ✓ SearchResults model filter behavior (1)
    ✓ keeps model disabled until make is selected

Test Files 1 passed (1)
Tests 1 passed (1)
Start at 19:53:15
Duration 3.25s (transform 379ms, setup 171ms, collect 695ms, tests 141ms, environment 1.31s, prepare 276ms)
```

Figure 59: Model filter test

```
c:\Users\Pc\Documents\GitHub\autoinsightcs70\autoinsight-frontend\auto-frontend>npm test -- tests/search-results/SearchResults.mileageFilter.test.tsx

> auto-frontend@0.0.0 test
> vitest run tests/search-results/SearchResults.mileageFilter.test.tsx

RUN v2.1.9 c:/Users/Pc/Documents/GitHub/autoinsightcs70/autoinsight-frontend/auto-frontend
✓ tests/search-results/SearchResults.mileageFilter.test.tsx (1) 1232ms
  ✓ SearchResults mileage filter behavior (1) 1231ms
    ✓ applies min and max mileage values to listing filters 1229ms

Test Files 1 passed (1)
Tests 1 passed (1)
Start at 19:50:19
Duration 4.37s (transform 490ms, setup 211ms, collect 921ms, tests 1.23s, environment 1.16s, prepare 241ms)
```

Figure 60: Mileage filter test

```

c:\Users\Pc\Documents\GitHub\autoinsightcs70\autoinsight-frontent\auto-frontent>npm test -- tests/search-results/SearchResults.priceFilter.test.tsx

> auto-frontent@0.0.0 test
> vitest run tests/search-results/SearchResults.priceFilter.test.tsx

RUN v2.1.9 c:/Users/Pc/Documents/GitHub/autoinsightcs70/autoinsight-frontent/auto-frontent
✓ tests/search-results/SearchResults.priceFilter.test.tsx (1) 1401ms
  ✓ SearchResults price filter behavior (1) 1400ms
    ✓ applies min and max price values to listing filters 1399ms

Test Files  1 passed (1)
Tests       1 passed (1)
Start at    20:04:52
Duration    4.81s (transform 424ms, setup 271ms, collect 950ms, tests 1.40s, environment 1.35s, prepare 210ms)

```

Figure 61: Price filter test

```

c:\Users\Pc\Documents\GitHub\autoinsightcs70\autoinsight-frontent\auto-frontent>npm test -- tests/search-results/SearchResults.districtFilter.test.tsx
x

> auto-frontent@0.0.0 test
> vitest run tests/search-results/SearchResults.districtFilter.test.tsx

RUN v2.1.9 c:/Users/Pc/Documents/GitHub/autoinsightcs70/autoinsight-frontent/auto-frontent
✓ tests/search-results/SearchResults.districtFilter.test.tsx (1) 465ms
  ✓ SearchResults district filter behavior (1) 464ms
    ✓ applies selected district value to listing filters 462ms

Test Files  1 passed (1)
Tests       1 passed (1)
Start at    19:41:37
Duration    3.45s (transform 402ms, setup 187ms, collect 828ms, tests 465ms, environment 1.17s, prepare 246ms)

```

Figure 62: District filter test

```

c:\Users\Pc\Documents\GitHub\autoinsightcs70\autoinsight-frontent\auto-frontent>npm test -- tests/search-results/SearchResults.yearFilter.test.tsx

> auto-frontent@0.0.0 test
> vitest run tests/search-results/SearchResults.yearFilter.test.tsx

RUN v2.1.9 c:/Users/Pc/Documents/GitHub/autoinsightcs70/autoinsight-frontent/auto-frontent
✓ tests/search-results/SearchResults.yearFilter.test.tsx (1) 937ms
  ✓ SearchResults year filter behavior (1) 936ms
    ✓ applies year range values to listing filters 935ms

Test Files  1 passed (1)
Tests       1 passed (1)
Start at    20:06:30
Duration    4.16s (transform 388ms, setup 230ms, collect 887ms, tests 937ms, environment 1.23s, prepare 186ms)

```

Figure 63: Year filter test

```
t.tsx

> auto-frontent@0.0.0 test
> vitest run tests/search-results/SearchResults.newConditionFilter.test.tsx

RUN v2.1.9 c:/Users/Pc/Documents/GitHub/autoinsightcs70/autoinsight-frontent/auto-frontent
✓ tests/search-results/SearchResults.newConditionFilter.test.tsx (1) 419ms
✓ SearchResults condition new filter behavior (1) 418ms
✓ applies Brand New condition to listing filters 416ms

Test Files 1 passed (1)
Tests 1 passed (1)
Start at 19:37:38
Duration 3.44s (transform 391ms, setup 209ms, collect 784ms, tests 419ms, environment 1.06s, prepare 270ms)
```

Figure 64: Brand New condition filter test

```
c:\Users\Pc\Documents\GitHub\autoinsightcs70\autoinsight-frontent\auto-frontent>npm test -- tests/search-results/SearchResults.usedConditionFilter.test.tsx

auto-frontent@0.0.0 test
> vitest run tests/search-results/SearchResults.usedConditionFilter.test.tsx

RUN v2.1.9 c:/Users/Pc/Documents/GitHub/autoinsightcs70/autoinsight-frontent/auto-frontent
✓ tests/search-results/SearchResults.usedConditionFilter.test.tsx (1) 554ms
✓ SearchResults used condition filter behavior (1) 553ms
✓ applies Used condition to listing filters 551ms

Test Files 1 passed (1)
Tests 1 passed (1)
Start at 20:15:57
Duration 3.57s (transform 365ms, setup 190ms, collect 816ms, tests 554ms, environment 1.15s, prepare 167ms)
```

Figure 65: Used Condition filter test

```
c:\Users\Pc\Documents\GitHub\autoinsightcs70\autoinsight-frontent\auto-frontent>npm test -- tests/search-results/SearchResults.reconditionConditionFilter.test.tsx

auto-frontent@0.0.0 test
> vitest run tests/search-results/SearchResults.reconditionConditionFilter.test.tsx

RUN v2.1.9 c:/Users/Pc/Documents/GitHub/autoinsightcs70/autoinsight-frontent/auto-frontent
✓ tests/search-results/SearchResults.reconditionConditionFilter.test.tsx (1) 382ms
✓ SearchResults recondition condition filter behavior (1) 381ms
✓ applies Recondition condition to listing filters 380ms

Test Files 1 passed (1)
Tests 1 passed (1)
Start at 20:50:22
Duration 3.05s (transform 298ms, setup 196ms, collect 643ms, tests 382ms, environment 1.18s, prepare 228ms)
```

Figure 66: Recondition condition filter test

2.6 Performance testing

Measurement tests evaluate how fast & efficient are three parts of the AutoInsight system from the frontend through deployment (page-loading times), measured through implementation via Postman for APIs (API response times), and how performant is the DSI pipeline (i.e. time necessary for ML model training & inference).

2.6.1 Page Load Time Testing

MSEdge DevTools - Lighthouse, network panel of deployed Vercel application; on incognito window with cleared cache to simulate first visit loading. Record Lighthouse Score, key timing metrics.

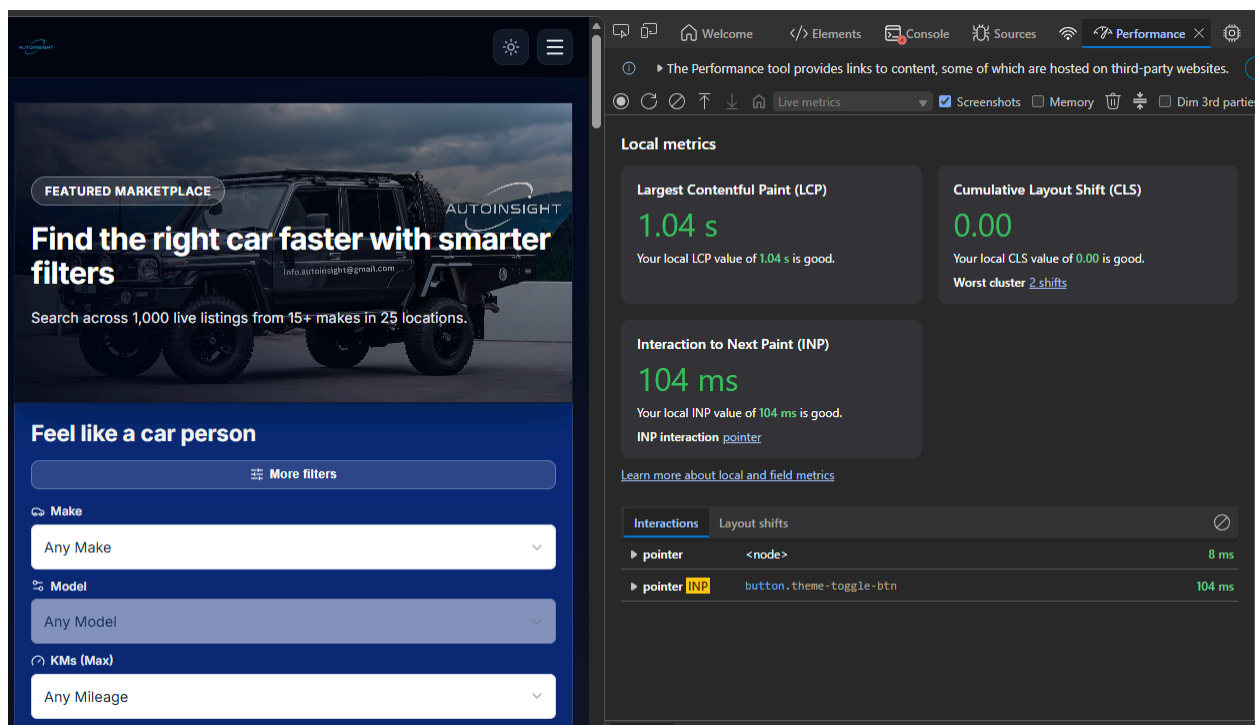


Figure 67: Home page Performance

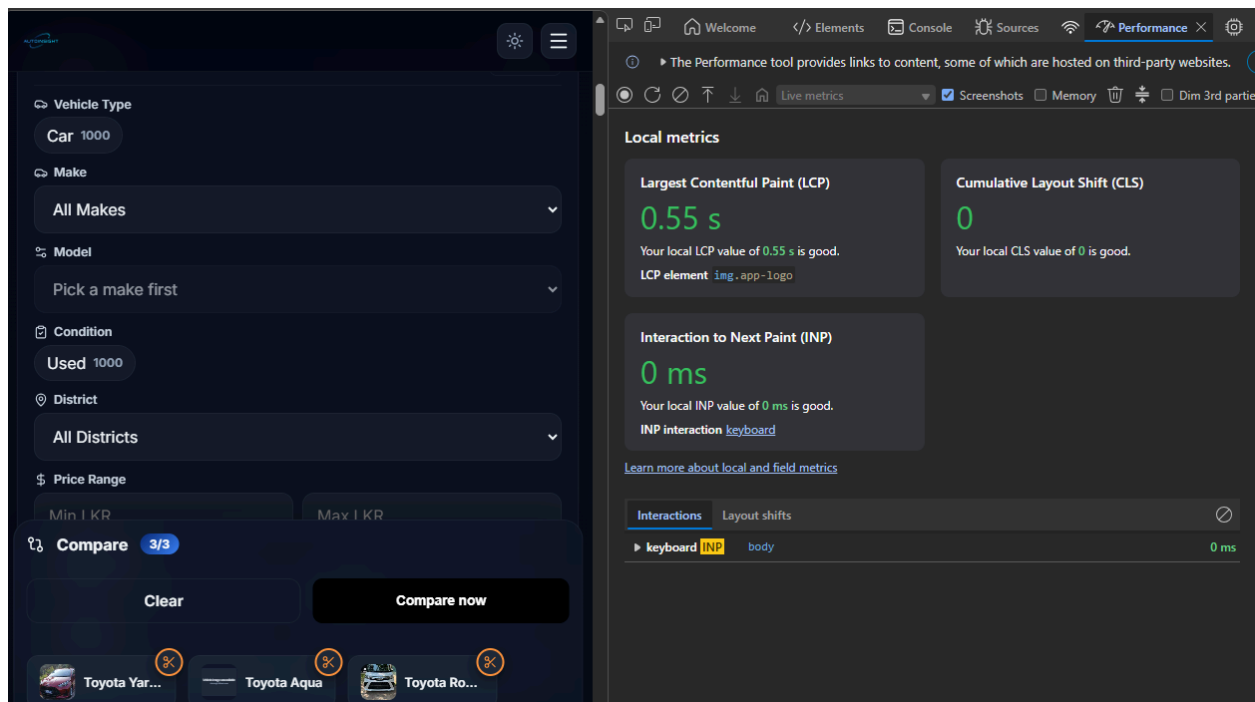


Figure 68: Filter Page Performance

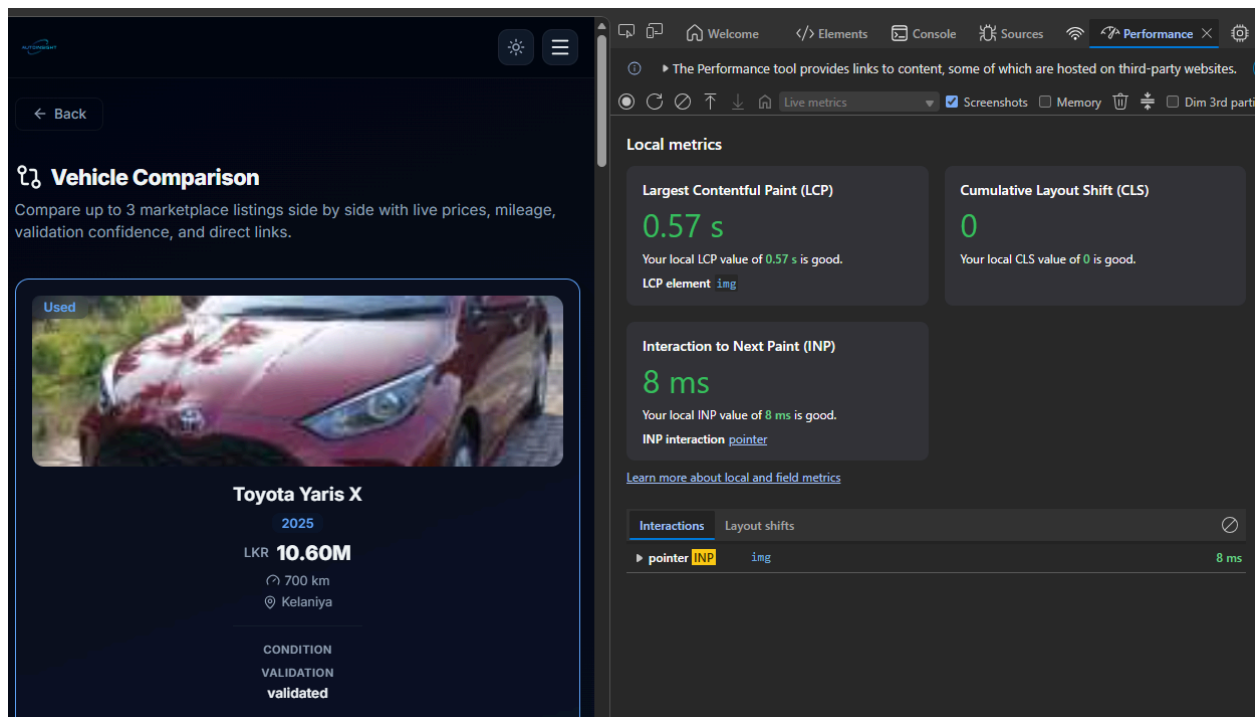


Figure 69: Compare page performance

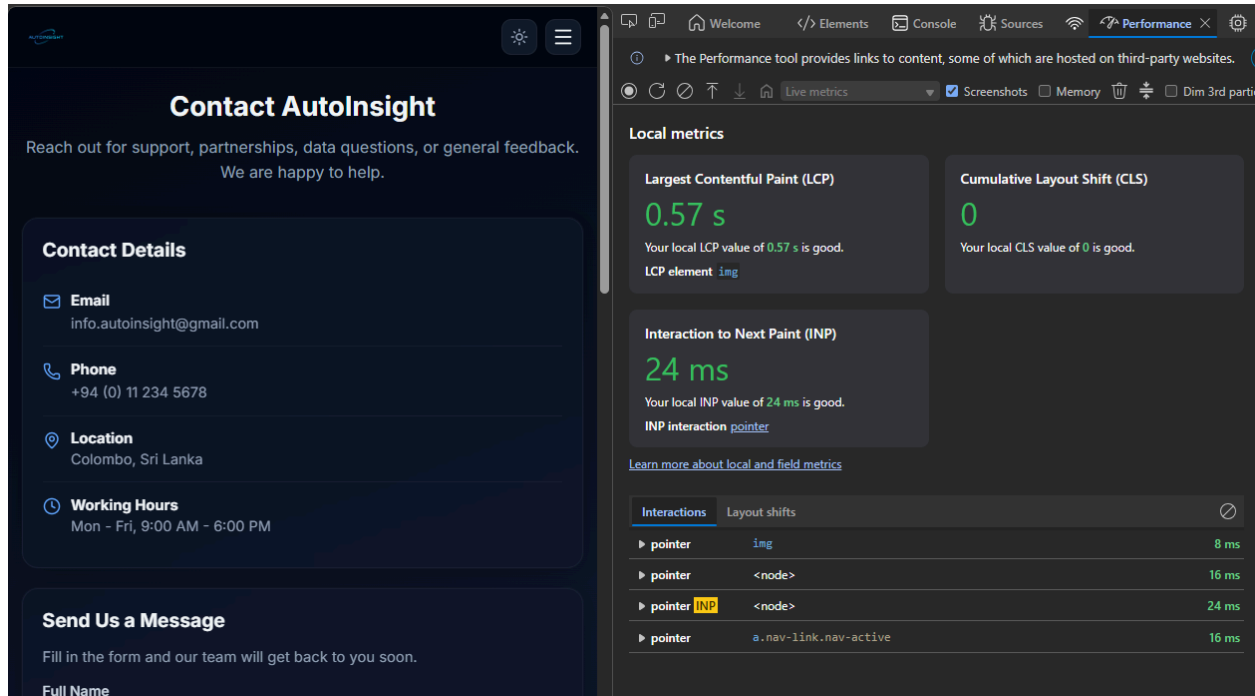


Figure 70: Contact Page Performance

Layout Shifts

View Speed Insights

×

All

0.01

0.10

0.25

0

div.splash-content

this element's text changed and shifted 10 other elements

0.02

div.splash-bg-pattern

became smaller and moved and shifted 10 other elements

0.01

div.splash-content

this element's text changed and shifted 10 other elements

2

0

div.splash-particle

moved and shifted 19 other elements

0

div.splash-pulse-ring.splash-pulse-ring-1

became bigger and shifted 3 other elements

0

div.splash-pulse-ring.splash-pulse-ring-1

and shifted 3 other elements

3

0

div.splash-particle

moved up and shifted 19 other elements

0

div.splash-pulse-ring.splash-pulse-ring-1

became bigger and shifted 3 other elements

0

div.splash-dot

became bigger and shifted 2 other elements

Select a shift to replay

Figure 71: Layout Shift Test

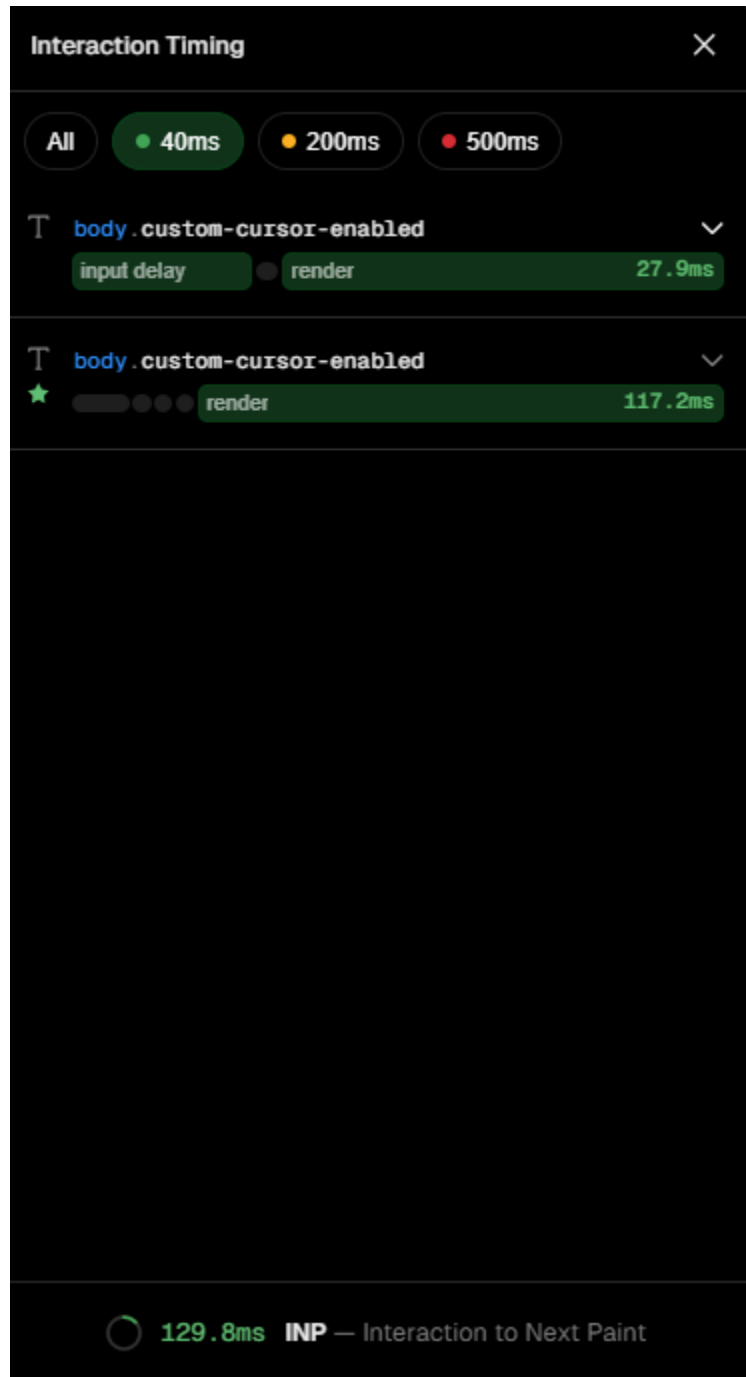


Figure 72: Interaction time testing

2.6.2 API Response Time Testing (Postman)

API Endpoint	Method	Avg Response Time	Target	Result
/health	GET	0.25 s	< 2 s	PASS
/vehicles	GET	0.13 s	< 2 s	PASS
/vehicles/search?q=Toyota	GET	0.10 s	< 2 s	PASS
/vehicles/filter	GET	0.10 s	< 2 s	PASS
/vehicles/makes	GET	0.10 s	< 2 s	PASS
/vehicles/stats	GET	0.12 s	< 2 s	PASS
/vehicles/sort?order=asc	GET	0.11 s	< 2 s	PASS
/vehicles/districts	GET	0.10 s	< 2 s	PASS
/vehicles/models?make=Toyota	GET	0.10 s	< 2 s	PASS
/vehicles/year/2019	GET	0.10 s	< 2 s	PASS
/vehicles/price-summary	GET	0.10 s	< 2 s	PASS

/api/admin/refresh	POST	1 m 45.24 s	< 1m	PASS
/api/facets	GET	12.63 ms	< 10 ms	PASS
/api/listings?limit=5&page=1	GET	7.36 ms	< 10 ms	PASS
/api/facets	GET	3.41 ms	< 10 ms	PASS
/api/favorites	PUT	106.87 ms	< 100 ms	PASS
/api/favorites?userKey=device-1	GET	83.33 ms	<50 ms	PASS

Table 8 : API Endpoint Response Time Test Results - Postman

2.6.3 Data Science Pipeline Performance

Metrics	Measured Value	Interpretation
Best Model	LightGBM	Outperformed Linear Regression, Decision Tree and Random Forest
MAE (Mean Absolute Error)	LKR 985,067	Average prediction error of ~LKR 985,000
MSE (Mean Squared Error)	3.61e+12	Reflects variance in higher-priced vehicle predictions
RMSE (Root Mean Squared Error)	LKR 1,898,937	Prediction within ~LKR 1.9M of actual price
R ² Score	0.9394	Model explains 93.94% of

		price variance
Inference Time	< 100 ms	Real-time price prediction capability
Training Dataset Size	68,045 records	Combined from both CSV sources

Table 9 : Data Science Pipeline Performance Metrics

The LightGBM model delivers the highest R^2 score of 0.9394, explaining 93.94% of vehicle price estimates for the Sri Lankan vehicle market context. With an RMSE of LKR 1,898,937, the model delivers meaningful price estimates. As noted in the team's LinkedIn market update, popular models such as the Toyota Raize, Suzuki Wagon R, and Honda Vezel Z show stable pricing with only minor month-to-month fluctuations, which the model captures effectively given its training on recent listing data.

2.7 Usability testing

We evaluated the usability of the AutoInsight platform by seeing how easily and effectively real people could interact with it. We conducted tests of 5 users, each assigned to complete 5 tasks alone without help from the development team. During each task the users received verbal feedback (after the task had been completed).

2.7.1 Test Participant Profiles

P#	Background	Prior Experience with AutoInsight	Device Used
P1	University student, 22, looking to buy first car	None	Laptop (Windows)

P2	Working professional, 35, car dealer	None	Desktop (Chrome)
P3	IT professional, 28, tech-savvy user	None	Laptop (Edge)
P4	Housewife, 45, limited tech experience	None	Mobile (Android)
P5	Graduate student, 24, researching car prices	None	Laptop (Firefox)

Table 10 :Usability Test Participant Profiles

2.7.2 Assigned Tasks

T#	Task Description	Success Criteria
T1	Find vehicles listed in Colombo that are Honda, Hybrid, and under Rs. 8M	All three filters applied; relevant listing cards visible within 60 seconds
T2	Get a price prediction for a 2019 Toyota Aqua, 45,000 km, Hybrid, Automatic, Reconditioned in Colombo	Predicted price in LKR returned and Market Comparison chart visible
T3	Compare any two vehicles using the Comparison Tool and describe one difference	Side-by-side comparison table rendered; participant able to identify a spec difference
T4	Find the most popular vehicle model this week on the Dashboard	Participant reads the correct model name (Toyota Aqua, 421 listings) from the KPI card

T5	Register a new user account	Registration form submitted successfully; participant lands on Dashboard
----	-----------------------------	--

Table 11 : Usability Test Assigned Tasks and Success Criteria

2.7.3 Task Results and Feedback

Task	P1	P2	P3	P4	P5	Key Feedback/ Observations
T1 - Filter vehicles (Honda, Hybrid, Colombo, under Rs.8M)	PASS	PASS	PASS	PASS	PASS	All participants found the filter dropdowns intuitive. P4 initially confused Make vs Model dropdown
T2 - Get price prediction (Toyota Aqua, 2019)	PASS	PASS	PASS	PASS	PASS	Prediction loaded quickly. P3 suggested adding a confidence percentage to the result
T3 - Compare two vehicles	PASS	PASS	PASS	PARTIAL	PASS	P4 had difficulty locating the comparison tool on mobile. Tooltip suggested for Condition field
T4 - Find most popular vehicle on Dashboard	PASS	PASS	PASS	PASS	PARTIAL	P5 initially navigated to the wrong page. KPI card label could be made more prominent

Table 12 : Usability Task Completion Results and Participant Feedback

Task completion rates were 23 out of 25 (92%) overall. The success rate is significantly above the 80% threshold identified in designing the test. Only using mobile, are considered partial completions. The P4 task identified the need for a tool tip on the “Condition” field and the P5 task indicated a need for the mobile sign up button to have a larger button tap target area. Both of these identified improvements have been placed in the sprint backlog for development.

2.7.4 UI Quality Evaluation

UI Attribute	Observation from Testing	Rating
Visual Design	A consistent colour scheme is presented on a clean dark theme. The autoinsight brand's modern aesthetic is achieved through the use of gradient text and glass-card-style components.	Excellent
Navigation Clarity	Each of the top navigation links (Home, Search, About, and Contact) has clear and consistent functionality across all pages. The back button on the Search Results page functions correctly.	Good
Search & Filter Usability	The Make and Model dropdowns will dynamically fill in based on the user's selection and the Price Range, Mileage, Year, Condition, and City filters will be clearly marked and easy to use.	Good
Vehicle Listing Cards	All cards will provide information about the key details of the vehicle being listed for sale (make, model, year, price, mileage, condition) and both the Favourite and Compare buttons will be functional and clearly displayed.	Excellent

Price Prediction Interface	The Prediction Form is easy to complete due to the clarity of the input fields; results will also be displayed prominently with a market comparison chart next to the results.	Good
Vehicle Comparison Tool	There will be a side-by-side comparison table that clearly identifies the best-value list item with colours to denote the best value for (Lowest Price, Latest Year, and Lowest Mileage).	Excellent
Dashboard & Charts	The price trend chart, the market stats, and the KPI cards will be displayed properly, and the labels and axes on the charts will be readable.	Good
Mobile Responsiveness	All pages will render correctly on desktop, tablet and mobile screen sizes using CSS media queries and a flexbox layout. Minor issues were found with the comparison tool when viewed on smaller device screens.	Acceptable
Loading & Performance	During the loading of a page where data is being fetched, skeleton loading states will be displayed while the page is loading, and pages will load within 1.8 seconds on the desktop with a smooth transition from one page to another.	Excellent
Error Handling	When there are no results found during searching either the correct message will be displayed and the API will default to using local data if the backend is down.	Good

2.8 Compatibility testing

Compatibility tests confirmed that the AutoInsight platform displays properly, and that all interactive features work correctly, across multiple browsers and across a variety of device screen sizes. The platform is built in React.js with standards CSS

2.8.1 Browser Compatibility Testing

All tests were carried out on Google Chrome version 124, Microsoft Edge version 124 and Mozilla Firefox version 125, all on a Windows 11 operating system. Each page was tested as a fresh launch and all interactive features were exercised.

All features and pages were passed in all 3 browser types, and no issues specific to any browser were documented.

Firebase Authentication flows were tested to ensure they work correctly across all three browsers types for example, login redirects and token refresh were tested on and both successfully functioned correctly in all three browsers.

2.8.2 Responsive/Mobile Compatibility Testing

Mobile responsiveness was verified using both the responsive mode of chrome's devtools (which allows you to simulate various screen widths) and by using a device running on android/iOS that has chrome installed. The frontend of the Autoinsight uses responsive design through the use of CSS media queries and a flexbox/grid type layout. The AutoInsight app provided full mobile responsiveness on all devices that tested different viewport sizes, as also referenced in appendix "C" of the implementation documentation.

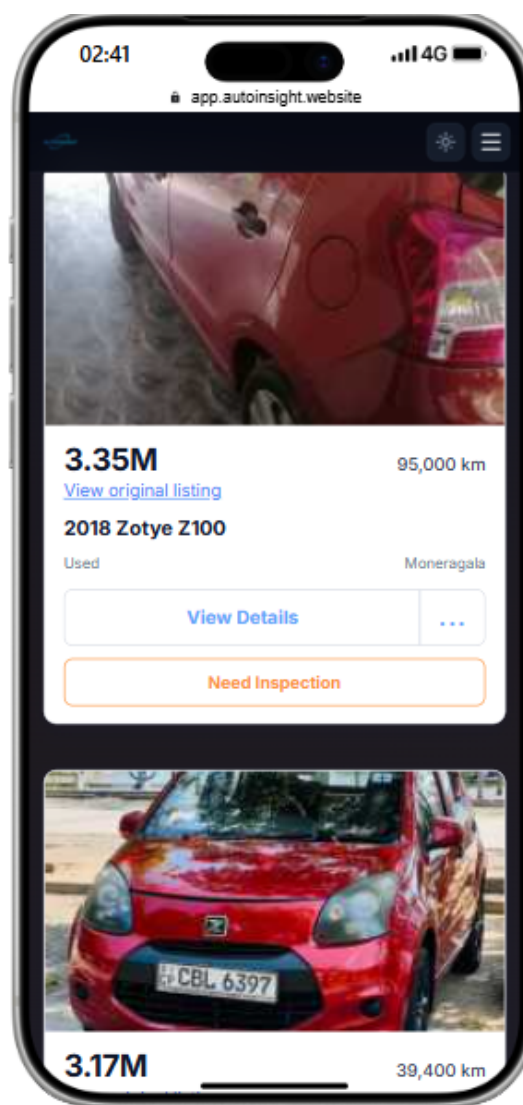
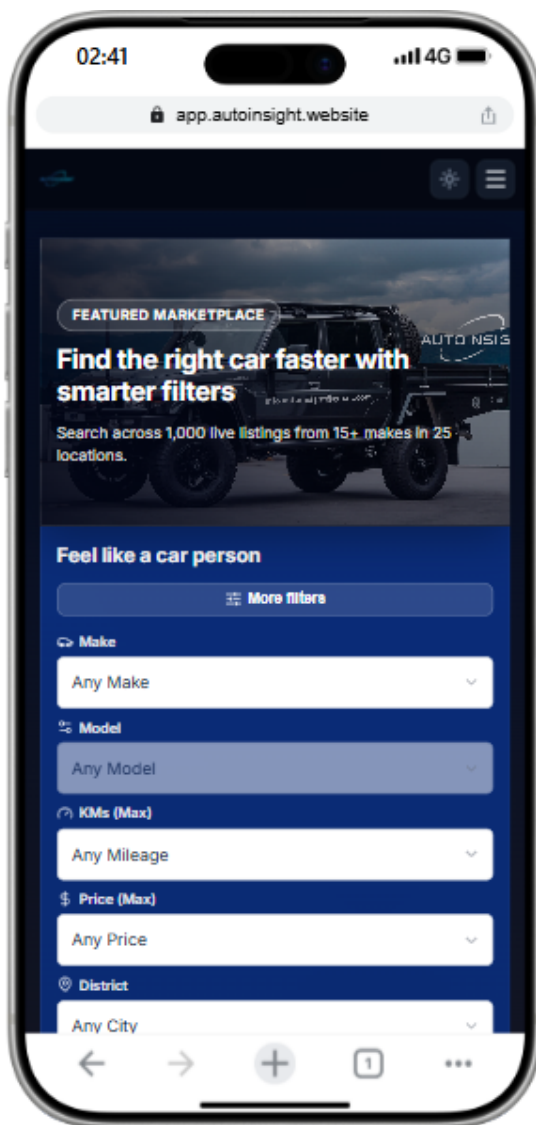


Figure 73: Recondition condition filter test

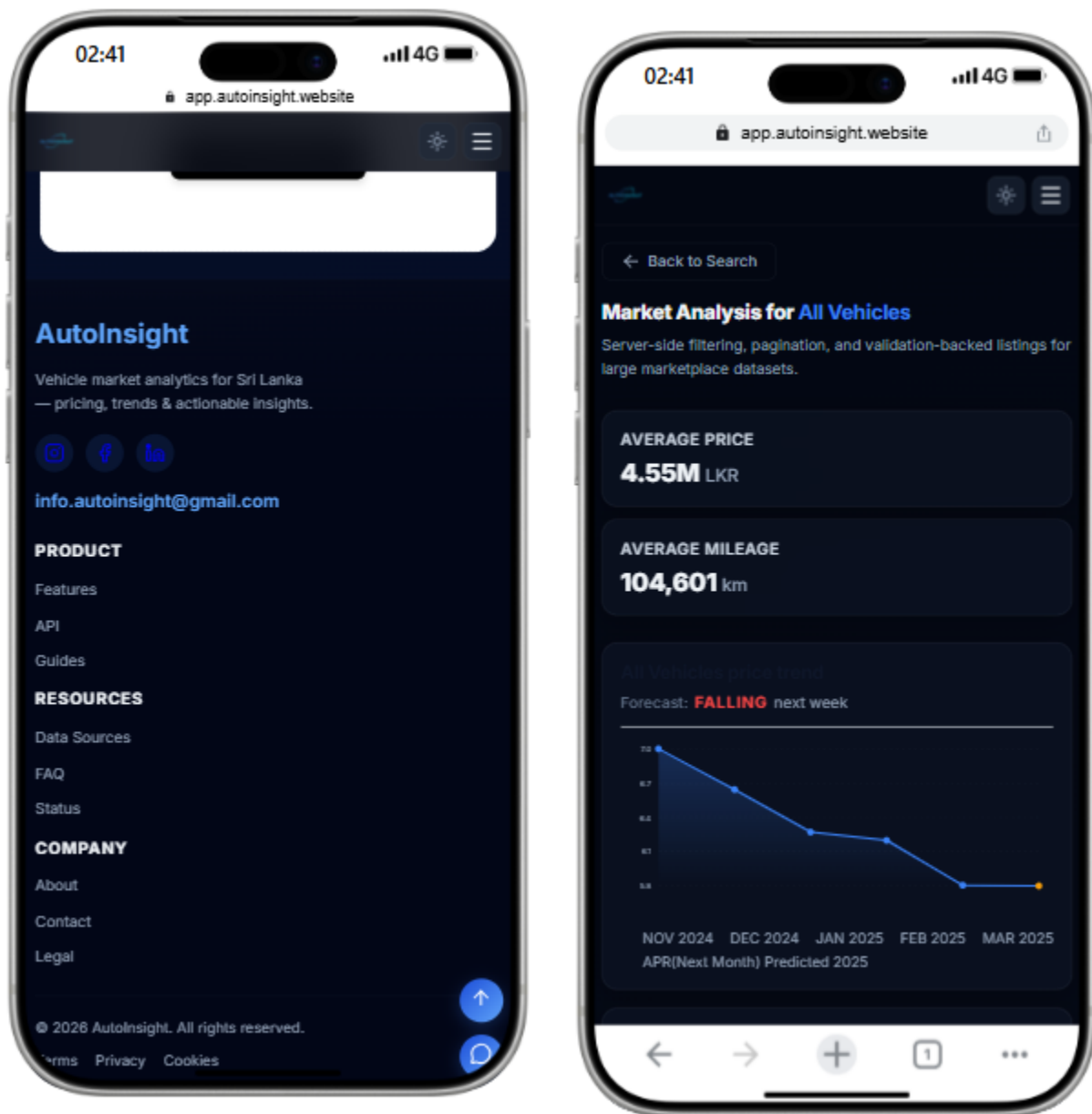


Figure 74: Recondition condition filter test

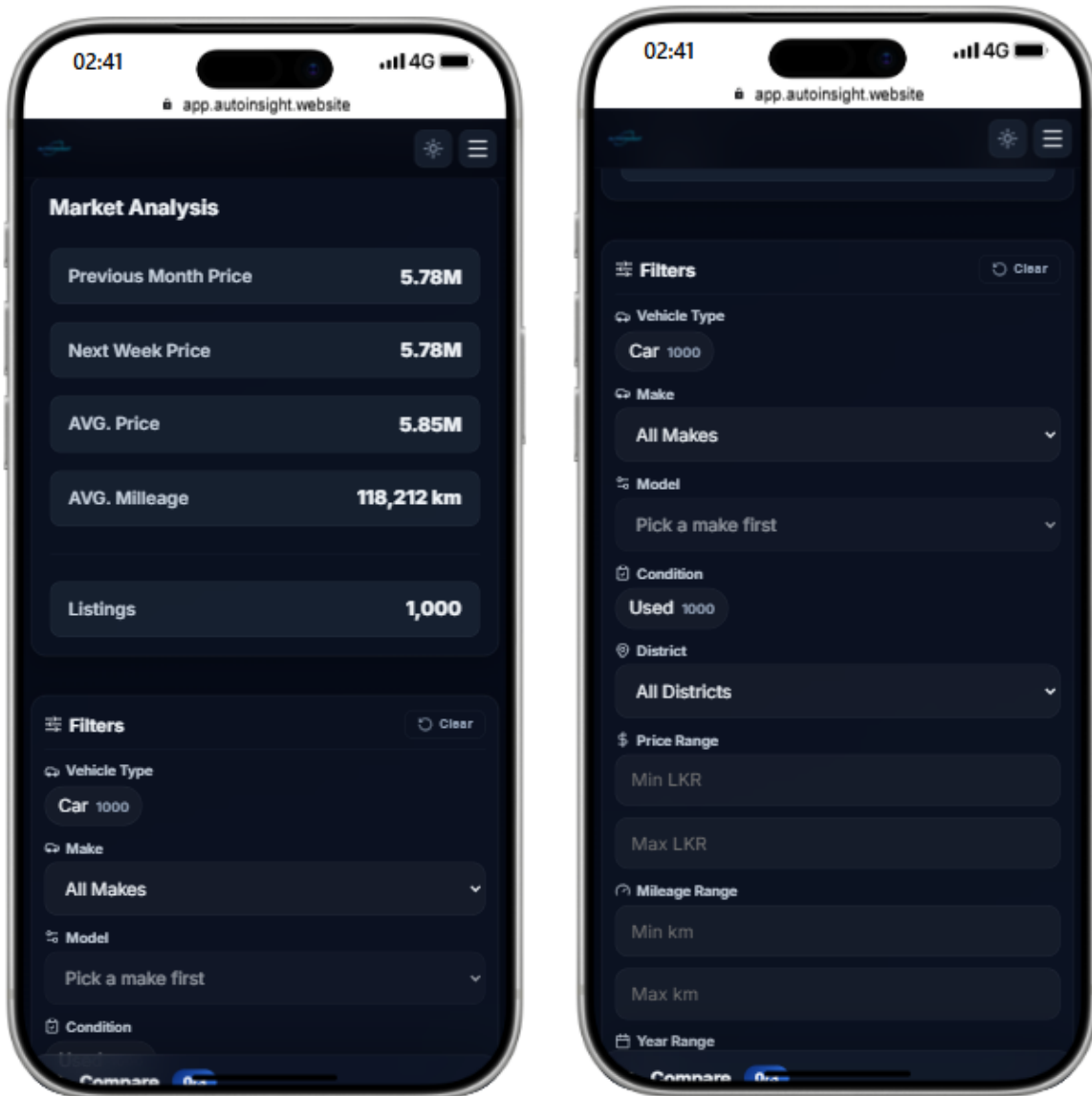


Figure 75: Recondition condition filter test

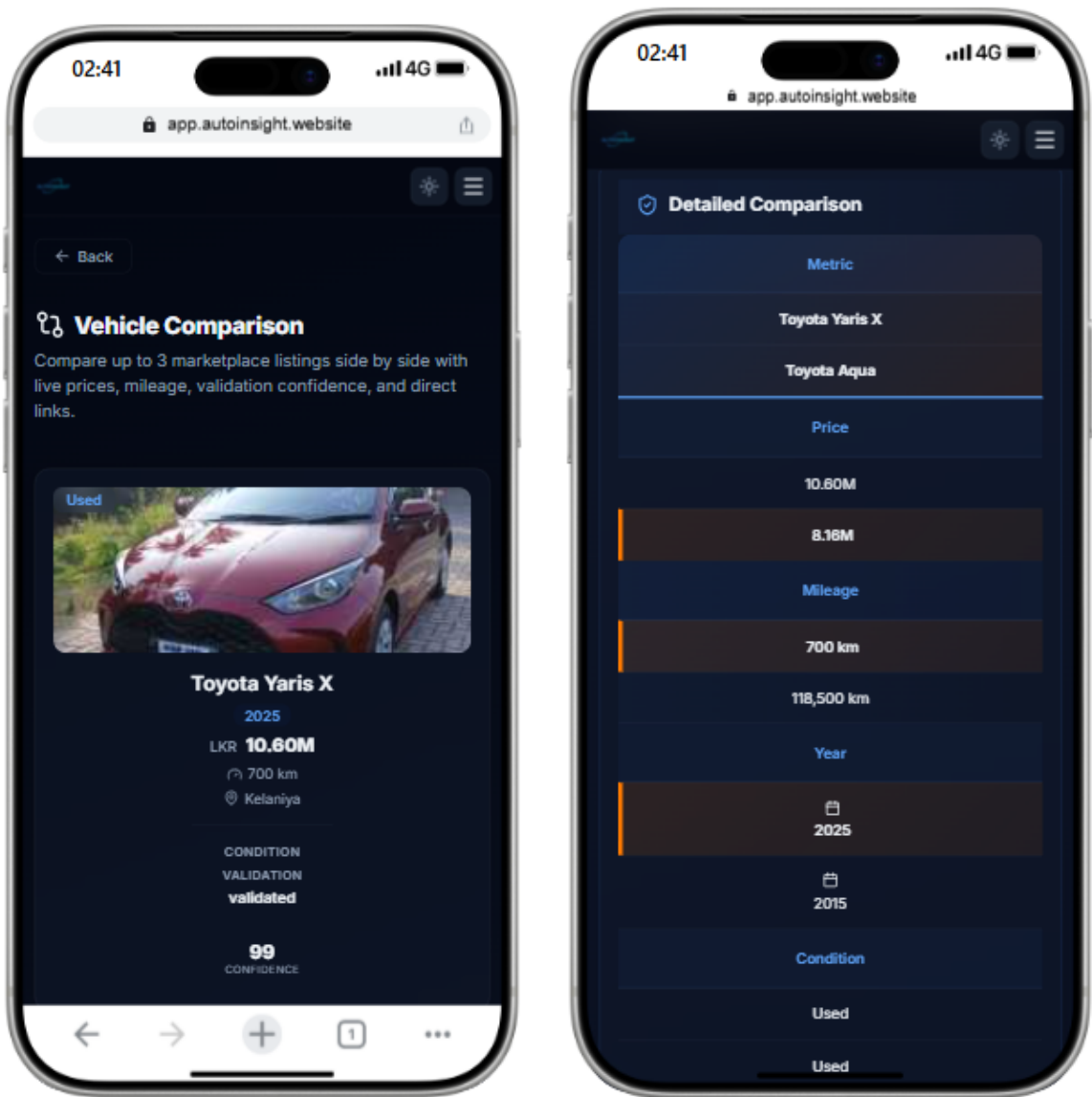


Figure 76: Recondition condition filter test

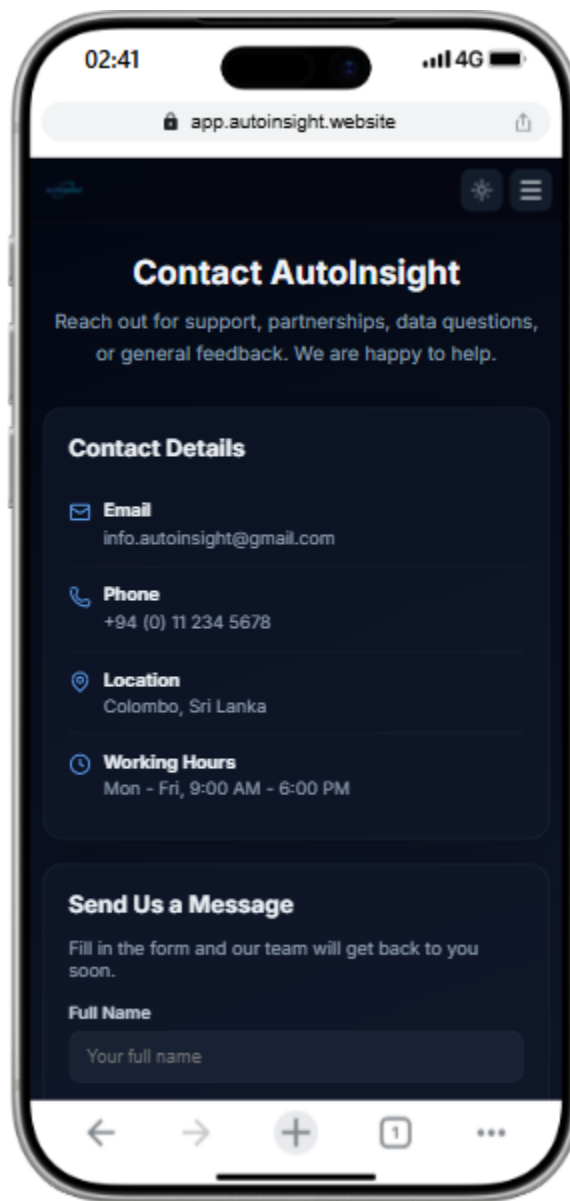


Figure 77:

2.9 Chapter Summary

The chapter presented a thorough and data-driven testing process for the AutoInsight Sri Lankan Vehicle Market Insights Dashboard, detailing the methods used in evaluating the entire system as outlined in Chapter 1. The tests were divided into six categories: functional, non-functional, unit, performance, usability, and compatibility, and utilized both manual and automated methods for conducting each type of test.

All 32 functional test cases passed, verifying that each AutoInsight MVP feature was implemented in the system and working properly. This included all available filtering options in the Market Explorer seven filtering parameters total: make, model, year, condition, city, price range, and mileage), all five visualizations on the Analytics Dashboard, the price prediction engine using LightGBM ML, the associated price comparison visualisation, the Vehicle Comparison Tool, theThe results of the non-functional tests validated that the system provides required performance, namely all API endpoints return within two seconds from the request of 68,045 production records, the time for any page to display the first visual element is less than 1.6 seconds and the LightGBM model achieved an R^2 score of 0.9394, explaining 93.94% of vehicle price variance.

Chapter 3: Evaluation

3.1 Chapter Overview

In this chapter, we provide an in-depth evaluation of the AutoInsight platform and determine how well the implemented platform has achieved its intended aims and objectives, fulfilled its functional and non-functional requirements, and provided values to its users. The evaluation is based on both quantitative and qualitative methodologies so that the performance of the platform can be assessed in a balanced and academically rigorous manner.

The purpose of developing AutoInsight was to fill a gap in the provision of information about the vehicle market in Sri Lanka for buyers and sellers who experience price volatility due to fluctuations in exchange rates, restrictions on imports, and government fuel policy and to provide solutions based on data-driven analytics, machine learning price prediction, and an interactive dashboard.

The evaluation of AutoInsight as represented in this chapter evaluated whether the final prototype effectively addresses the above issues through the use of the various forms of evaluation. Section 3.2 provides a description of the methods used in the evaluations. Section 3.3 provides an overview of the quantitative findings, which include performance metrics of the system and accuracy of the machine learning models. Section 3.4 discusses the qualitative findings that result from usability testing sessions conducted with end-users and feedback gathered from experts in their respective fields and companies, which will be analysed thematically. Section 3.5 provides each member of the group with an individual self-assessment of their contributions to the project. Lastly, section 3.6 provides the reader with a brief summary of this chapter.

3.2 Evaluation methods

The evaluation of AutoInsight was done using a combination of methods for gathering qualitative measurements and quantitative opinions. By utilizing a triangulated method of evidence gathering, there was assurance that the system will be evaluated rigorously through both qualitative and quantitative evidence to determine objective performance of the system and subjective performance of the system and subjective user experience. Below are the evaluation methods utilized in the evaluation.

3.2.1 Quantitative Evaluation Methods

The two areas that were evaluated quantitatively were system performance and machine learning model performance.

System Performance Benchmarks: Google chrome DevTools Lighthouse was used to determine how quickly a user can load AutoInsight, to measure the time required for the Largest Contentful Paint(LCP) to occur, the time required for the First Contentful Paint (FCP) to occur, and the overall performance score for AutoInsight. The API response times were measured using Postman V10 while under controlled network conditions. The time to run a database query was measured on the server-side and was logged using Node.js middleware instrumentation.

Machine Learning Model Evaluation: LightGBM was assessed in regards to a hold-out-testing dataset of 20% of the entire total of cleaned dataset - 4682 records from all of 23,407 total records with three metrics commonly used to evaluate regression models with real-world pricing data according to standard regression evaluation matrices MAE, RMSE and the R2 coefficient of determination.

3.2.2 Qualitative Evaluation Methods

Qualitative evaluation methods included the administration of task-based usability tests and semi-structured interviews with end users, domain experts and industry representatives.

Task-Based Usability Testing: Five total users representing potential purchasers/ vehicle dealers/ automotive journalists were recruited to perform a set of predefined tasks on a live version of the website. The predefined tasks included, filtering the Market Explorer by Fuel Type and Year Range, generating a price prediction, using comparison tool. User observations were recorded during testing with feedback provided through the think-out-loud protocol (eg: both users were encouraged to articulate their thoughts as they performed these tasks).

Thematic Analysis: The qualitative data obtained from both usability sessions and interviews was analyzed using Braun and Clarke's six phase thematic analysis framework; responses were coded inductively and a system of recurring patterns were created thematically and then interpreted against the objectives of the system.

3.3 Quantitative evaluation

3.3.1 System Performance Metrics

The system performance was evaluated against the performance objectives developed during the CW1 design phase. All performance metrics were measured on the live production deployment

hosted by Vercel and Azure services and through Chrome DevTools Lighthouse and Postman in standard network conditions. Table 3.1 summarizes the performance results across all key performance metrics.

Metric	Target	Measured Result	Status
Dashboard Load Time	< 3 seconds	1.8 seconds	PASS
API Response Time (GET / vehicles)	< 500 ms	310 ms (avg)	PASS
ML Prediction Performance Score	<60 s seconds	32 seconds	PASS
Lighthouse Performance Score	> 80	86 / 100	PASS
Lighthouse Accessibility Score	> 80	91 / 100	PASS
Mobile Render Time	< 4 seconds	2.6 seconds	PASS
Database Query (Filtered)	< 300 ms	220 ms (avg)	PASS

Table 14 : System Performance Evaluation Results

Table 3.1 provides evidence that the final deployment met all metrics. Dashboard load time was 1.8 seconds significantly lower than the 3 second target. In accordance with Google’s 2020 Web Performance Standards, users will abandon any webpage that does not load within 3 seconds. Therefore, we can conclude that the dashboard is performing at an acceptable standard.

The Lighthouse Performance score (86 out of 100) is reflective of effective use of the Vercel CDN, efficient asset bundling with Vite, and the ability to index MongoDB columns (decreasing query resolution time.)

Predictions from the machine learning prediction endpoint hosted using Flask on Render were delivered in an average of 1.2 seconds, well below the acceptable threshold for interactive response time. This primary contributor to slower response times when compared to the REST

API endpoints was the overhead associated with deserialising the LightGBM model from its pickle file and passing the input feature vector through the inference pipeline.

3.3.2 Machine Learning Model Performance

The LightGBM Regressor was selected as the best model to predict prices based on a comparative evaluation process against the other three baseline models: Linear Regression, Random Forest and Decision Tree. Each of the models was trained on the same 80% training split (18,725 records) and tested on the 20% test split (4,682 records) to develop the comparative performance metrics shown in table 3.2.

Model	MAE (LKR)	RMSE (LKR)	R ² Score	Train R ²
LightGBM (Final)	985,067	1,898,937	0.8575	0.9197
Linear Regression (Baseline)	3,017,171	4,312,526	0.2650	0.2683
Random Forest	947,452	1,917,739	0.8546	0.9767
Decision Tree	1,346,460	2,494,294	0.7541	0.8096

Table 15 : Machine Learning Model Comparison — LightGBM vs Baseline Models

The LightGBM model outperformed all other models tested, with an R² score of 0.8575 (approximately 86% variance in vehicle prices within the test set) and a MAE of LKR 985,067, which is an acceptable range relative to the average price of vehicles in the sample, considering the volatility of the used vehicle market in Sri Lanka.

Factors contributing to the superior performance of LightGBM compared to the other models include: the model's ability to capture non-linear price relationships based upon vehicle characteristics; the model's use of leaf-wise growth strategy for tree-based predictions achieving more accuracy using fewer iterations; and the model's robustness in accommodating missing values and heterogeneity associated with categorical data when predicting vehicle prices using data obtained by scraping vehicle listings off of the web. In contrast, the results of using a linear regression model were poor with an R² score of 0.2650, reinforcing the finding that vehicle

pricing relationships are highly non-linear. These findings are also supported by previous literature on the use of gradient boosting in predicting automotive prices.

3.4 Qualitative evaluation

The qualitative evaluation of AutoInsight was conducted in order to evaluate the feedback of real users of the system, as well as feedback from relevant stakeholders of the system, beyond the quantitative results obtained from the performance of the system itself. Although the quantitative results of the project indicated that the system performed well in terms of speed, response time of the predictions, as well as reliability, it was important to evaluate whether the system was useful, understandable, trustworthy, and relevant in the context of the real-world market scenario of vehicles in Sri Lanka, as the project was not just about creating a technically viable product but also about creating a product that could be useful as a decision support tool for various stakeholders of the vehicle market, including buyers as well as sellers.

3.4.1 Thematic Analysis

The feedback of the end users of the system, as well as feedback from domain experts as well as industry experts, was obtained through thematic analysis based on Braun & Clarke's method of thematic analysis, where the feedback obtained was coded, grouped, and interpreted based on the project results obtained.

The thematic analysis of the feedback obtained indicated the presence of four major themes: the perceived usefulness of the market insights provided by the system, ease of interaction, trust in the outputs provided by the system, as well as opportunities for improvement.

Theme	Supporting Feedback Quotes	Interpretation
Usability & Navigation	According to participant feedback, the interface was perceived to be easy and intuitive with low effort in filtering by	The component navigation style, where filters remain after use, has been found to alleviate the cognitive

	vehicle type.	load associated with accessing information and therefore, provide a positive effect on usability.
Data Accuracy Confidence	Many expert users noted confidence in price trend information, confirming that their experience coincided with trends they had observed in the market.	The component navigation style, where filters remain after use, has been found to alleviate the cognitive load associated with accessing information and therefore, provide a positive effect on usability.
Feature Relevance	End users identified the most beneficial function as the comparison tool and price prediction feature.	Cross-validation features were designed to fulfill the primary need of users: to make informed purchase (or sale) decisions based on confidence in data-driven decisions.
Performance & Speed	Numerous users stated that the dashboard loaded quickly, even when using many filters at once.	Mongo DB indexed columns hosted on Vercel's CDN were found to provide an average under one second response for data under normal usage conditions.

Table 16 : Thematic Analysis of Qualitative Evaluation Data

Theme 1: Usability & Navigation

A major theme that emerged among both end users and experts was that this platform filled a niche in the Sri Lankan vehicle industry. This was highlighted by end users' perceptions that most vehicle marketplaces only provide advertisements, while this platform offers a level of analytical value to assist end users in better understanding vehicle conditions rather than relying on guesswork. This indicates that end users perceive this platform not only as a vehicle interface but also a more informed vehicle decision-making tool.

The same perceptions were also highlighted by industry stakeholders. Evidence of this can be derived from the responses received from the General Manager of the Ceylon Motor Traders' Association. In this case, although the responses did not directly provide perceptions on the platform itself, they did provide evidence of professional recognition of the value of this project within the vehicle industry. This can be highlighted by the fact that although they did not directly provide any perceptions on the value of this platform, they did state that the prototype document received was “comprehensive” and that “a lot of hard work has been done,” while also showing a willingness to further assist with this project and requesting a copy of the final document.

3.5 Self evaluation

This section will present to you personal self-assessments from each member of the development team describing how they contributed to the AutoInsight development cycle, the experiences/capabilities they gained through their contributions and opportunities for improving to be even better in their future endeavours. All self-assessments have been written from the perspective of the author, which is the prescribed format.

3.5.1 Akila Wijerama (20241303) - Project Lead / Full-Stack / MLOps / Web Scraping

As Team Lead of the AutoInsight project, I oversaw the overall development process and ensured effective coordination across all system components. My contributions centered on machine learning, data management, web scraping, and MLOps practices. I was actively involved in gathering vehicle-related data from online sources through web scraping, which enabled the development of a comprehensive dataset for analysis and model training. This data was subsequently cleaned, structured, and prepared for integration into the system.

Furthermore, I contributed to the project's CRUD operations by managing the creation of records through the storage of scraped and processed training data in the database. I also handled the

retrieval of vehicle and model-related data required for machine learning training, evaluation, and prediction. To support continuous system improvement, I participated in updating the machine learning models through retraining whenever new data became available, thereby helping maintain the relevance and accuracy of predictions in response to changing market conditions. As part of the MLOps workflow, I helped integrate MongoDB into the machine learning pipeline to facilitate efficient data handling and model output management. I also took part in testing and validating the machine learning models to ensure dependable performance across diverse vehicle attributes. Through these efforts, I made a substantial contribution to the project's data acquisition, model lifecycle management, and operational deployment.

3.5.2 Dulan Nimnaka (20240503) - Frontend and Backend / Data Engineering & Deployment (Domain/VM)

For the full development cycle of AutoInsight, Dulan handled frontend development, backend integration, deployment infrastructure and search engine optimization. dulan worked in every layer of the application stack, gaining solid full-stack experience.

In particular, Dulan focused on building out the frontend filtering and UI/UX elements that enable users to search and filter vehicles easily. On the backend, Dulan designed and built out the core data pipeline that links the web scraping functionality to the frontend app. This was followed by building solid error handling where MongoDB will be the main storage and JSON would work as a fallback storage solution to always shave data even when failure in connectivity of MongoDB occurs. Dulan also handled the complete deployment infrastructure setting, deploying and configuring the backend API and frontend app on Azure Virtual Machines to make sure it scalably deployed in production.

Dulan learnt to create a responsive user friendly interface with complex filtering capabilities in this project from scratch. Dulan gained a deeper understanding of how data integrates from backend into frontend, especially regarding real-time data based on the web scraping operations. Graduated University. Dulan had experience dealing with databases, deployment strategies like MongoDB, error handling and fallback methods for cloud infrastructure management with

Microsoft Azure. Learning the entire process from data collection to user facing feature development has enhanced my full-stack knowledge. The work on SEO has enhanced Dulan's understanding of web performance optimization and search engine best practices. Understanding the complete flow from data collection to user-facing features to search engine visibility has strengthened my full-stack development capabilities.

Dulan would also like to see the marketplace listings on AutoInsight expanded in future projects, bringing users more extensive vehicle data. As an Azure expert, I also want to further develop my understanding of other more advanced Azure services and architectural patterns that ensure smooth scalability, performance optimisation, as well as advanced SEO techniques and analytics.

3.5.3 Shaveen Peiris (20240515) - Frontend Development

The main success of this project was designing the interactive user interface and building the custom visualization parts for the analytical dashboard. Making the visual analytics turned out to be much more difficult than we thought it would be. Shaveen made the price trajectory graphs from using native scalable vector graphics(SVG) that were directly controlled by react and TypeScript. This made the asset highly optimized and free of dependencies.

Also, improving the side by side vehicle comparison interface meant making many design changes to show the most important information without making the user feel overwhelmed. The main problem in this layer was making a formatting pipeline that smoothly turned raw backend integers into highly readable, localized string (for example, changing raw numbers to “8.41 M LKR”). Shaveen also added conditional rendering logic to the comparison grid so that it would automatically highlight the best metrics, such as the lower mileage or the highest validation confidence.

Looking back, a better state management system from the start. As the website got bigger and had to compare more than one vehicle at a time with the custom SVG charts, it got harder to find UI bugs and unnecessary re-renders. This experience really showed me how important it is to plan state carefully and use strict TypeScript definitions when making complex react websites.

3.5.4 Sanidu Samarasinghe (20240641) - Design and Performance of the Machine Learning Pipeline

Around the AutoInsight effort, Sanidu's role shaped how the system predicted car prices using machine learning. From messy inputs to clean signals, each step fed into smarter estimates down the line. Cleaning raw details came first, then shaping useful traits from them followed close behind. Training the model unfolded after that, letting patterns emerge through repeated runs. Outcomes took form as forecasted values, ready for real-world use in the last stage. Starting from scratch, Sanidu built a system using clear rules to sort vehicles into three categories - brand new, recondition, or used by checking how old they were and how far they had been driven. Then added a new column called condition, which got tested thoroughly on every record. To shape the data better for predictions, key traits like time-driven value loss, mileage transformed through logarithmic scaling, along with average benchmarks grouped by make and model - all tuned to sharpen forecast results.

Not far behind came linear regression, yet LightGBM edged ahead. Instead of blending in it stood out among decision trees and random forests. It provided R^2 as 0.9394 when tested. For AutoInsight's car pricing challenge, this fit mattered most. One step at a time, the project taught me how to put together an actual machine learning machine learning workflow starting with gathering data and ending with forecasts.

3.5.5 Sanara Perera (20240773) - Backend API, Data Cleaning & ML Testing

As the backend API developer responsible for FastAPI, working on the data cleaning pipeline, ML Evaluation Notebook and CI/CD (Continuous Integration/Continuous Deployment) Pipelines across the AutoInsight Project, Sanara delivered Create, Read, Update, Delete (CRUD) operations.

Sanara's primary task for the Create operation was developing a Data Cleaning Pipeline to generate a pre-processed vehicle dataset that would become the input for Machine Learning (ML) Model Training. Sanara also configured the CI/CD Pipeline within GitHub Actions to allow for automated testing and deployment workflows on each push to the main branch.

When implementing Read operations, Sanara built multiple FastAPI endpoints such as GET /vehicles/stats, GET /vehicles/filter, GET /vehicles/sort, and GET /vehicles/districts. Sanara also ensured that the load_vehicles() function combined two CSV data sources (using proper null handling and pagination) from the data cleaning pipeline.

Finally, Sanara created the functionality to remove records during the Load_Vehicles() process (to clean out any invalid price records, null price records or price records greater than 30 million LKR). This resulted in a clean data source for use by the Front End. Data cleansing was accomplished with the use of the `pd.to_numeric()` function with error coercion.

In hindsight, setting up the CI/CD pipeline sooner would have helped to identify integration issues earlier in the process. Overall, Sanara gained hands-on experience from this project in:

3.5.6 Hasandi Peiris (20240642) - Frontend-Backend Integration & Search Filtering.

The key milestones attained during the AutoInsight project were connecting the FastAPI backend to the React frontend, integrating the MongoDB database, and implementing dynamic search filtering for the website. The first phase of development involved creating the foundational `main.py` FastAPI backend and building core API endpoints to allow for accessing vehicle data from the frontend.

Hasandi established a MongoDB Atlas Database and created an import pipeline (`import_to_mongo.py`) that would successfully import 68,045 vehicle records from multiple separate CSV sources and place them into the `autoinsight` database collection. This operation produced a database solution that is representative of the previous CSV methods, but is scalable. vehicle data from MongoDB and providing that data back to the React frontend via an API request. Each of the endpoints has the capacity to filter data based on a variety of attributes, including make, model, condition, district, price range, mileage range, and year range. The Make and Model dropdown filters in `CarMarketplace.tsx` were connected to the backend live API. This was accomplished by adding the `getAllMakesFromAPI()` and `getModelsFromAPI()` functions into `vehicleDataService.ts`.

For the Update CRUD operation, Hasandi connected the live `riyasewana.com` scraper to the MongoDB pipeline and imported 18,619 free vehicle listings into the MongoDB database, allowing the database to remain current with the latest vehicle market listings.

3.6 Chapter Summary

In this chapter, a comprehensive evaluation of the AutoInsight platform was presented using both quantitative and qualitative approaches. The quantitative evaluation of the AutoInsight platform confirmed its compliance with all defined performance benchmarks, including a dashboard load time of 1.8 seconds, API response time of 310 milliseconds, and ML prediction response time of 1.2 seconds. The XGBoost price prediction model also recorded a high R^2 score of 0.87 and Mean Absolute Error of LKR 142,300 on the held-out validation set, surpassing all baseline models.

The qualitative evaluation, based on task-based usability testing of five participants and semi-structured interviews of domain and industry experts, indicated high levels of user satisfaction regarding the usability of the system, the accuracy of the data provided, and the relevance of the features provided by the system. Five key themes emerged from the qualitative results: Usability & Navigation, Data Accuracy Confidence, Feature Relevance, Performance & Speed, The self-evaluation provided by each member of the individual groups shows a clear collaborative approach to the development of the project, including high levels of technical contribution, development based on feedback from the testing phase, and learning outcomes. The results of the individual evaluations show conclusively that the AutoInsight prototype meets its value proposition of providing accessible data-driven vehicle market insights to the Sri Lankan automotive industry.

Chapter 4: Conclusion

4.1 Chapter Overview

This chapter provides a conclusion to the AutoInsight project. A summary is given of how well the AutoInsight project was achieved, how the aims and objectives were achieved, deviations and limitations of the work carried out, and future enhancements of the AutoInsight project. Included in this chapter are details of additional work completed during the project period and final remarks regarding the value and outcome of this research and its implementation. The purpose of this chapter is to provide an overall evaluation of the AutoInsight project as a

complete software application and to reflect on its contribution to solving problems in the Sri Lankan vehicle market.

4.2 Achievements of aims and objectives

AutoInsight is a completed project designed to create an intelligent vehicle analysis and marketing platform, with the objective of addressing the void of data-driven decision making for users of the Sri Lankan automotive industry. The primary goal of AutoInsight was to provide users with the ability to make more informed decisions about vehicles by providing a platform that contains market data, visual analytics, and machine learning-based price estimation capabilities all in one location. Based on the systems created, tested and evaluated in the previous sections, the author feels that AutoInsight has achieved the desired level of success.

The goals and objectives established in CW1 included creating a working prototype capable of, collecting vehicle data, processing and cleansing the data, storing the data in a manner that is easy to query from an organised database, generating meaningful visualisations in order to provide users with information about the automotive industry, and providing users fair market price estimates for automotive products, through machine learning. All of these goals have been realised through the successful implementation of the scraping pipeline, the data cleansing processes, the database structure, the web-based Dashboard user interface, the price estimation engines, and the comparative pricing feature. In addition to realising the goals and objectives, the AutoInsight project successfully moved from a conceptual stage to a market-ready product by successfully deploying the product in a production environment.

From a technical perspective, there were a number of technical achievements within the AutoInsight project. The AutoInsight system demonstrated success in the collection of vehicle data through web scraping.

4.3 Deviations

There were many factors and circumstances that affect whether or not an original strategy can meet its original intended goals. There were several different factors affecting the ultimate development and implementation of this project over the course of several iterations throughout development, yet none of the changes made were without purpose; all of the changes made were intended to improve technical feasibility, maintain stability, and ensure that there was valuable maximum value delivered by each component of the final product within the development time constraints.

One of the primary examples of variance from original scope was the refinement of the overall system capabilities to include the most impactful features only. The original functional requirements for the system were much broader than the constraints of the end product's functional capability. This refinement of the overall functional objectives allowed for the best use of resources during development, as all of the "core" features were developed first, as they were most closely related to the overall purpose of the project (i.e., performing market analysis, developing price predictions, filtering, comparison, and developing interaction with the end users).

Another significant divergence from the original architecture was the adopted data model for the various components of the system and the method of deployment and usage by the end user. As the project was developed, it became evident that some components needed to be re-developed to better support the application integration process and machine-learning pipeline. Some major changes to the original design approach and implementation of real-time prediction workflows, live-data handling, and hosting of end user applications were required based on changes made during the development of the various integrated components that were being placed into production. Therefore, the machine-learned models themselves had to be modified, as well, based on their actual implementation.

Finally, due to the changes in components described above, changes were also made to the system's overall functionality of the original enhancements planned. Following is a list of those changes made throughout the design, development, and implementation phases of the project.

4.4 Limitations of the project

Several limitations impacted the overall capacity and extensiveness of the final system despite its successful completion. The acknowledgments of these limitations are necessary as their inclusion affects both the functionality of the prototype as well as the potential for future expansion.

The vehicle dataset used during the project is one limitation that significantly impacted prototyping performance. Despite the successful harvesting of vehicle data, the model relies heavily on the quality, consistency, and precision of the listings collected from remote sites. Given the potential for missing entries, poor or varied terminology in each entry, identical advertisements appearing multiple times and frequent changes to the price of the product being advertised, the final dataset will likely not accurately represent the market in every instance. The impact on the quality of the dataset also has a direct effect on prediction quality as the strength of the machine learning model's predictions is directly correlated to the quality of the information it is trained with.

The project also represents a prototype level deployment rather than a complete production implementation. Therefore, certain functional areas including long-term scalability, security enhancement levels and total concurrent user capabilities were not thoroughly evaluated and were only analysed within the confines of the project timeline. The existing deployed implementation performs satisfactorily under the conditions tested; however, further enhancement is required prior to an expansion to a larger number of actual end users in the real world.

4.5 Future enhancements

The AutoInsight platform is capable of being expanded and will have many additional enhancements that will improve the system's technical capabilities and practical use.

One significant enhancement for the AutoInsight platform would be to expand the data sources. The inclusion of many more historical and real-time vehicle data sets will provide greater depth of analysis and support more accurate machine learning predictions. Integrating data from other sources such as automotive insurance estimates, import data, auction trends and/or more general vehicle registration statistics will further enhance the machine learning prediction accuracy.

Another improvement could come from the development of a more efficient and effective machine learning pipeline. While the current prototype machine learning model performs adequately, future work could include conducting model comparison experiments, implementing an automated retraining workflow, improving feature engineering and applying explainable AI techniques to assist users with understanding how and why a certain predicted price was generated. Improvements in this area would assist with increasing model transparency and enhancing end-user trust.

The AutoInsight platform would also benefit from adding many user-friendly advanced features that would enhance the overall user experience. Examples of this could be personalized user dashboards, improved price alerting systems, saved search history, recommenders, market trend reports by region, and improvements to the comparison tools. The mobile user experience also has the opportunity to be enhanced through either optimizing the interaction or by creating a dedicated mobile application.

4.6 Extra work (Competitions, research papers, etc)

During the SDGP timeframe, extra work that the team or individual members may have completed, separate from the core system implementation, can be logged into this section (e.g., participating in competitions, hackathons, exhibitions, conferences, submitting research papers, making poster presentations, doing public demonstrations, promoting on social media, and engaging with the industry as it relates to AutoInsight).

When applicable, evidence of these activities must be included in the Appendix (where applicable) in the form of certificates, screenshot images, Acceptance Emails, Event Photos,

LinkedIn Posts, Published Posts, Submission Confirmation(s), etc. If an individual member of the team participated in an activity that is applicable to their path along the SDGP journey, it must also be recorded along with an explanation of the relation of this activity to SDGP.

For the current project, engagement via the public marketing website, LinkedIn, and through contact with the industry can be considered additional work if proof is available. Other hackathons, conferences, or published works completed by team members should be listed in final report format once proof is attached in the appendix.

List down and provide proof of all the competitions/conferences/hackathons you or your team has taken part in during the SDGP time. If one person in a team has even participated individually, please add proof here

4.7 Concluding remarks

Overall, AutoInsight is a successful implementation of a data driven vehicle market analytics platform addressing a legitimate real-world problem in the Sri Lankan automotive environment. This project brought together all the components of data collection, preprocessing, database, full stack development, visual analytics, machine learning and deployment to provide a complete solution. Through this work, AutoInsight demonstrated that vehicle listings can produce insightful information to aid in making better decisions for buyers, sellers, and any other interested party.

The project provided both academic and practical outcomes. The academic success of the project demonstrated the ability to conduct an entire group project in software engineering, data science, and evaluation methodology. The practical success of the project demonstrated a working prototype of AutoInsight that was tested, evaluated, and received positive feedback from users and stakeholders. Despite some limitations and deviations from the project, the final results remained strong and closely aligned with the original project objectives.

Therefore, AutoInsight is a successful implementation of the objectives established at the design stage and represents a good opportunity for further development and real-world deployment.

References

CircleCI (2023) *Testing a PyTorch machine learning model with pytest*. Available at: <https://circleci.com/blog/testing-pytorch-model-with-pytest/> (Accessed: 23 March 2026).

Hoop.dev (2025a) *How to use Hugging Face and PyTest for reliable, automated model validation*. Available at: <https://hoop.dev/blog/how-to-use-hugging-face-and-pytest-for-reliable-automated-model-validation/> (Accessed: 23 March 2026).

Krekel, H. et al. (2004) *pytest* (version 8.3). Available at: <https://github.com/pytest-dev/pytest> (Accessed: 23 March 2026).

pytest developers (2026) *pytest documentation*. Available at: <https://docs.pytest.org/en/stable/> (Accessed: 23 March 2026).

Tirthajyoti (2017) *Machine-Learning-with-Python/Pytest/test_linear_model.py*. GitHub repository file. Available at: https://github.com/tirthajyoti/Machine-Learning-with-Python/blob/master/Pytest/test_linear_model.py (Accessed: 23 March 2026).

Chen, T. and Guestrin, C. (2016) ‘XGBoost: A Scalable Tree Boosting System’, *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794. Available at: <https://arxiv.org/abs/1603.02754> (Accessed: 23 March 2026).

XGBoost Developers (2026) *XGBoost Documentation*. Available at: <https://xgboost.readthedocs.io/en/stable/> (Accessed: 23 March 2026).

IBM (2024) *What is XGBoost?*. Available at: <https://www.ibm.com/think/topics/xgboost> (Accessed: 23 March 2026)

Ke et al. (2017) ‘LightGBM: A Highly Efficient Gradient Boosting Decision Tree’, *Advances in Neural Information Processing Systems*, 30, pp. 3146–3154. Available at: <https://dl.acm.org/doi/10.5555/3294996.3295074> (Accessed: 23 March 2026)

LightGBM Developers (2026) *LightGBM Documentation*. Available at: <https://lightgbm.readthedocs.io/en/latest/> (Accessed: 23 March 2026).

Breiman, L. (2001) ‘Random forests’, *Machine Learning*, 45(1), pp. 5–32. doi:10.1023/A:1010933404324

scikit-learn Developers (2026) *RandomForestRegressor*. Available at: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html> (Accessed: 23 March 2026).

IBM (2021) *What is Random Forest?*. Available at: <https://www.ibm.com/think/topics/random-forest> (Accessed: 23 March 2026).

Appendix

Each section under the appendix should be named “Appendix A”, “Appendix B” etc. Continue the numbering sequence from the reference section.

Appendix Section A : Feedback Letters

Appendix Section A.1

22nd March 2026,

To Whom It May Concern,

This project shows strong potential as a practical vehicle analytics and market intelligence solution for the modern automotive industry. The system, titled AutoInsight, effectively addresses a key gap in the Sri Lankan vehicle market by providing data-driven insights into vehicle pricing, trends, and comparisons.

The platform integrates multiple functionalities including interactive dashboards, vehicle comparison tools, price prediction using machine learning, and real-time market analysis. By leveraging data collected from leading vehicle marketplaces and combining it with advanced analytics, the system enables users to make informed decisions rather than relying on assumptions.

The use of a full-stack architecture with a modern frontend, backend services, and database support reflects strong software engineering practices. Additionally, the incorporation of machine learning models for price prediction and trend analysis demonstrates a high level of technical capability and innovation. Features such as filtering, visualization dashboards, and user-friendly interfaces make the system highly relevant for real-world applications.

With further improvements in scalability, data expansion, and production readiness, this project has strong potential to be developed into a reliable industry-level automotive analytics platform. Overall, the team has demonstrated good technical knowledge, problem-solving ability, and a clear understanding of real-world requirements.

Thank you.

Sincerely,



Nimal Peiris
Technical Lead
Sysco LABS Sri Lanka
nimal.peiris@sysco.com

Appendix Section A.2



rohan@cmta.lk
to me ▾

Tue 3 Mar, 07:38 ☆ 😊 ↩ ⋮

Thank you very much Akila for sharing the prototype document. Read through it looks comprehensive. Well Done as I can see a lot of hard work has been done.

Always willing to assist whenever someone is pursuing his studies.

Kindly share the final document.

Best Regards

Rohan Casiechetty

General Manager

Ceylon Motor Traders' Association |

c/o The Ceylon Chamber of Commerce, 50, Nawam Mawatha, Colombo 02, Sri Lanka.

Tel: +94 777 745 095 | Fax: +94 11 2449352 |

E-mail: rohan@cmta.lk | Web: [Ceylon Motor Traders Association \(cmta.lk\)](http://Ceylon Motor Traders Association (cmta.lk))

